

KAMP 제조AI데이터셋 활용 권한

No. GB20260928-PU1786798671-383574



⚠ 제조AI데이터셋 활용 주의사항

연구/공식적으로 사용하실 때에는 꼭 아래와 같이 'KAMP 출처(reference)'를 남겨주시고, 인용 시 활용한 내용과 문서 등은 아래 이메일로 보내주시기를 바랍니다.
(E-mail : kamp@kaist.ac.kr)

국문 출처 표기 양식

중소벤처기업부, Korea AI Manufacturing Platform(KAMP), 자원 최적화 AI 데이터셋, KAIST(울산과학기술원, (주)유피시앤에스), 2021.12.27., www.kamp-ai.kr

영문 출처 표기 양식

Ministry of SMEs and Startups., and KAIST (Korea Advanced Institute of Science and Technology). (2021, December 27). Resource optimization AI dataset. Korea AI Manufacturing Platform(KAMP). <https://www.kamp-ai.kr/>

제조AI데이터셋 개요

제조AI데이터셋명 : 자원 최적화 AI 데이터셋

업종 : 기타 | **목적** : 자원 최적화 | **유형** : csv

알고리즘 : Random Forest, Deep Neural Network

사용조건 : 콘텐츠 변경허용

제공기관 : KAIST (수행기관: 울산과학기술원/(주)유피시앤에스)

등록일 : 2021-12-27

이용자 정보

회원 ID : MSS_swoo64

다운로드일 : 2026-09-28 21:09:14

활용목적 : 제조AI 활용 참조모델 확인

인공지능 제조 플랫폼(KAMP, Korea AI Manufacturing Platform)에서 다운로드한 제조AI데이터셋의 저작권을 위하여 문서 고유 번호와 워터마크를 부여하여, 해당 문서의 표준을 준수하였음을 증명합니다.

운영기관 : KAIST 제조AI빅데이터센터

주소 : 대전광역시 유성구 문지로 193 KAIST 문지캠퍼스 행정동

전화번호 : 042-350-1331 | 이메일 : kamp@kaist.ac.kr

「자원 최적화 AI 데이터셋」 분석실습 가이드북



「자원 최적화 AI 데이터셋」 분석실습 가이드북



Contents

1 분석요약	04
2 분석 실습	
1. 분석 개요	05
1.1 분석 배경	05
1) 공정(설비) 개요	
2) 이슈사항(Pain point)	
1.2 분석 목표	06
1) 분석 목표	
2) 데이터 정의 및 소개	
3) 데이터 분석 기대효과	
4) 시사점(implication) 요약기술	
2. 분석실습	08
2.1 제조데이터 소개	08
1) 데이터 수집 방법	
2) 데이터 유형/구조	
3) 데이터 (품질) 전처리	
2.2 분석 모델 소개	15
1) 데이터 흐름 및 인공지능 모델 적용 흐름도	
2) AI 분석모델	
2.3 분석 체험	18
1) 필요 SW, 패키지 설치 방법 및 절차 가이드	
2) 분석 단계별 프로세스 - 순환 신경망	
3) 분석 단계별 프로세스 - 랜덤 포레스트	
3. 유사 타현장의 「자원 최적화 AI 데이터셋」 분석 적용	63
부 록	
분석환경 구축을 위한 설치 가이드	65
데이터 품질 전처리 [실습 코드]	77

「자원 최적화 AI 데이터셋」 분석실습 가이드북

- 필요 SW : Python, Anaconda – Jupyter Notebook
- 필요 패키지 : Tensorflow, Keras, Pandas, Ortools, Sklearn, Matplotlib
- 분석 환경 : [운영체제] Window 10이상, Ubuntu 14.0 이상, [CPU] Intel Xeon E5-1650 v4 3.60GHz, [RAM] 128GB, [GPU] Nvidia Geforce 1080Ti
- 필요 데이터 : okm_augumented_2021.csv

1 분석요약

No	구분		내용
1	분석 목적 (현장 이슈, 목적)		한국전력공사는 제조현장에서의 3개월 간의 최대 피크 전력을 기준으로 1년간의 전기료를 부과하고 있으며, 제조공정에서의 의도치 않은 최대 피크 전력 발생은 전기료 증가로 인한 제조비용을 증가시키는 요인으로 작용한다. 따라서, 본 가이드북은 관련 데이터셋과 AI 모델을 통해 실제 제조공정에서 발생할 수 있는 최대 피크 전력을 낮추는 것을 목적으로 한다. 해당 목적을 달성하기 위해 주요 변수 간의 상관관계 분석을 통해 다양한 머신러닝 방법과 알고리즘을 활용한 에너지 사용량 예측 모델을 개발하여 전력 자원 최적화를 달성한다.
2	데이터셋 형태 및 수집방법 (csv, json, image 파일 등)		1) 분석에 사용된 변수 : 날짜, 시간, 15분, 30분, 45분, 60분, 평균, 생산량, 기온, 풍속, 습도, 강수량, 전기요금(계절), day, d, m, 공장인원, 인건비 2) 데이터 수집 방법 : 공장별 변압기 감지센서를 활용한 데이터 수집 (15분 단위) 3) 데이터셋 파일 확장자 : CSV
3	데이터 개수 데이터셋 총량		- 데이터 개수 : 3111,024개 - 데이터셋 총량 : 총 427KB
4	분석적용 알고리즘	알고리즘	Random Forest, Deep Neural Network
		알고리즘 간략소개	Random Forest : 분류를 위한 다수의 결정 트리(Decision Tree)를 구성하고, 이를 통해 앙상블(ensemble)된 결과를 사용하는 트리 기반(가지가 뻗은 나무 모양)의 모델이다. 이런 결정 트리에서는, 트리의 깊이, 잎 개수 등의 파라미터를 조절할 수 있다. 본 가이드북에서는 제조공정에 영향을 미치는 다양한 변수들을 사용하여 전력량을 예측하는 회귀(regression) 방식의 트리들을 생성하여 전력 사용량을 예측한다. Deep Neural Network : 분류를 위해 다수의 신경망을 연속적으로 쌓고, 최종적으로 각 클래스별 확률을 출력하도록 각 신경망의 모수를 업데이트하는 모델이다. 본 가이드 북에서는 연속적으로 쌓은 레이어(layer)를 통해 제조 공정 데이터에 관한 특징을 추출하고, 최종 레이어에서 전력 사용량을 예측한다.
5	분석결과 및 시사점		제조공정에서의 피크 전력량을 낮추기 위해 관련 생산 데이터를 수집하고, 분석하여 에너지 사용량 예측 모델을 개발하였다. 전력 사용에 영향을 미치는 생산량 등과 연계하여 제조공정 현장에서의 에너지 사용량 예측에 널리 적용될 것으로 판단된다.

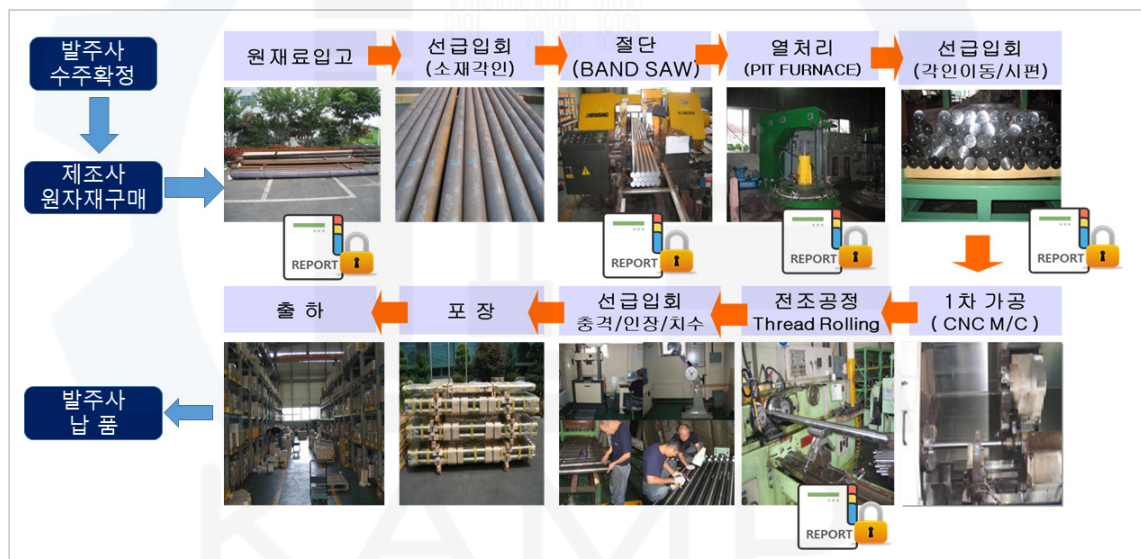
2 분석 실습

1. 분석 개요

1.1 분석 배경

1) 공정(설비) 개요

- 선박 엔진에 들어가는 볼트 · 너트의 제조공정은 원재료인 환봉의 입고에서부터 시작된다.
- 입고된 환봉은 소재 각인, 절단, 열처리, 각인 이동, 1차 가공, 전조공정, 선급입회의 단계를 거쳐 최종제품인 선박엔진용 볼트 · 너트로 생산이 이루어진다.
- 생산된 최종제품은 포장 및 출하 과정을 거쳐 납품 계획에 따라 고객사에 납품이 이루어진다.
- 이러한 일련의 과정에서 공장 내 인력, 설비, 전기 자원 등 다양한 자원이 소모되며 이러한 자원을 효율적으로 사용하는 것은 필수불가결한 요소이다.



[그림 1] 선박용 볼트·너트 제조공정 프로세스

2) 이슈사항(Pain point)

• 공정(설비)상의 문제 현황

- 선박엔진용 볼트 · 너트 제조공정은 입고에서부터 단조, 열처리, 가공 및 공조설비, 포장, 출하에 이르기까지 다양한 전력에너지를 소비한다. 제조공정에서의 전력에너지 소비는 기업의 제조원가에 막대한 영향을 미친다.
- 제조공장의 경우, 한국전력에서는 최대수요전력을 바탕으로 전기료를 산정한다. 최대 수요전력이란 공장 내 설치된 변압기를 통해 일정 시간 단위로 누적 계산되는 전력을

의미한다. 일정 시간 단위로 계산되는 이유는 설비 고장, 순간적인 대용량 설비의 가동 등 어느 한 시점의 최대 전력을 기준으로 부과 시 사용자에게 불합리한 비용을 청구할 수 있기에 일정 시간을 정하여 그 시간 동안의 평균전력으로 최대수요전력을 계산한다. 한국전력에서는 15분 단위로 해당 최대수요전력을 계산한다.

- 한국전력에서는 3개월 최대수요전력 패턴을 모니터링하여 가장 높은 최대수요전력을 기준으로 향후 1년간의 기본요금을 적용하여 전기료를 부과한다.
- 그렇기에 해당 기업의 입장에서는 최대수요전력을 효율적으로 관리하는 것이 중요하다. 공장을 운영하는 과정에서 일시적인 과부하로 인하여 최대수요전력이 증가하게 된다면 불필요한 전기요금이 발생하고, 이것은 운영 비용 상승을 초래하며 기업의 원가상승 요인으로 작용한다.

• 문제해결 장애요인

- 최대수요전력을 실시간으로 효과적으로 관리하기가 힘들며, 전력 사용에 영향을 미치는 내부 및 외부 요인이 복합적으로 작용한다.
- 또한, 변압기에 달린 센서를 통해 15분 단위로 최대수요전력을 확인할 수 있지만, 매번 확인하고 관리하기 위한 인력이 추가로 투입되어야 하기에 운영 비용 상승을 초래한다.

• 극복 방안

- 제조공정 상의 최대수요전력을 사전에 예측할 수 있다면 해당 전력 사용량을 효과적으로 관리할 수 있으며, 운영 비용을 절감시킬 수 있다.
- 실제 공정상의 최대수요전력에 영향을 미치는 다양한 변수(생산량, 온도, 습도, 강수량 등)들의 관계를 분석하여 회귀 모델, 머신러닝, 순환 신경망 기법을 적용하여 최대수요전력을 예측하는 모델을 개발하고자 한다.

1.2 분석 목표

1) 분석 목표

- 제조공정 상에서 발생하는 15분 간격의 최대수요전력과 생산량, 온도, 습도 등의 데이터를 수집 및 분석한다. 데이터 분석을 통하여 변수 간의 상관관계를 찾는다. 이 중 상관관계가 높은 변수들을 활용하여 다양한 머신러닝 및 순환신경망 기법을 활용하여 최대수요전력을 예측하는 모델을 만들고자 한다.

2) 데이터 정의 및 소개

- 독립변수란, 다른 변수에 영향을 받지 않는 변수로 입력값이나 원인을 의미한다.
- 종속변수란, 독립변수의 변화에 따라 어떻게 변화하는지를 알고 싶은 변수로, 결과물이나 효과를 의미한다.

공정 변수 조건	내용
독립변수	날짜
	시간
	생산량
	기온
	풍속
	습도
	강수량
	전기요금(계절)
	day
	d
	m
	공장직원
종속변수	인건비
	15분, 30분, 45분, 60분, 평균

[표 1] 제조데이터 독립 및 종속변수 정의

3) 데이터 분석 기대효과

- 제조공정 상에서 최대수요전력에 영향을 미치는 다양한 변수들의 상관관계를 분석하여 최대수요전력에 영향을 미치는 변수를 선정 후 시간대별 최대수요전력을 예측할 수 있는 모델을 제공한다.

4) 시사점(implication) 요약기술

- 현재 제조공정 상의 최대수요전력은 실제 최대수요전력이 발생한 후 15분 단위 이후에 확인할 수 있다. 또한, 해당 최대수요전력 발생에는 다양한 요인이 영향을 미치지만, 어떠한 요인이 해당 최대수요전력 발생에 가장 많은 영향을 끼치는지 파악할 수 없었다. 하여, 최대수요전력 발생에 영향을 미치는 다양한 변수를 파악하고 해당 전력을 예측할 수 있는 AI 모델을 구축하고 개발한다.
- 본 분석은, 제조공정 내의 15분 단위별 최대수요전력 데이터를 수집한다. 또한, 해당 일자의 생산량, 온도, 습도, 풍향, 강수량, 전기요금, 작업자 수 등의 데이터를 추가로

수집한다. 데이터를 수집 후, 가공/전처리, AI 모델 개발과 제조공정 상에서의 적용 및 검증 등을 통해 중소기업에서 데이터 및 AI를 적용하여 실질적인 최대수요전력을 사전에 예측하여 관련된 비용 절감에 기여할 수 있다는 점에서 시사하는 바가 크다고 판단된다.

2. 분석 실습

2.1 제조데이터 소개

1) 데이터 수집 방법

- **제조 분야** : 선박 엔진용 볼트·너트
- **제조 공정명** : 자재 입고에서 제품 출하까지의 전 제조공정
- **수집 장비** : 공장 내 변압기 센서 데이터, 기상청 데이터
- **수집 기간** : 2021년 1월 1일 ~ 2021년 9월 14일 (9개월)
- **수집주기** : 매 시간 15분 단위

2) 데이터 유형/구조

- **데이터셋 구조** : 테이블 형식(tabular)
- **데이터 개수** : column수 18개, row수 6,168개, 데이터수 111,024개
- **AI데이터셋 주요 변수 정의**

속성(column)	설명	비고
날짜	공장운영 날짜	int
시간	공장운영 시간	int
15분	0-15분까지의 피크전기 사용량	int
30분	15-30분까지의 피크전기 사용량	int
45분	30-45분까지의 피크전기 사용량	int
60분	45-60분까지의 피크전기 사용량	int
평균	15분,30분,45분,60분 속성의 평균값	int
생산량	해당 시점에 생산해야할 생산량	int
기온	해당 시점의 기온	float
풍속	해당 시점의 풍속	float
습도	해당 시점의 습도	int
강수량	해당 시점의 강수량	float
전기요금(계절)	해당 시점의 전기요금	float
day	해당시점의 요일의 값	int
d	해당 시점의 일의 값	int
m	해당 시점의 달의 값	int
공장직원	해당 시점에 공장이 보유한 생산력	float
인건비	야간 및 주간 인건비에 대한 비율값	float

[표 1] 데이터셋 주요 변수

• 주요 변수 기술 통계

속성(column)	데이터형	개수	평균	표준편차	최소값	중앙값	최대값
시간	int	6,168	12.428016	12.847309	0	12	188
15분	int	6,168	90.410182	55.349403	0	101	207
30분	int	6,168	92.695363	57.942122	0	104	222
45분	int	6,168	95.106355	59.285709	0	105	218
60분	int	6,168	95.037938	59.347554	0	107	214
평균	int	6,168	93.424125	57.355938	0	104	208
생산량	int	6,168	467.344682	857.571815	0	45	9830
기온	float	6,168	15.906064	9.160356	0	17.4	33.4
풍속	float	6,168	2.062630	1.164724	0	1.2	7.6
습도	int	6,168	70.098735	22.996164	8	74	98
강수량	float	6,168	2.243888	9.61275	0	0	122.4
전기요금(계절)	float	6,168	162.757198	30.820855	109.8	167.2	191.6
day	int	6,168	4.003891	2.006957	1	4	7
d	int	6,168	15.256809	8.799601	1	15	31
m	int	6,168	4.770428	2.452431	1	5	9
공장직원	float	6,168	0.898851	1.983335	0	0.111111	48.386364
인건비	float	6,168	1.313959	0.241700	1	1.5	1.5

[표 2] 주요 변수 기술 통계

• 비식별화된 원본 데이터

- 제조공정 상에서 발생하는 전력량을 공장 내에 설치된 변압기 센서 데이터에서 비식별화 및 품질 전처리를 수행한 데이터셋이다.
- 또한 전처리를 수행한 데이터셋에 추가로 ERP 프로그램에 입력된 일자별 생산량 데이터를 추가하였다.
- 각 일자별 시간대에 해당하는 날씨 관련 정보는 기상청에서 제공하는 데이터를 활용하여 추가하였다.

• 학습 통합 데이터 (labeled 데이터)

- ‘2.3.1 순환 신경망 활용 알고리즘’의 ‘[단계 ③] 데이터 정제 (전처리)’에서 확인할 수 있듯, 다양한 데이터가 혼합된 데이터셋에서 데이터 분석을 하고자 하는 변수에서 값이 없는 경우 (예를 들어, 해당 일자에 생산량이 없어서 빈칸으로 있는 경우)에는 해당 값을 0으로 변경하여 데이터셋을 완성한다.
- 데이터셋의 각 변수의 상관관계를 분석하여, 상관관계가 낮은 변수는 제거하여, 예측 학습 데이터에 사용하기 위한 변수만 남겨둔다.
- 순환 신경망 모델에 입력되는 데이터는 현재 데이터셋에 추가적으로 주말 (토요일 및 일요일), 법정 공휴일 (설 연휴, 광복절 등) 변수를 추가하여 학습 시 해당 휴일 정보를

- 반영하여 예측할 수 있도록 데이터를 추가한다.
- 또한 순환 신경망 모델을 활용하기 위하여 일주일 단위로 일자를 구분하여 데이터셋을 추가적으로 생성하여, 각 데이터셋을 학습 및 평가데이터를 구분하여 특정 일자의 최대수요전력을 예측한다.
- 랜덤 포레스트를 활용한 모델에서는 해당 일자의 전기요금, 해당 시간대의 인건비, 투입 인력, 주말 여부 등을 고려하여 최대수요전력 예측뿐만 아니라 주어진 조건을 활용하여 최적화 모델 또한 제공한다.

• 1차 가공 데이터

날짜	시간	15분	30분	45분	60분	평균	생산량
20210101	0	62	61	61	61	61	0
20210101	1	96	93	116	113	105	0
20210101	2	106	96	106	107	104	0
20210101	3	92	110	110	109	105	0
20210101	4	108	105	106	108	107	0
20210101	5	89	83	99	98	92	0
20210101	6	102	111	110	103	107	0
20210101	7	92	75	48	29	61	0
20210101	8	27	25	22	22	24	0

[그림 3] 초기 데이터 예시

- 1차 가공 데이터는 8가지 변수에 해당하는 데이터가 제공된다. 일자(날짜), 시간, 0분에서 15분 동안의 최대수요전력(15분), 15분에서 30분 동안의 최대수요전력(30분), 30분에서 45분 동안의 최대수요전력(45분), 45분에서 60분 동안의 최대전력수요(60분), 최대수요전력의 평균(평균), 각 시간대별 생산량(생산량), 총 8가지 데이터가 csv(column separated value)의 형태로 추출된다.
- 8가지 데이터는 변압기 센서 데이터와 ERP 프로그램에 등록된 생산량 데이터에서 추출하여 제공된다.

• 2차 가공 데이터

- 1차 가공 데이터에서 얻어진 데이터셋에 추가로 사용하는 기법 및 목적에 맞게 데이터를 추가하였다.
- 순환 신경망 모델에 사용하기 위하여 1차 가공 데이터의 데이터셋에 추가로 기상청에서 제공하는 기후 데이터의 4가지 데이터를 추가한다. 해당 일자별 및 시간대별 기온,

풍속, 습도, 강수량 등의 데이터를 제공한다.

- 또한, 해당 일자별 및 시간대별에 해당하는 전기요금은 한국전력에서 제공하는 데이터를 활용하여 제공한다.

날짜	시간	15분	30분	45분	60분	평균	생산량	기온	풍속	습도	강수량	전기요금	day	d	m	공장인원	인건비
20210101	0	62	61	61	61	61	0	-3.2	2.4	71	0	109.8	5	1	1	0	1.5
20210101	1	96	93	116	113	105	0	-4.5	1.5	77	0	109.8	5	1	1	0	1.5
20210101	2	106	96	106	107	104	0	-3.9	2.6	58	0	109.8	5	1	1	0	1.5
20210101	3	92	110	110	109	105	0	-4.1	2.6	56	0	109.8	5	1	1	0	1.5
20210101	4	108	105	106	108	107	0	-4.6	2.6	60	0	109.8	5	1	1	0	1.5
20210101	5	89	83	99	98	92	0	-5.5	2.1	66	0	109.8	5	1	1	0	1.5
20210101	6	102	111	110	103	107	0	-5.6	1.5	67	0	109.8	5	1	1	0	1.5
20210101	7	92	75	48	29	61	0	-6.8	0.6	73	0	109.8	5	1	1	0	1.5
20210101	8	27	25	22	22	24	0	-5.6	0	72	0	109.8	5	1	1	0	1.5
20210101	9	22	22	22	22	22	0	-3.3	1.2	62	0	109.8	5	1	1	0	1
20210101	10	22	22	22	22	22	0	-1.2	2.1	54	0	109.8	5	1	1	0	1
20210101	11	22	23	28	27	25	0	-0.1	3.5	50	0	109.8	5	1	1	0	1
20210101	12	22	23	22	22	22	0	1.5	2.8	45	0	109.8	5	1	1	0	1

[그림 4] okm_augumented_2021.csv 예시

- 랜덤 포레스트 모델에 사용하기 위하여 상기의 가공 데이터셋에 추가로 파이썬 라이브러리에서 제공하는 요일값(day), 해당 일자(d), 해당 월(m)을 변수로 추가한다.
- 또한, 최적화를 위한 주간 시간 대비 인건비 비율 (인건비), 해당 일자의 전기요금, 공장직원 수(공장직원) 등을 변수로 설정하여 데이터를 제공한다.
- 향후 해당 데이터에서 '날짜부터 ~ m'에 해당하는 Column의 데이터 값은 모델의 학습에 사용된다. 인건비, 전기요금, 공장직원의 Column은 랜덤 포레스트 모델이 예측한 해당 시점의 최대수요전력을 기반으로 주어진 생산량을 해결하기 위해 최적으로 필요한 직원의 수와 공장을 주간 또는 야간에 가동할지의 '여부'에 대한 값을 풀기 위한 변수들이다.

• 3차 가공 데이터

- 3차 가공 데이터는 2차 가공 데이터의 18가지 Column에서 상대적으로 상관관계가 높은 변수인 15분 단위 최대수요전력의 데이터를 기본으로 가공된 데이터이다.
- 순환 신경망 모델을 통해 최대수요전력을 예측하기 위해, 일주일 단위 (예를 들어, 1월 1일부터 7일까지 7일에 각 일자별 24시간 데이터를 곱한 168개)로 값을 구분한다.
- 3차 가공 데이터의 구성으로는 168개 Row 이후의 데이터부터 다시 시작하여, Column으로는 각 데이터셋명의 숫자만큼 (15분 단위 최대수요전력_t-1부터 t-168) Column이 생성되어 학습에 활용된다.

Date	Vacation	Weekend	Peak Consumption In 15 minute,t-1	Peak Consumption In 30 minute,t-1	Peak Consumption In 45 minute,t-1	Peak Consumption In 60 minute,t-1	Average Peak Consumption In 1 hour,t-1
2021-01-08 1:00	0	0	64	63	63	62	63
2021-01-08 2:00	0	0	87	94	105	110	99
2021-01-08 3:00	0	0	104	98	98	111	102.75
2021-01-08 4:00	0	0	90	105	99	104	99.5
2021-01-08 5:00	0	0	105	105	100	110	105
2021-01-08 6:00	0	0	82	72	94	101	87.25
2021-01-08 7:00	0	0	109	104	96	81	97.5
2021-01-08 8:00	0	0	72	77	107	117	93.25
2021-01-08 9:00	0	0	141	154	161	160	154
2021-01-08 10:00	0	0	165	160	160	156	160.25
2021-01-08 11:00	0	0	127	153	138	150	142
2021-01-08 12:00	0	0	159	160	167	171	164.25
2021-01-08 13:00	0	0	106	87	94	124	102.75
2021-01-08 14:00	0	0	157	174	168	160	164.75
2021-01-08 15:00	0	0	156	164	167	176	165.75
2021-01-08 16:00	0	0	139	173	153	180	161.25
2021-01-08 17:00	0	0	150	166	164	152	158
2021-01-08 18:00	0	0	127	110	147	161	136.25
2021-01-08 19:00	0	0	150	156	152	157	153.75
2021-01-08 20:00	0	0	155	144	140	128	141.75
2021-01-08 21:00	0	0	134	132	117	132	128.75
2021-01-08 22:00	0	0	132	123	116	114	121.25
2021-01-08 23:00	0	0	91	109	115	113	107
2021-01-09 0:00	0	1	105	114	104	82	101.25

[그림 5] daily_data_seasonality_new_168.csv 예시

3) 데이터 (품질) 전처리

• 데이터 품질 전처리 목적

- 실제 공정에서 발생하는 데이터들은 값이 의미 없거나 누락 및 오타가 발생하여 품질이 떨어진다.
- 아무리 좋은 분석 방법론을 가지고 있더라도 품질이 낮은 데이터를 이용하면 좋은 결과를 얻을 수 없다.
- 그렇기에 데이터 (품질) 전처리는 데이터 분석에 있어서 필수적인 단계이다,
- 따라서 데이터들의 6가지 품질 지수를 파악하고, 데이터 전처리를 통해 품질 지수를 향상한다.

• 데이터 품질 지수

- 완전성(Completeness) : 필수항목에 누락이 없어야 한다.
- 유일성(Uniqueness) : 데이터 항목은 유일해야 하며 중복되어서는 안 된다.
- 유효성(Validity) : 데이터 항목은 정해진 데이터 유효범위 및 도메인을 충족해야 한다.
- 일관성(Consistency) : 데이터가 지켜야 할 구조, 값, 표현되는 형태가 일관되게 정의되고, 서로 일치해야 한다.
- 정확성(Accuracy) : 실제 존재하는 객체의 표현 값이 정확하게 반영이 되어야 한다.
- 무결성(Integrity) : 데이터베이스 자료의 오류 없이 변화에 영향을 받지 않고 데이터의 정확성과 일관성, 유효성이 보호되어야 한다.

날짜	시간	15분	30분	45분	60분	평균	생산량	온도	풍속	습도	강수량	전기요금
20210101	0	62	61	61	61	61.25		-3.2	2.4	71	0	109.8
20210101	1	96	93	116	113	104.5		-4.5	1.5	77	0	109.8
20210101	2	106	96	106	107	103.75		-3.9	2.6	58	0	109.8
20210101	3	92	110	110	109	105.25		-4.1	2.6	56	0	109.8
20210101	4	108	105	106	108	106.75		-4.6	2.6	60	0	109.8
20210101	5	89	83	99	98	92.25		-5.5	2.1	66	0	109.8
20210101	6	102	111	110	103	106.5		-5.6	1.5	67	0	109.8
20210101	7	92	75	48	29	61		-6.8	0.6	73	0	109.8
20210101	8	27	25	22	22	24		-5.6	0	72	0	109.8
20210101	9	22	22	22	22	22		-3.3	1.2	62	0	109.8
20210101	10	22	22	22	22	22		-1.2	2.1	54	0	109.8
20210101	11	22	23	28	27	25		-0.1	3.5	50	0	109.8
20210101	12	22	23	22	22	22.25		1.5	2.8	45	0	109.8
20210101	13	22	23	22	22	22.25		2.6	3.5	44	0	109.8
20210101	14	22	23	22	23	22.5		2.6	3.3	43	0	109.8
20210101	15	28	26	23	22	24.75		3.5	3	42	0	109.8
20210101	16	23	23	22	23	22.75		3	3.1	43	0	109.8
20210101	17	22	22	22	23	22.25		2.2	1.6	50	0	109.8
20210101	18	22	23	27	27	24.75		1.1	1.2	55	0	109.8
20210101	19	22	22	22	22	22		1.3	1.1	55	0	109.8
20210101	20	22	22	22	22	22		1.2	1.8	57	0	109.8
20210101	21	23	22	22	27	23.5		0.9	1.6	59	0	109.8
20210101	22	26	22	22	22	23		-0.2	1.8	63	0	109.8
20210101	23	22	22	22	22	22		0	1.9	57	0	109.8

[그림 6] 데이터 품질 지수 완전성 위배 예시

→ Column에 null값이 들어가 있으므로 완전성을 위배한다.

날짜	시간	15분	30분	45분	60분	평균
20210101	0	62	61	61	61	61.25
20210101	1	96	93	116	113	104.5
20210101	2	106	96	106	107	103.75
1900-10-32	3	92	110	110	109	105.25
20210101	4	108	105	106	108	106.75
20210101	5	89	83	99	98	92.25
1900-10-34	6	102	111	110	103	106.5
120210101	7	92	75	48	29	61
1900-10-32	8	27	25	22	22	24
1	9	22	22	22	22	22
1900-10-32	10	22	22	22	22	22
20210101	11	22	23	28	27	25
3	12	22	23	22	22	22.25
1900-10-32	13	22	23	22	22	22.25
20210101	14	22	23	22	23	22.5
20210101	15	28	26	23	22	24.75
20210101	16	23	23	22	23	22.75

[그림 7] 데이터 품질 지수 유효성 위배 예시

→ 날짜 데이터에 유효하지 않은 날짜형식이 들어 있으므로 유효성을 위배한다.

• 데이터 품질 지수 세부 설명

- 완전성 품질 지수 = $(1 - (\text{결측치} / \text{전체 데이터 수})) * 100$

- ① 결측치 값이 30% 이상인 데이터들은 데이터의 완전성이 떨어지기 때문에 열별 결측치 값의 비율을 확인하여 삭제한다.
- ② 데이터의 결측치를 확인하기 위해 'isnull()' 함수를 사용한 뒤 'sum()' 함수를 이용하여 총 결측치 개수를 구한다.
- ③ 구한 결측치의 개수를 이용하여 완전성 품질 지수를 구한다.

- 유일성 품질 지수 = $((\text{유일한 데이터 수} / \text{전체 데이터 수})) * 100$

- ① 이 데이터는 유일한 값을 가지는 열이 존재하지 않으므로 유일성을 판단하지 않는다.
- ② 유일성을 판단하고 싶다면 날짜와 시간 두 열을 이용하여 합성키를 만든 뒤 기본키로 설정하여 중복을 판단하면 된다.

- 유효성 품질 지수 = $(\text{유효성 만족 데이터 수} / \text{전체 데이터 수}) * 100$

- ① 데이터가 유효범위 내에 있는가?
- ② 데이터 형식에 맞는가?
- ③ 수집된 날짜 안에 들어가 있는가? 등을 검증하는 것이다.

- 일관성 품질 지수 = $(\text{일관성 만족 데이터 수} / \text{전체 데이터 수}) * 100$

- ① 데이터 테이블 간 종속관계 확인
- ② 데이터 형식에 맞는가?
- ③ 수집된 날짜 안에 들어가 있는가? 등을 검증하는 것이다.
데이터가 지켜야 할 구조, 값, 표현되는 형태가 일관되게 정의되고, 서로 일치해야 한다.

- 정확성 품질 지수 = $(1 - (\text{정확성 위배 데이터 수} / \text{전체 데이터 수})) * 100$

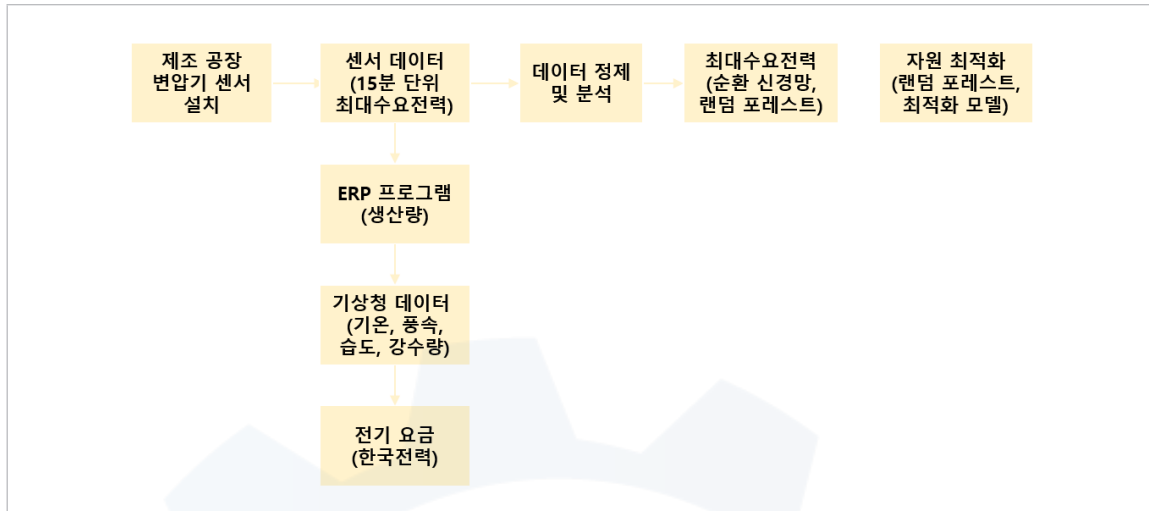
해당 최대수요전력 예측과 관련하여 중복되는 예측값은 존재할 수 있으나 누락이 발생해서는 안된다.

- 무결성 품질 지수 = $(1 - (\text{유일성, 유효성, 일관성 지수 중 100%가 아닌 지수 개수} / 3)) * 100$

무결성 품질 지수는 고려되는 모든 데이터가 '유일성', '유효성', '일관성' 지수를 만족하는지 확인하는 지수이다. 본 가이드북에서는 세 가지 지수 중 100% 품질을 담당하는 지수의 비율을 무결성 품질 지수로 정의한다.

2.2 분석 모델 소개

1) 데이터 흐름 및 인공지능 모델 적용 흐름도



[그림 8] AI 분석 모델 도식화

2) AI 분석모델

• 해당 AI 방법론(알고리즘) 선정 이유 기술

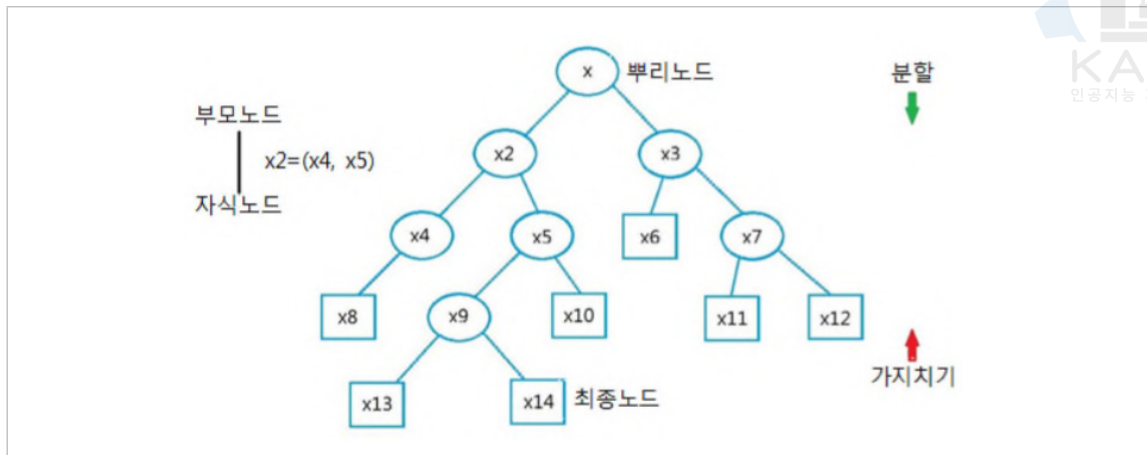
- 최대수요전력 데이터 분석 방법 : 랜덤 포레스트 (Random Foreset) & 순환 신경망 (Recurrent Neural Network)

- 랜덤 포레스트란 여러 개의 결정트리 (Decision Tree)를 이용하여 학습시키는 방법이다. 직관적인 머신러닝 알고리즘으로 분류와 회귀 모델 모두 사용할 수 있지만, 최대수요전력을 예측하기 위해 본 분석에서는 회귀 모델을 사용한다. 최대수요전력과 관련된 데이터 샘플링 시 일부 변수들만 무작위하게 선별하여 각 모델이 다른 변수를 학습할 수 있도록 한다. 회귀 모델을 사용하였을 때 모델을 어떻게 정의하고 접근하는지 아래에서 서술한다.
- 순환 신경망은 순차적 데이터를 분석하기 위한 방법론으로 많이 사용된다. 즉, 시간의 흐름에 따른 연속적인 데이터를 분석하기 위한 방법론이다. 전력 사용의 경우, 시간의 흐름에 따라 발생한다는 점에 착안하여 순환 신경망 모델을 통한 최대수요전력 예측 모델을 해결책으로 제시한다. 시간의 흐름에 따른 데이터 분석에서 해당 모델을 어떻게 정의하고 접근하는지 아래에서 서술한다.

• 적용하고자 하는 AI 분석 방법론(알고리즘)의 구체적 소개

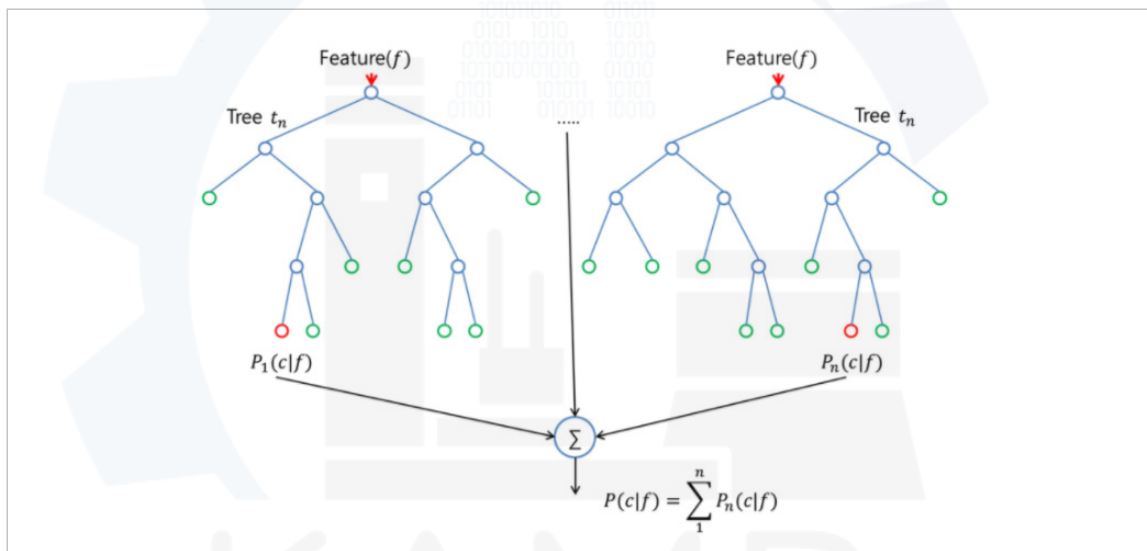
- 랜덤 포레스트란?

- 랜덤 포레스트 (Random Forest)란 앙상블 (Ensemble) 학습 방법의 일종으로 분류와 회귀 분석에 모두 사용되는 직관적인 머신러닝 알고리즘이다. 여러 개의 결정 트리를 생성하여 새로운 데이터를 각 트리에 통과시켜, 각 트리가 분류한 결과에서 투표를 실시하여 가장 많이 득표한 결과를 최종 분류 결과로 선택한다.
- 랜덤 포레스트의 기본이 되는 의사결정 나무 구조는 다음과 같다.



[그림 9] 의사결정 나무 구조 예시

- 여러 개의 결정 트리를 활용한 랜덤 포레스트의 구조는 다음과 같다.

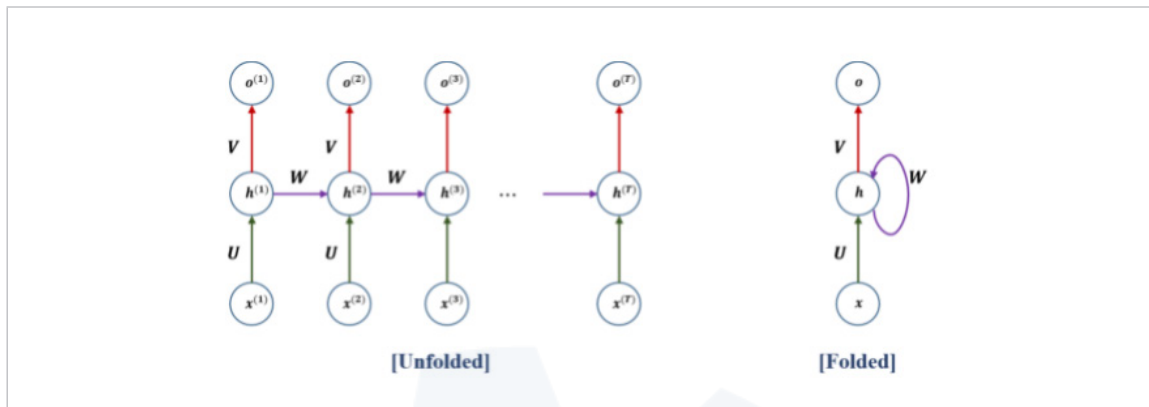


[그림 10] Random Forest의 앙상블 머신러닝 모델

- 순환 신경망이란?

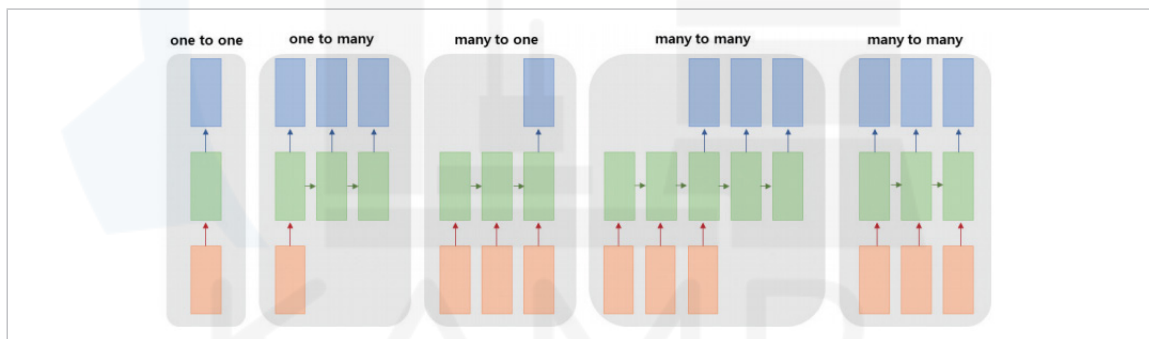
- 순환 신경망이란 인공 신경망의 한 종류로, 유닛 간의 연결이 순환적 구조를 갖는 특징을 갖고 있다. 즉, 입력층에서 출력층까지 한 방향으로만 흐르는 순방향(Feed Forward) 신경망에 비해, 순환 신경망은 순방향 신경망과 비슷하지만, 출력 결과를 다시 입력부로 받는 기능이 있는 것이 주요 특징이다. 이러한 구조는 시간에 따라 변하는 데이터의 동적(Dynamic) 특징을 학습할 수 있도록 신경망 내부에 상태(state)를 저장할 수 있게 해주므로, 순방향 신경망과 달리 내부의 메모리를 이용해 시퀀스(sequence) 형태의 입력을 처리할 수 있다. 순환 신경망은 시퀀스 데이터를 모델링 하기 위해 등장한 모델로서, 자연어나 음성신호, 시계열 데이터와 같은 연속적인

(sequential) 특징이 내재된 데이터에 아주 적합한 데이터라고 볼 수 있다.



[그림 11] 순환 신경망 기본 구조

- 기본 구조에서 살펴볼 수 있듯이, 과거 및 현재 정보를 모두 반영하여 다음 시점으로 넘겨주고 계산하는 작업을 반복적으로 실행한다. 반복 구조의 성질은 시퀀스나 리스트로 이어지는 것을 알려주고, 이러한 특성을 가지고 있는 데이터를 다루기에 최적화된 구조라는 것을 알 수 있다. 순환신경망은 입·출력의 형태에 따라 다양한 구조로 순환 신경망을 구성할 수 있다. 그렇기에 다양하고 유연한 구조를 만들 수 있다는 장점을 가지고 있다.



[그림 12] 순환 신경망의 다양한 구조 예시

• AI 분석 방법론(알고리즘) 구축 절차 설명

- 최대수요전력을 예측하기 위해서는 적합한 형태로 데이터셋을 변환하는 것이 필요하다. 전처리 과정을 통하여 수집된 데이터에 대해 정제 및 필터링을 진행한다. 이후 데이터셋을 학습 및 검증 데이터로 분리하여 일련의 학습 과정 내에서 목적에 부합하게 사용한다. 본 분석에서는 최대수요전력 예측 및 자원 최적화를 목적으로 순환 신경망 및 랜덤 포레스트를 활용한다. 순환 신경망의 경우 은닉층의 노드 개수, 활성화 함수 등을 사용자의 선택에 따라 다양하게 조정할 수 있다. 따라서 먼저 적합한 초매개변수를 설정하여 모델을 정의한다. 랜덤 포레스트의 경우 다수의 의사결정 트리들에 가

중치를 적용한 후 앙상블 기법으로 트리들의 의사를 하나로 모은 후 최대수요 전력에 대한 예측 결과 값을 출력하는 모델이다. 최종적으로 학습이 종료된 모델을 저장하고 실제 값과 비교 및 분석을 진행한 후, 예측된 최대수요전력 값을 바탕으로 자원최적화를 수행한다.

2.3 분석 체험

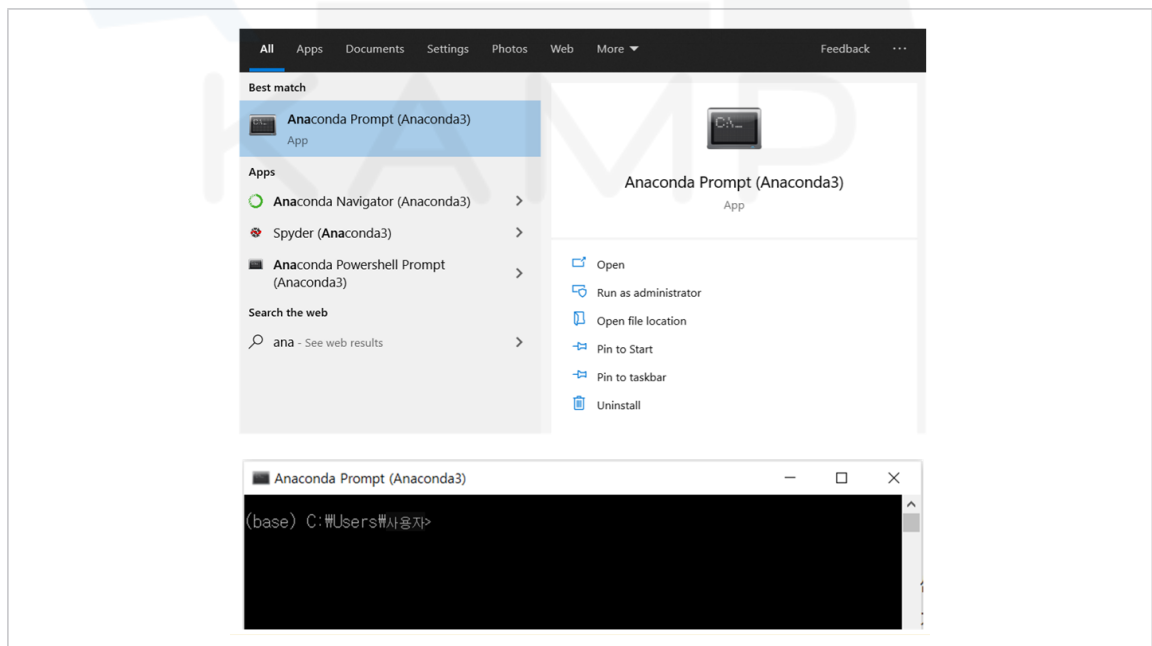
1) 필요 SW, 패키지 설치 방법 및 절차 가이드

▶ 필요 SW 설치

- 프로그래밍 언어인 파이썬(Python)을 사용하며 개발 환경으로는 주피터 노트북(Jupyter Notebook)을 활용한다. 관련 SW 설치는 3. 부록 [분석 환경 구축을 위한 설치 가이드]를 참고하면 된다.

▶ 가상환경 구축 가이드

- 3. 부록 [분석 환경 구축을 위한 설치 가이드]에서 설치한 아나콘다에서 파이썬의 독립적인 가상의 작업 환경을 구축한다. 파이썬의 모듈은 다양한 버전이 존재하기 때문에 충돌이 일어나기 쉬우며, 이러한 충돌을 방지하기 위해 실습마다 독립적인 가상 환경 구축을 권장한다. 가상 환경을 사용하기 위해서는 환경을 구축하고 실행(활성화)해야하고, 실습이 끝난 이후에는 환경을 비활성화 시켜야 한다. 주피터 노트북에서 가상 환경을 실행하는 구체적인 순서 및 실행 방법은 아래와 같다.
- 아나콘다 프롬프트를 실행한다.



- 가상환경 생성

```
conda create -n KAMP python=3.8.8
```

```
(base) C:\Users\사용자>conda create -n KAMP python=3.8.8
Collecting package metadata (current_repodata.json): done
Solving environment: failed with repodata from current_repodata.json, will retry with next repodata source
Collecting package metadata (repodata.json): done
Solving environment: done

==> WARNING: A newer version of conda exists. <==
  current version: 4.10.1
  latest version: 4.10.3

Please update conda by running

  $ conda update -n base -c defaults conda
```

...

```
Proceed ([y]/n)? y
```

해당 문구가 뜨면 'y' 를 입력한다.

...

```
done
#
# To activate this environment, use
#
#     $ conda activate KAMP
#
# To deactivate an active environment, use
#
#     $ conda deactivate
#
(base) C:\Users\사용자>
```

- 다음의 명령어를 실행하여 생성한 가상환경을 실행한다.

```
# conda activate 가상환경이름
```

```
conda activate KAMP
```

```
(base) C:\Users\사용자>conda activate KAMP
(KAMP) C:\Users\사용자>
```

- 분석이 끝난 후에는 다음의 명령어를 입력하여 가상 환경을 종료한다.

```
conda deactivate
```

```
(KAMP) C:\Users\사용자>conda deactivate
(base) C:\Users\사용자>
```

- 주피터 노트북에서 가상 환경 실행

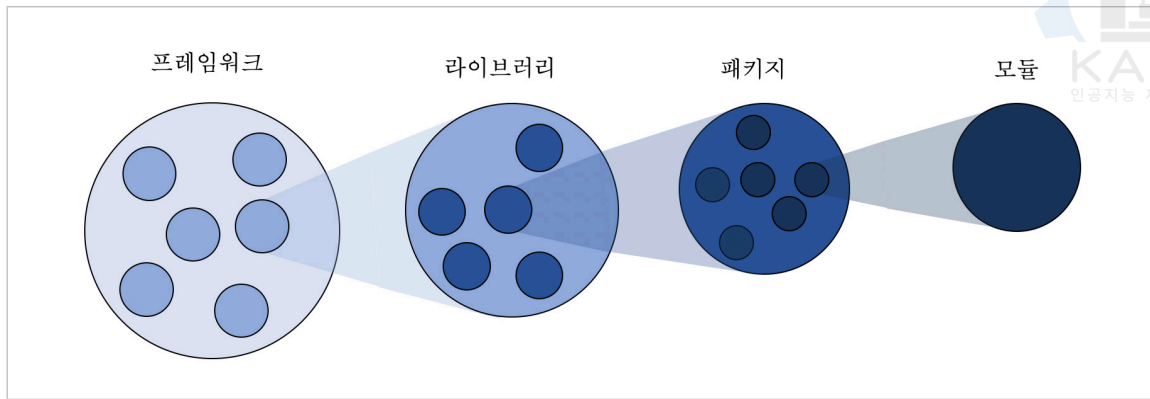
주피터 노트북을 실행한 후 상기의 과정을 통하여 생성한 가상환경이 주피터 노트북의 실행창의 빨간 박스 내에 존재함을 확인할 수 있다.



▶ 패키지 설치 가이드

- 패키지 설치 전 파이썬 관련 용어 정리

모듈 (module)	• 함수나 변수 또는 클래스를 모아 놓은 파일 • 파이썬 확장자(.py) 파일에 특정 함수, 변수, 클래스 등이 저장된 코드이다.
패키지 (package)	• 여러 모듈의 집합 • 디렉토리(경로)에 놓인 모듈들의 집합을 의미하며, 일반적으로 <code>__init__.py</code>라는 패키지 초기화 파일이 존재한다. 예) NumPy: 수치해석 패키지, pandas: 데이터 분석 패키지
라이브러리 (library)	• 여러 패키지의 집합 • 파이썬을 설치할 때 기본적으로 설치되는 라이브러리를 표준라이브러리라고 한다. 파이썬 공식이 아닌 외부에서 개발한 모듈과 패키지를 묶어 외부 라이브러리라고 한다. 예) matplotlib: 데이터 시각화를 생성하기 위한 표준 라이브러리
프레임워크 (framework)	• 라이브러리의 집합 • 목적에 맞게 코드를 작성할 수 있는 기본 아키텍처이며, 필수 구성 요소를 제공한다.



[그림 13] 파이썬 관련 용어 개념

▶ 필요 패키지 설치 방법

- 분석 환경인 주피터 노트북 내에서 데이터 분석 및 AI 분석모델 적용을 위한 패키지를 설치한다. `pip install <패키지 이름>` 을 입력하여 원하는 라이브러리(혹은 패키지)를 설치할 수 있다.

```
!pip install pandas
!pip install numpy
!pip install matplotlib
!pip install scikit-learn
!pip install keras
!pip install tensorflow
!pip install datetime
!pip install seaborn
!pip install ortools
```

- 이미 설치되어 있는 패키지 외에 신규 설치가 필요한 패키지들이 자동으로 설치된다.

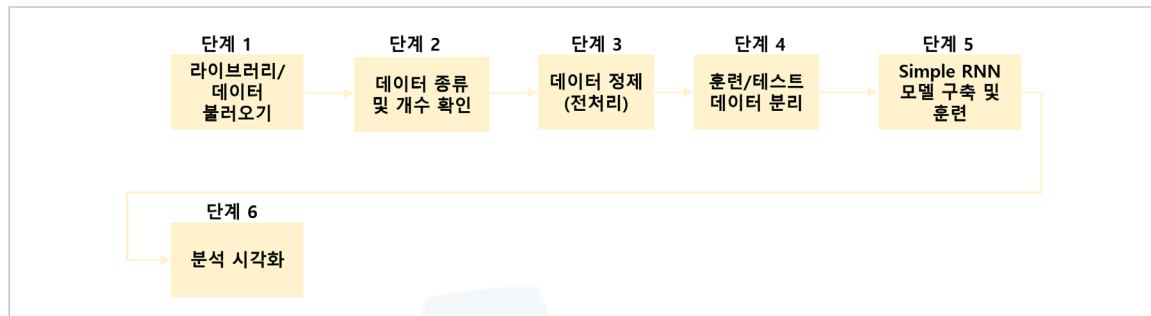
```
Requirement already satisfied: pandas in /home/ubuntu/anaconda3/lib/python3.6/site-packages (0.23.0)
Requirement already satisfied: python-dateutil>=2.5.0 in /home/ubuntu/anaconda3/lib/python3.6/site-packages (from pandas) (2.7.3)
Requirement already satisfied: pytz>=2011k in /home/ubuntu/anaconda3/lib/python3.6/site-packages (from pandas) (2018.4)
Requirement already satisfied: numpy>=1.9.0 in /home/ubuntu/anaconda3/lib/python3.6/site-packages (from pandas) (1.19.5)
Requirement already satisfied: six>=1.5 in /home/ubuntu/anaconda3/lib/python3.6/site-packages (from python-dateutil>=2.5.0->pandas) (1.15.0)
```

- 설치된 패키지들의 전체 목록을 확인할 수 있다.

```
pip list
```

Package	Version
abs1-py	0.14.1
alabaster	0.7.10
alembic	1.4.1
anaconda-client	1.6.14
anaconda-navigator	1.8.7
anaconda-project	0.8.2

2) 분석 단계별 프로세스 - 순환 신경망 알고리즘



[그림 14] 지도 학습 적용을 위한 순환 신경망 모델 구현 과정

[단계 ①] 라이브러리/데이터 불러오기

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from datetime import datetime
```

[코드 1] 필요 패키지 불러오기 1

①-1. 필요 데이터 다운로드 및 불러오기(import) 가이드

- 레이블 된 데이터를 불러온다. csv 파일을 불러올 때에는 pandas의 'read_csv('파일 경로')' 기능을 사용한다. 순환 신경망 기반의 최대수요전력 예측 알고리즘 방식 이므로 정수형 변수가 존재하는 데이터를 사용한다.
- 기존 원본 파일에 기상청 데이터로부터 가져온 기후 데이터와 한국전력에서 가져온 전기요금 데이터를 추가한 최종 파일인 'okm_augumented_2021.csv' 파일을 불러온다.

```
df = pd.read_csv('./okm_augumented_2021.csv')
```

[코드 2] okm_augumented_2021.csv 파일 읽기

[단계 ②] 데이터 종류 및 개수 확인

②-1. 데이터 종류 및 개수 확인 가이드

- 가장 기본적인 데이터 종류 및 개수를 확인하는 방법은 상기에서 csv 파일을 불러오 기 위해 선언한 변수를 사용하는 방법이다.

df

	날짜	시간	15분	30분	45분	60분	평균	생산량	기온	풍속	습도	강수량	전기요금(계절)	day	d	m	공장인원	인건비
0	20210101	0	62	61	61	61	61	0	-3.2	2.4	71	0.0	109.8	5	1	1	0.000000	1.5
1	20210101	1	96	93	116	113	105	0	-4.5	1.5	77	0.0	109.8	5	1	1	0.000000	1.5
2	20210101	2	106	96	106	107	104	0	-3.9	2.6	58	0.0	109.8	5	1	1	0.000000	1.5
3	20210101	3	92	110	110	109	105	0	-4.1	2.6	56	0.0	109.8	5	1	1	0.000000	1.5
4	20210101	4	108	105	106	108	107	0	-4.6	2.6	60	0.0	109.8	5	1	1	0.000000	1.5
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
6163	20210914	19	152	151	171	139	153	1497	21.7	3.6	85	9.4	167.2	2	14	9	2.442088	1.5
6164	20210914	20	124	130	128	130	128	45	22.2	4.2	78	9.4	167.2	2	14	9	0.087891	1.5
6165	20210914	21	134	130	125	124	128	149	22.2	4.3	76	9.4	167.2	2	14	9	0.290448	1.5
6166	20210914	22	100	109	120	114	111	66	22.0	2.5	79	9.4	167.2	2	14	9	0.148984	1.5
6167	20210914	23	117	119	112	91	110	405	22.0	2.5	79	9.4	167.2	2	14	9	0.922551	1.5

6168 rows × 18 columns

[코드 3] okm_augumented_2021.csv 내용 확인하기

- 상기의 방법으로 총 해당 파일의 변수, 값, 행 및 열의 수 등을 확인할 수 있다.
- 상기 방법 외에도 어떤 데이터 종류가 있는지 간략히 확인할 수 있는 방법에는 'head()'를 사용한다. 기본으로 위에서 5개의 row를 읽어온다. 'head()'의 ()안에 읽어들 row의 수를 입력할 수 있다.

df.head()

	날짜	시간	15분	30분	45분	60분	평균	생산량	기온	풍속	습도	강수량	전기요금(계절)	day	d	m	공장인원	인건비
0	20210101	0	62	61	61	61	61	0	-3.2	2.4	71	0.0	109.8	5	1	1	0.0	1.5
1	20210101	1	96	93	116	113	105	0	-4.5	1.5	77	0.0	109.8	5	1	1	0.0	1.5
2	20210101	2	106	96	106	107	104	0	-3.9	2.6	58	0.0	109.8	5	1	1	0.0	1.5
3	20210101	3	92	110	110	109	105	0	-4.1	2.6	56	0.0	109.8	5	1	1	0.0	1.5
4	20210101	4	108	105	106	108	107	0	-4.6	2.6	60	0.0	109.8	5	1	1	0.0	1.5

[코드 4] head()를 활용한 기본 데이터 호출하기

df.head(10)

	날짜	시간	15분	30분	45분	60분	평균	생산량	기온	풍속	습도	강수량	전기요금(계절)	day	d	m	공장인원	인건비
0	20210101	0	62	61	61	61	61	0	-3.2	2.4	71	0.0	109.8	5	1	1	0.0	1.5
1	20210101	1	96	93	116	113	105	0	-4.5	1.5	77	0.0	109.8	5	1	1	0.0	1.5
2	20210101	2	106	96	106	107	104	0	-3.9	2.6	58	0.0	109.8	5	1	1	0.0	1.5
3	20210101	3	92	110	110	109	105	0	-4.1	2.6	56	0.0	109.8	5	1	1	0.0	1.5
4	20210101	4	108	105	106	108	107	0	-4.6	2.6	60	0.0	109.8	5	1	1	0.0	1.5
5	20210101	5	89	83	99	98	92	0	-5.5	2.1	66	0.0	109.8	5	1	1	0.0	1.5
6	20210101	6	102	111	110	103	107	0	-5.6	1.5	67	0.0	109.8	5	1	1	0.0	1.5
7	20210101	7	92	75	48	29	61	0	-6.8	0.6	73	0.0	109.8	5	1	1	0.0	1.5
8	20210101	8	27	25	22	22	24	0	-5.6	0.0	72	0.0	109.8	5	1	1	0.0	1.5
9	20210101	9	22	22	22	22	22	0	-3.3	1.2	62	0.0	109.8	5	1	1	0.0	1.0

[코드 5] head(10)를 활용한 기본 데이터 호출하기

- 'head()'와 반대로 뒤에서부터 데이터 구성을 확인할 수 있는 방법이 있다. 'tail()'을 사용하면 가능하며 기본으로 밑에서 5개의 row를 읽어온다. 'tail()'의 () 안에 읽어올 row의 수를 입력할 수 있다.

df.tail()

	날짜	시간	15분	30분	45분	60분	평균	생산량	기온	풍속	습도	강수량	전기요금(계절)	day	d	m	공장인원	인건비
6163	20210914	19	152	151	171	139	153	1497	21.7	3.6	85	9.4	167.2	2	14	9	2.442088	1.5
6164	20210914	20	124	130	128	130	128	45	22.2	4.2	78	9.4	167.2	2	14	9	0.087891	1.5
6165	20210914	21	134	130	125	124	128	149	22.2	4.3	76	9.4	167.2	2	14	9	0.290448	1.5
6166	20210914	22	100	109	120	114	111	66	22.0	2.5	79	9.4	167.2	2	14	9	0.148984	1.5
6167	20210914	23	117	119	112	91	110	405	22.0	2.5	79	9.4	167.2	2	14	9	0.922551	1.5

[코드 6] tail()을 활용한 기본 데이터 호출하기

df.tail(10)

	날짜	시간	15분	30분	45분	60분	평균	생산량	기온	풍속	습도	강수량	전기요금(계절)	day	d	m	공장인원	인건비
6158	20210914	14	161	162	154	156	158	1023	21.5	2.9	92	7.2	167.2	2	14	9	1.616114	1.0
6159	20210914	15	132	169	175	168	161	1178	20.6	1.5	98	8.6	167.2	2	14	9	1.829193	1.0
6160	20210914	16	164	165	156	153	160	1395	21.7	2.5	95	9.1	167.2	2	14	9	2.186520	1.0
6161	20210914	17	117	94	133	146	123	230	21.8	2.8	91	9.1	167.2	2	14	9	0.469388	1.0
6162	20210914	18	158	164	147	163	158	973	20.9	3.5	94	9.1	167.2	2	14	9	1.539557	1.5
6163	20210914	19	152	151	171	139	153	1497	21.7	3.6	85	9.4	167.2	2	14	9	2.442088	1.5
6164	20210914	20	124	130	128	130	128	45	22.2	4.2	78	9.4	167.2	2	14	9	0.087891	1.5
6165	20210914	21	134	130	125	124	128	149	22.2	4.3	76	9.4	167.2	2	14	9	0.290448	1.5
6166	20210914	22	100	109	120	114	111	66	22.0	2.5	79	9.4	167.2	2	14	9	0.148984	1.5
6167	20210914	23	117	119	112	91	110	405	22.0	2.5	79	9.4	167.2	2	14	9	0.922551	1.5

[코드 7] tail(10)을 활용한 기본 데이터 호출하기

- 'info()'를 사용하여, 전체적인 데이터의 구조 및 타입을 확인할 수 있다.

```
df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6168 entries, 0 to 6167
Data columns (total 18 columns):
#   Column                Non-Null Count  Dtype
---  -
0   날짜                  6168 non-null   int64
1   시간                  6168 non-null   int64
2   15분                  6168 non-null   int64
3   30분                  6168 non-null   int64
4   45분                  6168 non-null   int64
5   60분                  6168 non-null   int64
6   평균                  6168 non-null   int64
7   생산량                6168 non-null   int64
8   기온                  6168 non-null   float64
9   풍속                  6165 non-null   float64
10  습도                  6168 non-null   int64
11  강수량                6167 non-null   float64
12  전기요금(계절)        6168 non-null   float64
13  day                   6168 non-null   int64
14  d                     6168 non-null   int64
15  m                     6168 non-null   int64
16  공장인원              6151 non-null   float64
17  인건비                6168 non-null   float64
dtypes: float64(6), int64(12)
memory usage: 867.5 KB
```

[코드 8] info()를 사용한 csv의 전체 데이터 구성 확인하기

[단계 ③] 데이터 정제 (전처리)

③-1. 결측치 처리

- 현재 데이터에는 결측치가 존재한다. 이는 데이터의 완전성 등 데이터 품질 지수를 위반할 수 있으므로 결측치에 대한 전처리를 시행한다.
- 데이터에 존재하는 결측치를 찾기 위해서는 먼저 다음과 같이 어떤 변수에 몇 개의 변수에 결측치가 존재하는지 확인할 수 있다.
- 'isnull()'만을 사용할 경우, 결측치 값에는 True, 그 외에는 False가 표현된다. 직관적으로 확인이 가능하지만, 해당 변수의 몇 개의 결측치가 존재하는지 파악하기 위해서는 추가로 'sum()'을 사용하여야 한다.

df.isnull()

	날짜	시간	15분	30분	45분	60분	평균	생산량	기온	풍속	습도	강수량	전기요금(계절)	day	d	m	공장인원	인건비
0	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False
1	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False
2	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False
3	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False
4	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
6163	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False
6164	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False
6165	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False
6166	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False
6167	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False

6168 rows × 18 columns

[코드 9] isnull()을 활용한 결측치 값 결과 확인하기

df.isnull().sum()

```

날짜      0
시간      0
15분      0
30분      0
45분      0
60분      0
평균      0
생산량    0
기온      0
풍속      3
습도      0
강수량    1
전기요금(계절)  0
day      0
d        0
m        0
공장인원  17
인건비    0
dtype: int64

```

[코드 10] isnull().sum()을 활용한 결측치 개수 결과 확인하기

- 상기에서 확인한 바와 같이 현재 풍속, 강수량, 공장인원에서 결측치가 존재하는 것을 확인할 수 있다. 예를 들어, 현재 강수량이라는 변수의 데이터에는 결측치가 1개가 존재하는 것을 확인할 수 있다.
- 학습을 위해서 결측치가 존재하는 데이터를 사용할 수는 없다. 그렇기에 해당 데이터의 값을 변환시켜야한다. 예를 들어, 강수량의 결측치 값을 확인해보니 실제 비가 오지 않은 날에는 강수량의 정보가 누락 되는 경우가 있어 어떠한 값도 해당 데이터에 없음을 확인할 수 있었다. 이 경우에는 결측치 값을 '0'으로 대체할 수 있다.

- 하기와 같이 결측치가 존재하는 특정 변수를 지정할 수 있으며, 'fillna()' 함수를 이용하여 현재 결측치를 나타내는 True 때, 값을 0으로 채울 수 있다.

```
df['강수량'].fillna(0, inplace = True)
df
```

	날짜	시간	15분	30분	45분	60분	평균	생산량	기온	풍속	습도	강수량	전기요금(계절)	day	d	m	공장인원	인건비
0	20210101	0	62	61	61	61	61	0	-3.2	2.4	71	0.0	109.8	5	1	1	0.000000	1.5
1	20210101	1	96	93	116	113	105	0	-4.5	1.5	77	0.0	109.8	5	1	1	0.000000	1.5
2	20210101	2	106	96	106	107	104	0	-3.9	2.6	58	0.0	109.8	5	1	1	0.000000	1.5
3	20210101	3	92	110	110	109	105	0	-4.1	2.6	56	0.0	109.8	5	1	1	0.000000	1.5
4	20210101	4	108	105	106	108	107	0	-4.6	2.6	60	0.0	109.8	5	1	1	0.000000	1.5
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
6163	20210914	19	152	151	171	139	153	1497	21.7	3.6	85	9.4	167.2	2	14	9	2.442088	1.5
6164	20210914	20	124	130	128	130	128	45	22.2	4.2	78	9.4	167.2	2	14	9	0.087891	1.5
6165	20210914	21	134	130	125	124	128	149	22.2	4.3	76	9.4	167.2	2	14	9	0.290448	1.5
6166	20210914	22	100	109	120	114	111	66	22.0	2.5	79	9.4	167.2	2	14	9	0.148984	1.5
6167	20210914	23	117	119	112	91	110	405	22.0	2.5	79	9.4	167.2	2	14	9	0.922551	1.5

6168 rows x 18 columns

[코드 11] fillna()를 이용한 특정 변수 결측치 값 채우기

- 또한, 특정 변수를 지정하지 않고 모든 결측치 값을 0으로 채워넣을 수 있다. 이 경우에는 반드시 0으로 할 필요는 없으며, 실제 값을 확인하고 진행하면 된다. 해당 데이터에서는 확인 결과, 결측치 값들은 모두 0임을 확인하고 진행하였다.
- 추가로 해당 결측치를 제거한 데이터를 랜덤 포레스트 모델에 사용하기 위하여 “df_frame”이라는 변수를 생성하여 현재까지 전처리한 데이터를 미리 저장하여, ‘2.3.2 랜덤포레스트 알고리즘을 이용한 분석’ 시에 사용한다.

```
df.fillna(0, inplace = True)
df_frame = df
```

[코드 12] fillna()를 전체 변수 결측치 값 채우기

- 상기와 같이 진행 후 다시 결측치를 확인하면 없음을 확인할 수 있다.

```
df.isnull().sum()
```

```
날짜      0
시간      0
15분     0
30분     0
45분     0
60분     0
평균      0
생산량    0
기온      0
풍속      0
습도      0
강수량    0
전기요금(계절)  0
day       0
d         0
m         0
공장인원  0
인건비    0
dtype: int64
```

[코드 13] isnull().sum()을 활용한 결측치 개수 확인하기

③-2. 데이터 특성 파악

- 지금까지 각 변수의 데이터 값에 대한 결측치를 제거하였다. 결측치가 존재하지 않기에 지금부터는 정확한 수치형 데이터들의 요약통계량을 확인할 수 있다. 이 경우, 'describe()' 함수를 사용한다.

```
df.describe()
```

	날짜	시간	15분	30분	45분	60분	평균	생산량	기온	풍속	습도
count	6.168000e+03	6168.000000	6168.000000	6168.000000	6168.000000	6168.000000	6168.000000	6168.000000	6168.000000	6168.000000	6168.000000
mean	2.021049e+07	12.428016	90.410182	92.695363	95.106355	95.037938	93.424125	467.344682	15.906064	2.062630	70.098735
std	2.447756e+02	12.847309	55.349403	57.942122	59.285709	59.347554	57.355938	857.571815	9.160356	1.164724	22.996164
min	2.021010e+07	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	-12.000000	0.000000	8.000000
25%	2.021031e+07	6.000000	23.000000	23.000000	23.000000	23.000000	23.000000	0.000000	9.600000	1.200000	53.000000
50%	2.021051e+07	12.000000	101.000000	104.000000	105.000000	107.000000	104.000000	45.000000	17.400000	1.900000	74.000000
75%	2.021071e+07	18.000000	133.000000	143.000000	149.000000	149.000000	144.000000	637.250000	23.300000	2.800000	91.000000
max	2.021091e+07	188.000000	207.000000	222.000000	218.000000	214.000000	208.000000	9830.000000	33.400000	7.600000	98.000000

[코드 14] describe()를 활용한 데이터 요약통계량 확인하기

- 해당 함수를 활용하면 상기와 같이 데이터의 개수, 평균, 표준편차, 최소값, 최대값 등을 확인할 수 있다.
- 해당 수치형 데이터들의 상관관계 또한 확인할 수 있다. 'corr()' 함수를 사용하여 각 데이터 간 상관관계를 파악할 수 있다.

df.corr()

	날짜	시간	15분	30분	45분	60분	평균	생산량	기온	풍속	습도	강수량	전기요금 (계절)	인공 day
날짜	1.000000	0.065435	0.035555	0.034746	0.037285	0.036244	0.036572	0.070719	0.863028	-0.268152	0.548722	0.160855	0.835836	-0.011838
시간	0.065435	1.000000	0.130084	0.124554	0.115947	0.097021	0.117885	0.031816	0.150262	0.009283	-0.030201	0.049157	0.067609	-0.036188
15분	0.035555	0.130084	1.000000	0.980295	0.966859	0.950034	0.984688	0.520424	0.050760	0.115215	-0.085205	-0.008241	0.055881	-0.428312
30분	0.034746	0.124554	0.980295	1.000000	0.982443	0.959004	0.991255	0.515841	0.049524	0.114173	-0.084102	-0.010410	0.056203	-0.426674
45분	0.037285	0.115947	0.966859	0.982443	1.000000	0.984789	0.994774	0.513789	0.052818	0.115768	-0.082522	-0.013136	0.059044	-0.432957
60분	0.036244	0.097021	0.950034	0.959004	0.984789	1.000000	0.984828	0.499723	0.055651	0.128605	-0.090875	-0.014812	0.057878	-0.437576
평균	0.036572	0.117885	0.984688	0.991255	0.994774	0.984828	1.000000	0.517948	0.052996	0.119837	-0.086508	-0.011759	0.058096	-0.436280
생산량	0.070719	0.031816	0.520424	0.515841	0.513789	0.499723	0.517948	1.000000	0.117780	0.115691	-0.109503	0.008171	0.068930	-0.259129
기온	0.863028	0.150262	0.050760	0.049524	0.052818	0.055651	0.052996	0.117780	1.000000	-0.192121	0.401553	0.118524	0.809825	0.010130
풍속	-0.268152	0.009283	0.115215	0.114173	0.115768	0.128605	0.119837	0.115691	-0.192121	1.000000	-0.440112	0.048412	-0.254750	0.004037
습도	0.548722	-0.030201	-0.085205	-0.084102	-0.082522	-0.090875	-0.086508	-0.109503	0.401553	-0.440112	1.000000	0.219997	0.525520	0.052820
강수량	0.160855	0.049157	-0.008241	-0.010410	-0.013136	-0.014812	-0.011759	0.008171	0.118524	0.048412	0.219997	1.000000	0.139347	-0.007520
전기요금 (계절)	0.835836	0.067609	0.055881	0.056203	0.059044	0.057878	0.058096	0.068930	0.809825	-0.254750	0.525520	0.139347	1.000000	-0.024459
day	-0.011838	-0.036188	-0.428312	-0.426674	-0.432957	-0.437576	-0.436280	-0.259129	0.010130	0.004037	0.052820	-0.007520	-0.024459	1.000000
d	-0.035210	-0.010330	0.052554	0.055754	0.056047	0.056354	0.055824	0.044836	0.108487	-0.010588	-0.023053	-0.035694	0.017165	0.018233
m	0.999357	0.065681	0.033601	0.032679	0.035203	0.034153	0.034499	0.068976	0.857490	-0.267261	0.548503	0.161829	0.833626	-0.012469
인공 인간 비	-0.002136	0.012563	0.295909	0.288822	0.288091	0.276778	0.289860	0.785115	0.030540	0.088944	-0.121135	-0.004847	-0.020939	-0.159877
인간 비	0.005469	-0.034275	-0.202557	-0.195760	-0.211635	-0.243434	-0.215982	-0.280625	-0.184611	-0.350115	0.308435	-0.012680	0.005660	-0.003020

[코드 15] corr()를 활용한 상관관계 확인하기

- 상관관계 값의 경우, -1과 1 사이에 존재한다. 1에 가까울수록 양의 상관관계가 높으며 -1에 가까울수록 음의 상관관계가 높다고 볼 수 있다. 대략적인 상관관계수의 절대값의 의미는 다음과 같다.

| 0.0 ~ 0.2 | : 상관관계가 거의 없다.

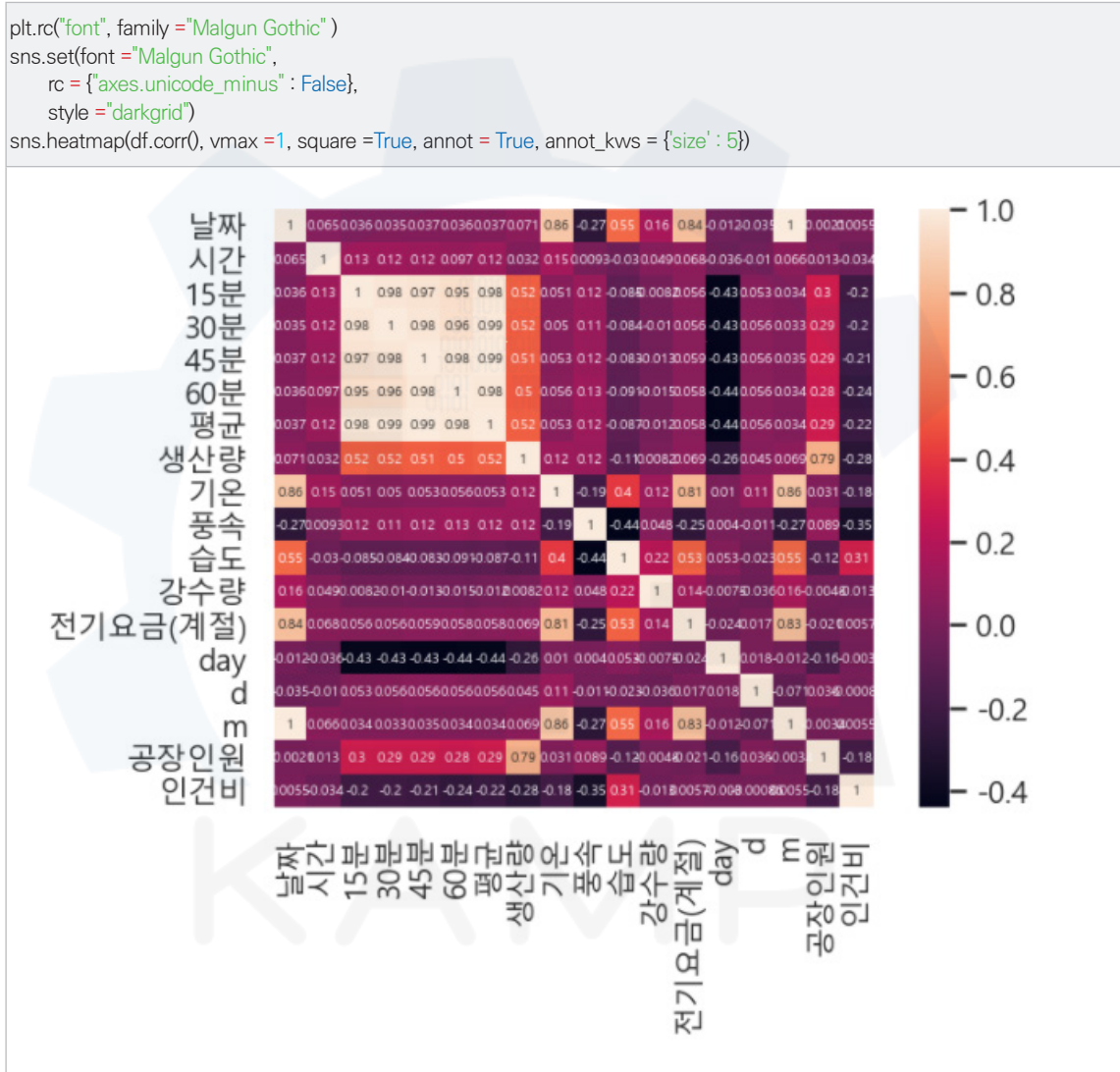
| 0.2 ~ 0.4 | : 상관관계가 낮다.

| 0.4 ~ 0.6 | : 상관관계가 있다.

| 0.6 ~ 0.8 | : 상관관계가 높다.

| 0.8 ~ 1.0 | : 상관관계가 매우 높다.

- 각 데이터 간의 상관관계를 확인하였을 때, 서로 이어진 시간의 흐름끼리 상관관계가 매우 높다고 볼 수 있다(0.9와 1 사이의 값). 또한, 추가로 생산량이 상관관계가 있다고 볼 수 있다(0.5 정도의 값).
- 상관관계와 관련해서, 상기의 방법과 추가로 시각화하여 확인할 수도 있다. 여기에서는 처음에 선언한 seaborn 라이브러리를 사용하여 시각화할 수 있다. seaborn 라이브러리로 생성한 시각화 자료의 글씨가 깨지지 않도록 해당 글씨체를 지정한다.



[코드 16] heatmap()을 사용한 상관관계 시각화하기

- seaborn 라이브러리의 함수 'heatmap()'을 사용하면 상기와 같이 상관관계를 직관적으로 확인할 수 있도록 시각화를 제공한다. 여기에서는 색이 연한 색이 될수록 양의 상관관계가 높다는 것을 확인할 수 있다.
- 해당 시각화를 통해서도 알 수 있듯이, 15분, 30분, 45분, 60분의 시간의 흐름에 따

른 서로의 최대수요전력들이 상관관계가 가장 높은 것을 확인할 수 있다.

- 하여 해당 실습에서는 상관관계가 낮은 변수들은 제거 후, 상관관계가 높은 Column들을 가지고 실습을 진행한다.

③-3. 데이터 정제

- 사용하지 않는 Column을 제거하기 위해서는 'drop()' 함수를 사용하며, 하기와 같이 제거할 Column 명을 입력 후 실행하면 된다. 여기서는 서로 연결된 15분 시간 단위 Column을 제외하고 제거한다.

```
df=df.drop(columns=["생산량", "기온", "풍속", "습도", "강수량", "전기요금(계절)", "day", "d", "m", "공장인원", "인건비"])
```

df

	날짜	시간	15분	30분	45분	60분	평균
0	20210101	0	62	61	61	61	61.25
1	20210101	1	96	93	116	113	104.50
2	20210101	2	106	96	106	107	103.75
3	20210101	3	92	110	110	109	105.25
4	20210101	4	108	105	106	108	106.75
...	...	...	...	...	...	...	...
6163	20210914	19	152	151	171	139	153.25
6164	20210914	20	124	130	128	130	128.00
6165	20210914	21	134	130	125	124	128.25
6166	20210914	22	100	109	120	114	110.75
6167	20210914	23	117	119	112	91	109.75

6168 rows × 7 columns

[코드 17] drop()을 사용한 변수 및 데이터 제거하기

- 상기와 같이 drop() 함수에 넣은 Column들은 삭제되고, 그 외의 변수들(날짜, 시간, 15분, 30분, 45분, 60분, 평균)의 데이터는 남아 있음을 확인할 수 있다.
- 추가로 변수 이름을 변경할 수 있다. 'rename()' 함수를 사용하여 변경하고자 하는 대상 변수와 변경하고자 하는 변수 이름을 연결하여 변경할 수 있다. 하기의 예시를 통해서 살펴보면 다음과 같다.

```
df = df.rename(columns={"15분": "Peak Consumption in 15 minute",
                        "30분": "Peak Consumption in 30 minute",
                        "45분": "Peak Consumption in 45 minute",
                        "60분": "Peak Consumption in 60 minute",
                        "평균": "Average Peak Consumption in 1 hour"})
```

	날짜	시간	Peak Consumption in 15 minute	Peak Consumption in 30 minute	Peak Consumption in 45 minute	Peak Consumption in 60 minute	Average Peak Consumption in 1 hour
0	20210101	0	62	61	61	61	61.25
1	20210101	1	96	93	116	113	104.50
2	20210101	2	106	96	106	107	103.75
3	20210101	3	92	110	110	109	105.25
4	20210101	4	108	105	106	108	106.75
...	...	...	...	...	...	...	...
6163	20210914	19	152	151	171	139	153.25
6164	20210914	20	124	130	128	130	128.00
6165	20210914	21	134	130	125	124	128.25
6166	20210914	22	100	109	120	114	110.75
6167	20210914	23	117	119	112	91	109.75

6168 rows x 7 columns

[코드 18] rename()을 사용한 변수 이름 변경하기

- 현재 인덱스는 단순히 숫자로 구성되어 있으며, 향후 학습 또는 시각화를 위해서 날짜 데이터로 인덱스를 구성하는 것이 좋다. 현재 구성된 Column들이 각 날짜의 해당 시간의 15분 단위별 최대수요전력을 나타내기 때문에, 인덱스를 해당 날짜의 시간대로 변경하는 것이 향후 학습에 있어 중요하다.
- 인덱스를 날짜 및 시간으로 생성하기 위해서 다음과 같이 변수를 선언한다. Pandas 라이브러리의 'date_range()'의 함수를 사용할 수 있는데 start와 end로 범위를 설정 후, 여기서는 freq로 Hour을 선택하여 1시간 단위로 해당 인덱스를 생성하였다.

```
index_column = pd.date_range(start='2021-01-01 00:00', end='2021-09-14 23:00', freq='H')
index_column
```

```
DatetimeIndex(['2021-01-01 00:00:00', '2021-01-01 01:00:00',
               '2021-01-01 02:00:00', '2021-01-01 03:00:00',
               '2021-01-01 04:00:00', '2021-01-01 05:00:00',
               '2021-01-01 06:00:00', '2021-01-01 07:00:00',
               '2021-01-01 08:00:00', '2021-01-01 09:00:00',
               ...,
               '2021-09-14 14:00:00', '2021-09-14 15:00:00',
               '2021-09-14 16:00:00', '2021-09-14 17:00:00',
               '2021-09-14 18:00:00', '2021-09-14 19:00:00',
               '2021-09-14 20:00:00', '2021-09-14 21:00:00',
               '2021-09-14 22:00:00', '2021-09-14 23:00:00'],
              dtype='datetime64[ns]', length=6168, freq='H')
```

[코드 19] date_range()를 활용한 인덱스 값 변경하기

- 여기서 주의할 사항은 기존의 인덱스 개수와 변경하고자 하는 인덱스 개수가 동일해야 한다. 동일하지 않다면, 오류가 발생한다. 여기에서는 기존 인덱스와 변경하고자 하는 인덱스가 6,168개로 동일한 것을 확인할 수 있다.
- 기존의 인덱스에 변경하고자 하는 인덱스를 적용하기 위해서 기존의 인덱스에 상에서 선언한 인덱스 변수를 할당한다. 또한, 인덱스의 명칭을 설정하기 위해 하기의 예시와 같이 적용한다.

```
df.index = index_column
df.index.names = ['Date']
df.head()
```

	날짜	시간	Peak Consumption in 15 minute	Peak Consumption in 30 minute	Peak Consumption in 45 minute	Peak Consumption in 60 minute	Average Peak Consumption in 1 hour
Date							
2021-01-01 00:00:00	20210101	0	62	61	61	61	61
2021-01-01 01:00:00	20210101	1	96	93	116	113	105
2021-01-01 02:00:00	20210101	2	106	96	106	107	104
2021-01-01 03:00:00	20210101	3	92	110	110	109	105
2021-01-01 04:00:00	20210101	4	108	105	106	108	107

[코드 20] 인덱스 명칭 설정 결과

- 상에서 볼 수 있듯이, 새롭게 인덱스가 설정된 것을 확인할 수 있다. 새롭게 설정한 인덱스와 날짜와 시간 Column이 중복되기에 drop 함수를 사용하여 날짜와 시간 Column을 제거한다.

```
df = df.drop(columns = ["날짜", "시간"])
df
```

Date	Peak Consumption in 15 minute	Peak Consumption in 30 minute	Peak Consumption in 45 minute	Peak Consumption in 60 minute	Average Peak Consumption in 1 hour
2021-01-01 00:00:00	62	61	61	61	61.25
2021-01-01 01:00:00	96	93	116	113	104.50
2021-01-01 02:00:00	106	96	106	107	103.75
2021-01-01 03:00:00	92	110	110	109	105.25
2021-01-01 04:00:00	108	105	106	108	106.75
...	...	...	...	...	...
2021-09-14 19:00:00	152	151	171	139	153.25
2021-09-14 20:00:00	124	130	128	130	128.00
2021-09-14 21:00:00	134	130	125	124	128.25
2021-09-14 22:00:00	100	109	120	114	110.75
2021-09-14 23:00:00	117	119	112	91	109.75

6168 rows × 6 columns

[코드 21] drop()을 사용한 변수 및 데이터 제거하기

- 현재까지 처음과 비교하였을 때, 데이터가 많이 정리된 것을 확인할 수 있다. 추가로 데이터의 일관성을 위해서 해당 데이터 값들을 실수형(float type)으로 변경한다. 실수형으로 변경하는 이유는 향후 학습을 위한 계산을 용이하게 하기 위한 사전 작업이기 때문이다.

```
df['Peak Consumption in 15 minute'] = [float(str(val).replace('.', '')) for val in df['Peak Consumption in 15 minute'].values]
df['Peak Consumption in 30 minute'] = [float(str(val).replace('.', '')) for val in df['Peak Consumption in 30 minute'].values]
df['Peak Consumption in 45 minute'] = [float(str(val).replace('.', '')) for val in df['Peak Consumption in 45 minute'].values]
df['Peak Consumption in 60 minute'] = [float(str(val).replace('.', '')) for val in df['Peak Consumption in 60 minute'].values]
df.head()
```

Date	Peak Consumption in 15 minute	Peak Consumption in 30 minute	Peak Consumption in 45 minute	Peak Consumption in 60 minute	Average Peak Consumption in 1 hour
2021.01.01 00:00:00	62.0	61.0	61.0	61.0	61.25
2021.01.01 01:00:00	96.0	93.0	116.0	113.0	104.50
2021.01.01 02:00:00	106.0	96.0	106.0	107.0	103.75
2021.01.01 03:00:00	92.0	110.0	110.0	109.0	105.25
2021.01.01 04:00:00	108.0	105.0	106.0	108.0	106.75

[코드 22] Float type으로 데이터 변경하기

③-4. 데이터 시각화 및 파일 저장

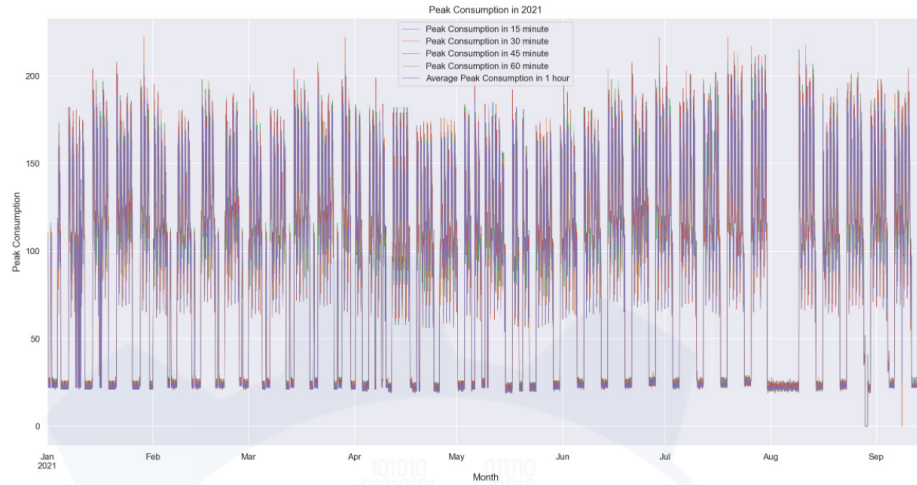
- 데이터를 시각화하기 위해서 seaborn 및 matplotlib 라이브러리를 사용한다. 여기서는 seaborn 라이브러리의 기본 스타일을 사용하고 그림 크기를 하기와 같이 설정한다.

```
sns.set(rc = {"figure.figsize": (20, 10)})
```

[코드 23] 그림 크기 지정하기

- 2021년도에 해당하는 모든 데이터를 그래프로 시각화하며, 그래프, x축 및 y축의 이름을 설정할 수 있다. 다음 코드 예시와 같이 표현할 수 있다.

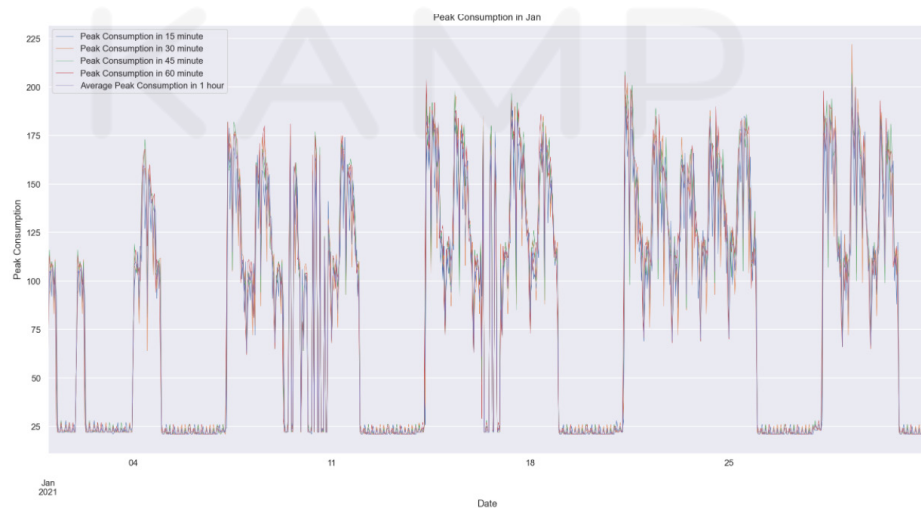

```
df["2021"].plot(linewidth = 0.5);
plt.title("Peak Consumption in 2021")
plt.xlabel("Month")
plt.ylabel("Peak Consumption")
plt.show()
```



[코드 24] 2021년도 최대수요전력 값 시각화하기

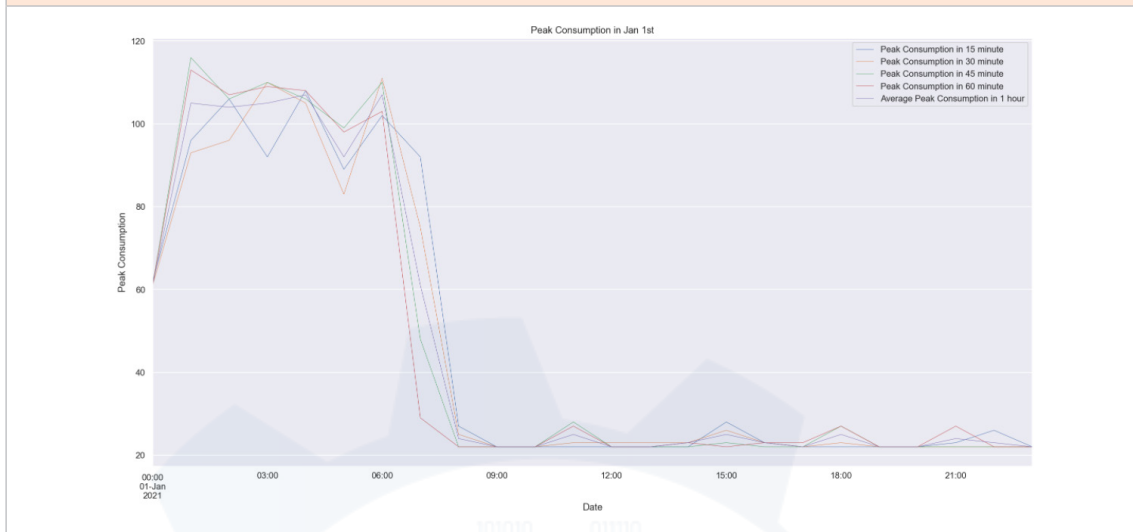
- 데이터 전처리 과정에서 인덱스를 날짜 및 시간으로 설정하였기에, 특정 일의 데이터 또는 특정 월의 데이터만을 가지고도 시각화를 할 수 있으며, 범위를 설정하여 시각화 또한 가능하다. 다음의 예시를 통해 살펴볼 수 있다.

```
df["2021-01"].plot(linewidth = 0.5)
plt.title("Peak Consumption in Jan")
plt.ylabel("Peak Consumption")
plt.show()
```



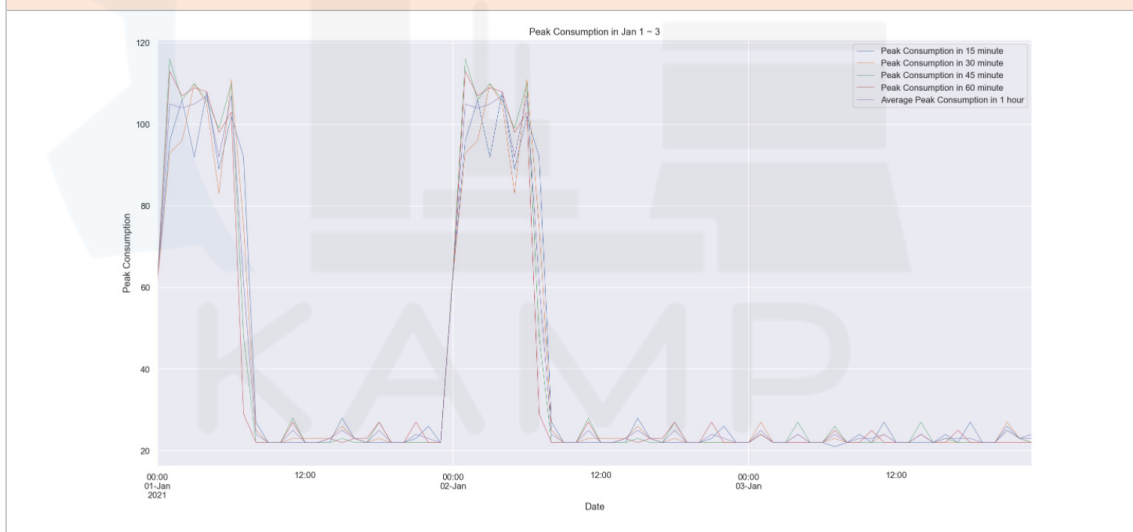
[코드 25] 2021년도 1월 최대수요전력 값 시각화하기


```
df["2021-01-01"].plot(linewidth = 0.5)
plt.title("Peak Consumption in Jan 1st")
plt.ylabel("Peak Consumption")
plt.show()
```



[코드 26] 2021년도 1월 1일 최대수요전력 값 시각화하기

```
df["2021-01-01" : "2021-01-03"].plot(linewidth = 0.5)
plt.title("Peak Consumption in Jan 1 ~ 3")
plt.ylabel("Peak Consumption")
plt.show()
```



[코드 27] 2021년도 1월 1일부터 3일까지의 최대수요전력 값 시각화하기

③-5. 주말 및 공휴일 데이터 구분

- 현재 저장된 파일에는 날짜와 시간과 관련된 인덱스, 그에 따른 최대수요전력이 표현되어 있다. 사실 날짜 데이터와 관련해서는 주말과 공휴일이라는 특수한 사실이 포함되어야 한다. 공장 운영에 있어서 평일과 휴일의 전력 사용량은 많은 차이가 발생한다. 그렇기에 해당 데이터에서도 주말과 휴일을 구분해주어야 한다.

- 해당 데이터의 Column에 공휴일 (Vacation)과 주말 (Weekend)을 추가하기 위해 해당 Column의 row 값에 0을 할당한다.

```
df["Vacation"] = np.zeros(len(df))
df["Weekend"] = np.zeros(len(df))
df.head()
```

Date	Peak Consumption in 15 minute	Peak Consumption in 30 minute	Peak Consumption in 45 minute	Peak Consumption in 60 minute	Average Peak Consumption in 1 hour	Vacation	Weekend
2021-01-01 00:00:00	62.0	61.0	61.0	61.0	61.25	0.0	0.0
2021-01-01 01:00:00	96.0	93.0	116.0	113.0	104.50	0.0	0.0
2021-01-01 02:00:00	106.0	96.0	106.0	107.0	103.75	0.0	0.0
2021-01-01 03:00:00	92.0	110.0	110.0	109.0	105.25	0.0	0.0
2021-01-01 04:00:00	108.0	105.0	106.0	108.0	106.75	0.0	0.0

[코드 28] 변수 추가 및 값에 0 할당하기

- 현재 Date로 표현된 인덱스로 요일을 파악하기 위해서는 'dayofweek' 함수를 이용한다. 해당 함수는 요일을 숫자로 표현해주는데 요일에 해당하는 '월화수목금토일'을 숫자 0~6으로 반환해준다. 하기의 예시를 통해 해당 내용을 확인할 수 있다.

```
df["2021-01-01"].index.dayofweek
```

```
Int64Index([4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4],
            dtype='int64', name='Date')
```

[코드 29] dayofweek를 활용한 요일 값 확인하기

- 실제 2021년 1월 1일은 금요일로, 'dayofweek' 함수에서 금요일은 4에 해당하므로 4가 출력된 것을 확인할 수 있다.
- 그래서 'dayofweek' 함수를 이용하게 된다면 토요일과 일요일에는 각각 5와 6이 할당되는 것을 확인할 수 있다. 전 단계에서 생성했던 주말 (Weekend) Column의 값에 조건을 부여하여 5와 6에 해당하는 요일에는 값으로 1을 부여하도록 설정한다.
- 즉, 다음과 같이 'dayofweek'의 값이 5이거나 6일 때에는 Weekend라는 Column에 1을 부여한다는 의미이다. 여기에서 '|'는 'or'을 의미한다.

```
df[Weekend][df.index.dayofweek==5][df.index.dayofweek==6] = 1
df
```

Date	Peak Consumption in 15 minute	Peak Consumption in 30 minute	Peak Consumption in 45 minute	Peak Consumption in 60 minute	Average Peak Consumption in 1 hour	Vacation	Weekend
2021-01-03 00:00:00	22.0	22.0	22.0	22.0	22.00	0.0	1.0
2021-01-03 01:00:00	24.0	27.0	24.0	24.0	24.75	0.0	1.0
2021-01-03 02:00:00	22.0	22.0	22.0	22.0	22.00	0.0	1.0
2021-01-03 03:00:00	22.0	22.0	22.0	22.0	22.00	0.0	1.0
2021-01-03 04:00:00	22.0	24.0	27.0	22.0	23.75	0.0	1.0
2021-01-03 05:00:00	22.0	22.0	22.0	22.0	22.00	0.0	1.0
2021-01-03 06:00:00	22.0	22.0	22.0	22.0	22.00	0.0	1.0
2021-01-03 07:00:00	21.0	23.0	26.0	25.0	23.75	0.0	1.0
2021-01-03 08:00:00	22.0	22.0	22.0	22.0	22.00	0.0	1.0
2021-01-03 09:00:00	24.0	22.0	22.0	22.0	22.50	0.0	1.0
2021-01-03 10:00:00	22.0	22.0	22.0	25.0	22.75	0.0	1.0
2021-01-03 11:00:00	27.0	24.0	22.0	22.0	23.75	0.0	1.0

[코드 30] 토요일 및 일요일에 값 1 할당하기

- 주말 뿐만 아니라 공휴일 또한 주말과 유사한 방법으로 값을 할당한다. 단, 주말의 경우에는 토요일과 일요일로 'dayofweek' 함수를 사용하여 쉽게 값을 할당할 수 있지만, 공휴일의 경우에는 각 나라마다 다르기 때문에 해당 함수를 사용할 수가 없다. 또한, 우리나라의 경우 추석이나 설의 경우에는 음력 날짜를 사용하기 때문에 실제 쉬는 날은 매년 다르게 설정될 수 있다.
- 그렇기에 본 연구에서는 2021년 9월 14일까지 데이터 중, 공휴일을 미리 정해서 연구를 진행하였다.
- 실제 2021년도 주말을 제외한 공휴일을 확인해보니, 다음과 같이 확인되어 holidays라는 변수 안에 해당 날짜들을 선언하였다.

```
holidays = ["2021-01-01", "2021-02-11", "2021-02-12", "2021-03-01", "2021-05-05", "2021-05-19", "2021-08-16"]
```

[코드 31] 공휴일 변수 선언하기

- 'iterrows()' 함수를 사용하여 반복문으로 해당 날짜들에 1이라는 값을 할당할 수 있다. 여기서 str(gun)[:10]의 값이 holidays라는 변수 안에 있는 날짜일 경우 Vacation 변수에 1이라는 값을 부여한다는 의미이다.

```
for gun,j in df.iterrows():
    if str(gun)[:10] in holidays:
        df["Vacation"].loc[gun] =1
```

[코드 32] 반복문을 활용한 값 부여하기

- 실제 데이터를 확인해보면 다음과 같이 1이라는 값이 할당된 것을 확인할 수 있다.

df.head(5)

Date	Peak Consumption in 15 minute	Peak Consumption in 30 minute	Peak Consumption in 45 minute	Peak Consumption in 60 minute	Average Peak Consumption in 1 hour	Vacation	Weekend
2021-01-01 00:00:00	62.0	61.0	61.0	61.0	61.25	1.0	0.0
2021-01-01 01:00:00	96.0	93.0	116.0	113.0	104.50	1.0	0.0
2021-01-01 02:00:00	106.0	96.0	106.0	107.0	103.75	1.0	0.0
2021-01-01 03:00:00	92.0	110.0	110.0	109.0	105.25	1.0	0.0
2021-01-01 04:00:00	108.0	105.0	106.0	108.0	106.75	1.0	0.0

[코드 33] 공휴일에 값 1 할당한 결과 확인하기

- 데이터에 이상이 없음을 확인 후, 다시 해당 파일을 새롭게 저장한다.

```
df.to_csv("daily_data_seasonality.csv", index =True)
```

[코드 34] csv 파일 저장하기

③-6. Window Features 생성

- 시계열 데이터를 학습하기 위해서 Window를 얼마나 가져갈 것인지 결정하는 것이 중요하다.
- 여기서 Window란 시간 단위로 생각하면 된다. 1시간 단위로 학습을 할 것인지, 1일 단위로 할 것인지, 1주일 단위로 할 것인지 결정하는 것으로 이해하면 된다.
- 본 연구에서는 1주일 단위로 지정과 동시에 하루가 24시간으로 구성되어 있기 때문에 $7 * 24 = 168$ 로 결정하였다.
- 168로 Window를 설정하였을 때, 현재의 데이터가 6,168개이기 때문에 168단위로 168부터 6048까지 Window를 설정할 수 있다. 이에 생성될 수 있는 다양한 길이의 Window features 중 본 가이드북에서는 168이라는 Window Feature를 가진 데이터를 사용한다.

- Window features를 생성하기 위해서 3-5의 최종 단계에서 저장한 데이터 파일을 불러온다.

```
daily_data = pd.DataFrame(pd.read_csv("daily_data_seasonality.csv", header = 0, index_col = 0, parse_dates = True, squeeze = True))
```

[코드 35] 파일 불러오기

- 해당 가이드북에서는 RNN을 사용하여 15분 최대수요전력을 예측하는 것을 보여주 고자 한다. 30분, 45분, 60분 최대수요전력은 같은 코드 구조로 구현할 수 있기 때 문이다.
- 하여 다음과 같이 데이터에 변수를 할당한다.

```
daily_data = daily_data[["Peak Consumption in 15 minute"]]  
daily_data
```

Peak Consumption in 15 minute	
Date	
2021-01-01 00:00:00	62
2021-01-01 01:00:00	96
2021-01-01 02:00:00	106
2021-01-01 03:00:00	92
2021-01-01 04:00:00	108
...	...
2021-09-14 19:00:00	152
2021-09-14 20:00:00	124
2021-09-14 21:00:00	134
2021-09-14 22:00:00	100
2021-09-14 23:00:00	117

6168 rows × 1 columns

[코드 36] 특정 변수값 데이터만 호출하기

- 각 시간대의 15분 최대수요전력을 가지고 Window features와 미리 예측하고자 하 는 시간 범위를 설정한다. 인덱스 시간 기준을 1시간 단위로 설정을 하였기에 다음 과 같이 변수를 선언하고 값을 할당한다.

```
n_times_advance = 1  
n_times_windows = 168
```

[코드 37] Window features 변수 선언하기

- 상기에서 선언한 시간 범위와 Window feature를 활용하여 학습에 이용할 Window features 변수들을 생성한다. 이때 반복 구문인 for 문을 사용하여 변수를 빠르게 생성하는 것이 가능하다.
- 이 때 사용할 반복문은 다음과 같이 구성되어 있다. 'range()' 함수를 사용하여 기존에 설정한 사전에 예측하고자 하는 시간 범위와 Window features의 범위의 숫자에서 변수를 생성하도록 for 문을 구성한다. 즉 여기서 range() 함수의 내용을 풀어보면 range(1, 169)와 같은 의미이며, k는 1부터 168까지의 숫자 범위에 있다고 보면 된다. 참고로 range() 함수 내에서 마지막에 선언된 숫자는 범위에 포함되지 않는다.

```
for k in range(n_times_advance, n_times_advance + n_times_windows):
    daily_data["Peak Consumption in 15 minute_t-%i" % k] = np.zeros(len(daily_data["Peak Consumption in 15 minute"]))
daily_data
```

Date	Peak Consumption in 15 minute	Peak Consumption in 15 minute_t-1	Peak Consumption in 15 minute_t-2	Peak Consumption in 15 minute_t-3	Peak Consumption in 15 minute_t-4	Peak Consumption in 15 minute_t-5	Peak Consumption in 15 minute_t-6	Peak Consumption in 15 minute_t-7	Peak Consumption in 15 minute_t-8	Peak Consumption in 15 minute_t-9	...
2021-01-01 00:00:00	62.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...
2021-01-01 01:00:00	96.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...
2021-01-01 02:00:00	106.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...
2021-01-01 03:00:00	92.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...
2021-01-01 04:00:00	108.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...
...	...	...	...	...	...	...	...	...	...	...	...
2021-09-14 19:00:00	152.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...
2021-09-14 20:00:00	124.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...
2021-09-14 21:00:00	134.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...
2021-09-14 22:00:00	100.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...
2021-09-14 23:00:00	117.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...

6168 rows x 169 columns

[코드 38] 시간 흐름에 따른 변수 선언 및 값 0 할당하기

- 새롭게 추가할 변수의 이름으로는 현재 기준에서 선언한 Window features의 내용을 반영할 수 있도록 't-반복 k의 범위'를 추가적으로 명시하여 생성한다. 또한, 새롭게 생성된 변수에는 현재 데이터에서 가지고 있는 row의 수 6,168개에 모두 0을 할당한다. 다음과 같이 선언 후 실행을 하면, 하기와 같이 새롭게 생성된 변수의 이름과 값이 선언한대로 할당된 것을 확인할 수 있다.
- 새롭게 추가한 Column의 값에 0 대신 실제 해당하는 최대수요전력의 값을 할당하여야 한다. 그러기 위해 추가로 반복 구문을 통한 작업이 필요하다.

```
for i in range(n_times_advance + n_times_windows, len(daily_data["Peak Consumption in 15 minute"])):
    for j in range(n_times_advance, n_times_advance + n_times_windows):
        daily_data["Peak Consumption in 15 minute_t-%i" % j][i] = daily_data["Peak Consumption in 15 minute"][i-j]
```

[코드 39] 해당 변수에 맞는 값 할당하기

- 상기의 코드를 위에서부터 하나씩 살펴보면 다음과 같다.

```
for i in range(n_times_advance + n_times_windows, len(daily_data["Peak Consumption in 15 minute"])):

```

[코드 39-1] 반복문의 범위 설정하기 1

- ‘n_times_advance’는 사전에 정의해 놓은 ‘1’이라는 값을 갖고 있다. ‘n_times_window’는 ‘168’이라는 값을 갖는다. ‘len(daily_data[‘Peak Consumption in 15 minute’])’은 row의 개수에 해당하는 ‘6168’이라는 값을 가진다. 즉, range(169, 6168)의 범위에 있는 정수가 i에 할당되어 반복한다.

```
for j in range(n_times_advance, n_times_advance + n_times_windows):

```

[코드 39-2] 반복문의 범위 설정하기 2

- 해당 구문은 첫 번째 반복문 안에 반복문이 들어가 있는 구조로 이중 반복이라는 것을 알 수 있다. 즉, 해당 반복문은 j라는 변수에 range(1, 169) 범위의 정수값이 할당되어 반복된다. 즉 해당 이중 반복 구조는 다음과 같은 순서로 작동한다. 예를 들어, i에 169가 할당이 되면, j에 1부터 168까지 순차적으로 반복을 한다. 해당 Cycle이 종료되면 i에는 그 다음의 값인 170이 할당되고 다시 j에 값이 순차적으로 할당된다. 같은 작업이 6167까지 반복하고 종료한다.

```
daily_data["Peak Consumption in 15 minute_t-%i" % j][i] = daily_data["Peak Consumption in 15 minute"][i-j]
```

[코드 39-3] 반복문을 통한 결과값 할당하기

- 마지막으로 상기 반복 과정에서 다음과 같이 Column에 변수를 할당함으로 반복문 과정에서 생성되는 결과값을 할당한다.
- 상기의 과정을 진행 후, Window Features를 반영한 데이터를 생성해야 한다. 사전에 정의한 예측하고자 하는 시간 범위와 Window Features를 반영하여 row 169 줄에 해당하는 데이터부터 할당한다.


```
daily_data = daily_data.iloc[n_times_advance + n_times_windows:]
```

[코드 40] 각 row에 맞는 값 할당하기

- 순환 신경망 학습을 위해서 해당 데이터의 값을 가지고 있는 csv 파일을 생성한다. 마지막으로 해당 데이터의 값을 확인하면 다음과 같이 이전 시간대들의 최대수요전력 값이 각 열에 할당된 것을 확인할 수 있다.

```
daily_data.to_csv("daily_data_seasonality_168.csv", index = True)
daily_data.head(3)
```

Date	Peak Consumption in 15 minute	Peak Consumption in 15 minute_t-1	Peak Consumption in 15 minute_t-2	Peak Consumption in 15 minute_t-3	Peak Consumption in 15 minute_t-4	Peak Consumption in 15 minute_t-5	Peak Consumption in 15 minute_t-6	Peak Consumption in 15 minute_t-7	Peak Consumption in 15 minute_t-8	Peak Consumption in 15 minute_t-9	...
2021-01-08 01:00:00	87	64.0	110.0	84.0	110.0	101.0	141.0	154.0	123.0	158.0	...
2021-01-08 02:00:00	104	87.0	64.0	110.0	84.0	110.0	101.0	141.0	154.0	123.0	...
2021-01-08 03:00:00	90	104.0	87.0	64.0	110.0	84.0	110.0	101.0	141.0	154.0	...

3 rows x 169 columns

[코드 41] Window Feature가 168인 csv 파일 저장 및 값 확인하기

[단계 ④] 훈련/테스트 데이터 분리

- 순환 신경망 모델의 데이터를 훈련하기 위해서는 데이터를 적절히 분리하는 작업이 필요하다. 파이썬에서 제공하는 라이브러리를 활용하여 훈련 및 테스트 데이터를 분리한다.

```
import math
from sklearn.model_selection import train_test_split
```

[코드 42] 필요한 패키지 불러오기 2

- 훈련 데이터는 학습을 위해서 사용되는 데이터이며, 평가 데이터는 학습 후 성능 평가를 위한 데이터이다.
- 훈련 및 평가 데이터를 분리하기 위해, 필요한 데이터를 호출한다. 이 때 사용되는 데이터는 [단계 ③]에서 최종적으로 생성한 “daily_data_seasonality_168.csv”를 사용한다.

```
daily_data_168 = pd.read_csv("daily_data_seasonality_168.csv",
                             header=0, index_col=0, parse_dates = True, squeeze = True)
daily_data_168.head(3)
```

Date	Peak Consumption in 15 minute	Peak Consumption in 15 minute_t-1	Peak Consumption in 15 minute_t-2	Peak Consumption in 15 minute_t-3	Peak Consumption in 15 minute_t-4	Peak Consumption in 15 minute_t-5	Peak Consumption in 15 minute_t-6	Peak Consumption in 15 minute_t-7	Peak Consumption in 15 minute_t-8	Peak Consumption in 15 minute_t-9	...
2021-01-08 01:00:00	87	64.0	110.0	84.0	110.0	101.0	141.0	154.0	123.0	158.0	...
2021-01-08 02:00:00	104	87.0	64.0	110.0	84.0	110.0	101.0	141.0	154.0	123.0	...
2021-01-08 03:00:00	90	104.0	87.0	64.0	110.0	84.0	110.0	101.0	141.0	154.0	...

[코드 43] 데이터 불러오기

- 훈련 및 평가 데이터를 구분하는 방법은 다양하다. 보통 데이터의 비율을 7:3으로 맞추어 활용하는 경우가 많다. 해당 분석에서는 언급한 방법 대신 날짜 정보를 활용하여 훈련 및 평가 데이터를 분리한다.
- 본 분석에서는 21년도 9월 1일부터 9월 14일까지의 15분 최대수요전력을 예시로 보겠다. 9월 1일부터 9월 14일까지 총 개수 $24 * 14 = 336$ 개의 최대수요전력을 예측하는 것이다. 그렇기에 훈련 데이터의 숫자는 336개를 제외한 모든 날짜 데이터가 훈련 데이터로 설정된다.

```
length_train_168 = len(daily_data_168) - 336
```

[코드 44] 훈련 데이터 선언하기

- len() 함수를 사용하면 현재 데이터의 개수를 확인할 수 있다. 여기에 336개를 제외하면 훈련 데이터의 개수를 확인할 수 있다. 하여 훈련 데이터의 개수는 5,663개라는 것을 확인할 수 있다.
- 학습을 위해서 기존의 데이터를 가지고 사용하는 것도 좋지만, 만약 데이터의 값이 너무 크거나 혹은 작은 경우에 알고리즘 학습 과정에서 0으로 수렴 또는 무한대로 발산할 수 있다. 그렇기에 데이터 스케일링 작업이 필요하다. 데이터 스케일링의 대표적인 방법들은 다음과 같다.
 - Standard Scaler: 각 feature의 평균을 0, 분산을 1로 설정한다. 모든 특성들이 같은 스케일을 갖게 된다.
 - Robust Scaler: Standard Scaler와 유사하지만, 평균과 분산 대신에 median과 quartile을 사용한다. 해당 방식은 이상치에 영향을 받지 않는다.
 - MinMax Scaler: 모든 feature가 0과 1 사이에 위치한다.

- Normalizer: 상기 3가지 방식은 각 Column들의 통계치를 이용하지만, 해당 방식은 row를 기준으로 정규화된다. 유클리드 거리가 1이 되도록 데이터를 조정한다.
- 본 분석에서는 MinMax Scaler를 사용하여 데이터 스케일링을 진행한다. MinMax Scaler를 사용하기 위해서 파이썬에서 제공하는 라이브러리를 활용한다.

```
from sklearn.preprocessing import MinMaxScaler
```

[코드 45] 필요한 패키지 불러오기 3

- MinMax Scaler를 사용할 수 있는 라이브러리를 통해 다음과 같이 스케일링을 진행할 수 있다.

```
scaler_168 = MinMaxScaler(feature_range = (0,1))  
scaler_168
```

[코드 46] MinMax Scaler를 통한 스케일링 진행하기

- 생성한 MinMax Scaler를 활용하여 모델 학습을 진행할 수 있다. 훈련 데이터를 통해 학습을 진행할 때, 'fit_transform()' 함수를 사용할 수 있다. 상기에서 선언한 MinMax Scaler와 'fit_transform()' 함수를 상기에서 선언한 변수에 적용한다. 적용하면 0과 1 사이의 값이 적용된 것을 확인할 수 있다.

```
daily_data_sc_168 = scaler_168.fit_transform(daily_data_168)  
daily_data_sc_168
```

```
array([[0.42028986, 0.30917874, 0.53140097, ..., 0.44444444, 0.51207729,  
        0.46376812],  
       [0.50241546, 0.42028986, 0.30917874, ..., 0.52173913, 0.44444444,  
        0.51207729],  
       [0.43478261, 0.50241546, 0.42028986, ..., 0.42995169, 0.52173913,  
        0.44444444],  
       ...,  
       [0.647343, 0.59903382, 0.73429952, ..., 0.56521739, 0.50241546,  
        0.64251208],  
       [0.48309179, 0.647343, 0.59903382, ..., 0.37681159, 0.56521739,  
        0.50241546],  
       [0.56521739, 0.48309179, 0.647343, ..., 0.48309179, 0.37681159,  
        0.56521739]])
```

[코드 47] MinMax를 활용한 데이터 스케일링 결과하기

- 'fit_transform()' 함수를 사용하여 선언된 변수를 활용하여 훈련 데이터의 값을 변경한다. 기존 훈련 데이터의 값은 실제 값들을 가지고 있어 학습 과정에서 0 또는 무한대의 값을 가질 수 있다. 또한, 기존의 훈련 데이터의 개수가 5,663개이기 때문에 이 점 또한 반영하여야 한다.

```
data_train_168 = daily_data_sc_168[:length_train_168]
```

[코드 48] 스케일링한 값을 가지고 훈련 데이터 선언하기

- 평가 데이터의 경우, 9월 1일부터 9월 14일의 데이터이며 총 336개의 데이터이다. 이 경우 또한 훈련 데이터와 마찬가지로 기존에 선언했던 데이터들을 활용하여 선언할 수 있다.

```
data_test_168 = daily_data_sc_168[-336:]
```

[코드 49] 스케일링한 값을 가지고 평가 데이터 선언하기

- 스케일링이 된 데이터를 활용하여 훈련 및 평가 데이터의 X와 Y를 선언한다.

```
data_train_Y_168 = data_train_168[:,0]
data_train_X_168 = data_train_168[:,1:]
data_test_Y_168 = data_test_168[:,0]
data_test_X_168 = data_test_168[:,1:]
```

[코드 50] 최종 훈련 및 평가 데이터 선언하기

- 마지막으로 순환 신경망 모델을 사용하기 위하여 훈련 데이터의 모양을 reshape를 해야 한다. 기존 data_train_X_168은 (5663, 168)의 모양을 가지고 있다. 현재 모양에서는 순환 신경망 모델 학습을 진행 시 오류가 발생할 수 있기에 해당 데이터의 모양을 변경한다. Window features는 168이라는 값을 가진다. 시퀀스 크기는 7, 각 시퀀스는 24개의 데이터를 가진다. 하여 모양은 (5663, 24, 7)로 reshape을 진행한다.

```
data_train_X_168 = data_train_X_168.reshape((length_train_168, 24, 7))
```

[코드 51] 최종 훈련 데이터 모양 변경하기

[단계 ⑤] 모델 구축 및 훈련

- 순환 신경망 모델을 구축하기 위해서 keras 라이브러리를 사용한다.

```
import keras
import tensorflow as tf
from keras.models import Sequential
from keras.layers import Dense, SimpleRNN, Flatten
```

[코드 52] 필요한 패키지 불러오기 4

- keras는 layer를 쌓아서 모델을 구축한다. keras가 가지고 있는 수많은 방법 중 본 분석에서는 단순 순환 신경망(Simple RNN)을 사용하여 실습한다. 이 중 순차 모델인 'Sequential()'을 해당 model에 호출하고, 해당 모델에 'add()' 방식으로 layer를 쌓아간다.

```
model_168 = Sequential()
model_168.add(SimpleRNN(50, input_shape = (24, 7)))
model_168.add(Flatten())
model_168.add(Dense(1))
opt = tf.keras.optimizers.Adam(learning_rate = 0.0001)
model_168.compile(loss = 'mean_squared_error', optimizer = opt)
model_168.fit(data_train_X_168, data_train_Y_168, epochs = 500)
model_168.summary()
```

```
Epoch 1/500
177/177 [=====] - 1s 2ms/step - loss: 0.8463
Epoch 2/500
177/177 [=====] - 0s 2ms/step - loss: 0.0893
Epoch 3/500
177/177 [=====] - 0s 2ms/step - loss: 0.0617
Epoch 4/500
177/177 [=====] - 0s 2ms/step - loss: 0.0484
Epoch 5/500
177/177 [=====] - 0s 2ms/step - loss: 0.0429
Epoch 6/500
177/177 [=====] - 0s 2ms/step - loss: 0.0396
Epoch 7/500
177/177 [=====] - 0s 2ms/step - loss: 0.0346
Epoch 8/500
177/177 [=====] - 0s 2ms/step - loss: 0.0326
Epoch 9/500
177/177 [=====] - 0s 3ms/step - loss: 0.0295
```

[코드 53] 단순 순환 신경망 학습하기

- 'SimpleRNN()'의 layer에는 output_shape와 input_shape 등을 입력한다. output_shape는 단순 순환 신경망 layer의 결과로 몇 개의 node가 나올지를 나타내는 부분이다. 해당 부분은 "hidden_size", 즉 은닉 상태(hidden state)의 크기를 정의한다. 다음 시점의 메모리 셀과 출력층으로 보내는 값의 크기라고도 한다. 본 분석에서는 50개의 hidden_size로 지정하였다. 이는 순환 신경망의 내부의 hidden state의 개수를 50개로 지정한다는 의미이며, 시계열 형식의 데이터는 50개의 hidden state를 가진 layer를 통과한다는 것을 의미한다.
- 이를 위해 데이터가 시계열 형식을 갖도록 변형이 필요하다. 해당 데이터는 일주일 단위로 분리하여 표현한 데이터이므로, (5663, 168)의 형태를 가지는데, 이를 하루 단위의 시간(24)으로 나누고 일주일을 하루 단위(7)로 나누는 방식으로 시계열의 정보를 추가하여 (5663, 24, 7)로 변형하였다. 시계열 데이터로 변경한 후 순환 신경망을 통과시킨 후 이를 바탕으로 결과값인 최대수요전력을 예측하기 위해 Flatten layer로 다차원의 정보들을 하나의 차원으로 합치는 작업을 한다. 그 후 모델의 마지막 층에는 최대수요 전력을 예측하는 Dense layer를 넣어주어야 한다. 본 분석에서는 최대수요전력이라는 결과 1가지를 예측하기에 1로 지정하였다.

- 또한, learning_rate = 0.0001로 학습하였으며, 학습하는 과정에서 MSE를 산출할 수 있도록 지정하였다. 훈련 데이터를 사용하며 epochs는 500으로 지정하였다. epochs = 500의 의미는 전체 데이터를 500번 사용해서 학습을 거치는 것을 의미한다. 해당 부분 또한 사용자가 적절하게 사용하면 된다. 그 후 모델 학습을 진행한다.
- 학습이 완료되면, 해당 모델을 바탕으로 예측을 진행한다. 예측을 할 때 사용하는 데이터는 테스트 데이터를 가지고 진행한다. 상기에서 훈련 데이터의 모양을 변경한 것처럼, 테스트 데이터의 모양 또한 변경해서 적용해야 한다.

```
preds_168 = model_168.predict(data_test_X_168.reshape((336, 24, 7)))
```

[코드 54] 예측 데이터 모양 변경하기

- 'concatenate()' 함수를 이용하여 예측한 데이터와 테스트 데이터를 연결한다. 'axis = 1'을 사용하여 좌에서 우로 진행되도록 한다. 만약 'axis = 0'을 사용하면 위에서 아래로 진행된다. 또한, 2차원 데이터의 경우, 'axis = 2'는 존재하지 않으므로 주의한다.

```
data_predicted_168 = np.concatenate((preds_168, data_test_X_168), axis=1)
data_predicted_168
```

```
array([[0.39528656, 0.49758454, 0.52657005, ..., 0.55555556, 0.43478261,
        0.39130435],
       [0.43341273, 0.40096618, 0.49758454, ..., 0.43478261, 0.55555556,
        0.43478261],
       [0.49717319, 0.47342995, 0.40096618, ..., 0.52657005, 0.43478261,
        0.55555556],
       ...,
       [0.597983, 0.59903382, 0.73429952, ..., 0.56521739, 0.50241546,
        0.64251208],
       [0.48385614, 0.647343, 0.59903382, ..., 0.37681159, 0.56521739,
        0.50241546],
       [0.53651911, 0.48309179, 0.647343, ..., 0.48309179, 0.37681159,
        0.56521739]])
```

[코드 55] Concatenate()를 사용한 값 선언하기

- 상기의 예측된 결과를 그대로 사용하여 실제 최대수요전력의 값과 비교하기에는 어려움이 존재한다. 그렇기에 'inverse_transform()' 함수를 사용하여 다시 스케일링한 데이터를 실제 값과 비교할 수 있도록 변환해준다. 하기 부분을 살펴보면 실제 값과 비교할 수 있도록 값이 변경된 것을 확인할 수 있다.

```
data_predicted_168 = scaler_168.inverse_transform(data_predicted_168)
data_predicted_168
```

```
array([[ 78.21442482, 103.      , 109.      , ..., 115.      ,
         90.      , 81.      , ],
       [ 94.10961038, 83.      , 103.      , ..., 90.      ,
         115.      , 90.      , ],
       [119.94822514, 98.      , 83.      , ..., 109.      ,
         90.      , 115.      , ],
       ...,
       [148.84059656, 124.      , 152.      , ..., 117.      ,
         104.      , 133.      , ],
       [ 91.71833897, 134.      , 124.      , ..., 78.      ,
         117.      , 104.      , ],
       [135.05842602, 100.      , 134.      , ..., 100.      ,
         78.      , 117.      , ]])
```

[코드 56] inverse_transform()을 활용한 값 변환하기

- 상기 방법으로 생성된 예측값을 가진 데이터 파일을 새로 저장한다. 현재 본 분석에서 예측하고자 하는 부분은 9월 1일부터 9월 14일의 15분 최대수요전력이다. 그렇기에 데이터 전처리 과정에서 살펴보았듯이 인덱스를 날짜 기준으로 설정한다. 그런 다음, 하기와 같이 작성하면 해당 데이터 값을 담은 파일이 저장된다.

```
index_column = pd.date_range(start='2021-09-01 00:00', end='2021-09-14 23:00', freq='H')
best_pred_168 = pd.DataFrame(data=data_predicted_168[:,0],
                             index=index_column)
best_pred_168.rename(columns={0: "forecasting"}, inplace=True)
best_pred_168.to_csv("the_best_rnn_pred_168.csv", index=True)
```

[코드 57] 예측값 csv 파일로 저장하기

[단계 ⑥] 분석 시각화

- 실제 값과 예측된 결과값을 직관적으로 비교하기 쉽게 그래프로 시각화를 진행한다. 본 분석에서는 실제 해당 시간대의 최대수요전력의 실제값과 예측값을 가지고 시각화를 진행한다. 실제 만들어 놓은 변수에서 비교하고자 하는 실제 특정 값을 하나의 변수로 생성한다.

```
data_original = daily_data[["Peak Consumption in 15 minute"]]
```

[코드 58] 원본 데이터 선언하기

- 단순 순환 신경망을 활용하여 생성한 결과값 파일의 해당 변수를 호출한다.


```
forecast = pd.read_csv('./the_best_rnn_pred_168.csv')['forecasting']
forecast
```

```
0      69.591321
1      96.997795
2     108.466863
3      93.757849
4     111.239161
...
331    152.782590
332    135.038093
333    132.614730
334    112.592509
335    115.974424
Name: forecasting, Length: 336, dtype: float64
```

[코드 59] 예측 데이터 호출하기

- 예측한 값의 결과가 9월 1일부터 14일까지의 최대수요전력이기에 실제 비교하고자 하는 값 또한 해당 일치해야 한다. 하여 9월 1일부터 14일까지의 row가 336개로 구성되어 있기에 하기와 같이 변수를 선언한다.

```
compare_result = pd.DataFrame(np.array(data_original['Peak Consumption in 15 minute'][-336:]), columns=['Original 15 minute'])
```

[코드 60] row에 맞는 값 가져오기

- 선언한 변수에 예측한 값을 추가하여 비교하고자 하는 실제값과 예측값을 하나의 데이터셋으로 만들어준다.

```
compare_result['forecasting'] = forecast
compare_result
```

	Original 15 minute	forecasting
0	83.0	73.768862
1	98.0	97.364572
2	121.0	112.381218
3	90.0	104.187840
4	114.0	115.029272
...	...	...
331	152.0	153.254105
332	124.0	123.589537
333	134.0	137.689810
334	100.0	103.945556
335	117.0	117.025648

336 rows x 2 columns

[코드 61] 예측값과 실제값을 포함하나 데이터셋 구성하기

- 기존에 살펴보았던 것처럼, 해당 인덱스를 나타내고자 하는 날짜 데이터로 변경한다.
- 변경한 인덱스를 사용하여, 값을 비교하고자 생성해 놓은 변수에 해당 인덱스를 'Date'로 지정한다.

```
index_column = pd.date_range(start='2021-09-01 00:00', end='2021-09-14 23:00', freq='H')
index_column
```

```
DatetimeIndex(['2021-09-01 00:00:00', '2021-09-01 01:00:00',
                '2021-09-01 02:00:00', '2021-09-01 03:00:00',
                '2021-09-01 04:00:00', '2021-09-01 05:00:00',
                '2021-09-01 06:00:00', '2021-09-01 07:00:00',
                '2021-09-01 08:00:00', '2021-09-01 09:00:00',
                ...,
                '2021-09-14 14:00:00', '2021-09-14 15:00:00',
                '2021-09-14 16:00:00', '2021-09-14 17:00:00',
                '2021-09-14 18:00:00', '2021-09-14 19:00:00',
                '2021-09-14 20:00:00', '2021-09-14 21:00:00',
                '2021-09-14 22:00:00', '2021-09-14 23:00:00'],
                dtype='datetime64[ns]', length=336, freq='H')
```

[코드 62] date_range()를 활용한 인덱스 변경하기

```
compare_result.index = index_column
compare_result.index.names = ['Date']
compare_result.head()
```

	Original 15 minute	forecasting
Date		
2021-09-01 00:00:00	83.0	73.768862
2021-09-01 01:00:00	98.0	97.364572
2021-09-01 02:00:00	121.0	112.381218
2021-09-01 03:00:00	90.0	104.187840
2021-09-01 04:00:00	114.0	115.029272

[코드 63] 날짜 인덱스 설정하기

- 'plot()' 함수를 이용하여 실제값과 예측값을 시각화하여 비교한다.

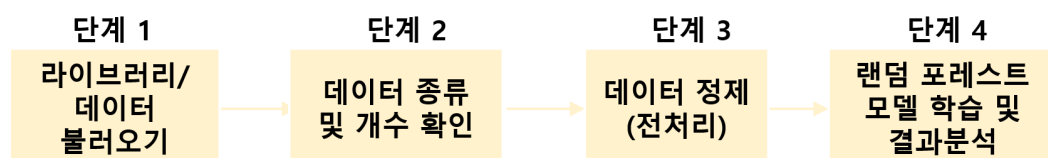
```
compare_result["2021-09"].plot(linewidth=0.5)
plt.title("Peak Consumption in Sep")
plt.ylabel("Peak Consumption")
plt.show()
```



[코드 64] 최대수요전력 실제값 및 예측값 시각화하기

- 일자 기준으로 실제값과 예측값을 비교할 경우에는, 해당 최대수요전력이 대략적으로 일치하는 것을 확인할 수 있다.
- 하지만 실제 시간별 15분당 최대수요전력을 비교하면, 주말 데이터의 경우 제대로 반영하지 않는 것을 확인하였다.
- 하여 이러한 문제를 해결하기 위해 랜덤 포레스트 기법을 사용하여 최대수요전력 예측 및 자원 최적화 분석을 추가로 진행한다.

3) 분석 단계별 프로세스 - 랜덤 포레스트 알고리즘



[그림 15] 지도 학습 적용을 위한 랜덤 포레스트 모델 구현 과정

[단계 ①] 라이브러리/데이터 불러오기

①-1. 필요 데이터 다운로드 및 불러오기(import) 가이드

```
import os
import sklearn
```

[코드 65] 필요한 패키지 불러오기 5

- 랜덤 포레스트 회귀 모델 기반의 최대수요전력 예측 알고리즘 방식이므로 정수형 변수가 존재하는 데이터를 사용한다. 이 때 사용할 데이터는 순환신경망 모델 전처리 시, 생성해 놓은 “df_frame” 변수를 사용한다.

df_frame

	날짜	시간	15분	30분	45분	60분	평균	생산량	기온	풍속	습도	강수량	전기요금(계절)	day	d	m	공장인원	인건비
0	20210101	0	62	61	61	61	61	0	-3.2	2.4	71	0.0	109.8	5	1	1	0.000000	1.5
1	20210101	1	96	93	116	113	105	0	-4.5	1.5	77	0.0	109.8	5	1	1	0.000000	1.5
2	20210101	2	106	96	106	107	104	0	-3.9	2.6	58	0.0	109.8	5	1	1	0.000000	1.5
3	20210101	3	92	110	110	109	105	0	-4.1	2.6	56	0.0	109.8	5	1	1	0.000000	1.5
4	20210101	4	108	105	106	108	107	0	-4.6	2.6	60	0.0	109.8	5	1	1	0.000000	1.5
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
6163	20210914	19	152	151	171	139	153	1497	21.7	3.6	85	9.4	167.2	2	14	9	2.442088	1.5
6164	20210914	20	124	130	128	130	128	45	22.2	4.2	78	9.4	167.2	2	14	9	0.087891	1.5
6165	20210914	21	134	130	125	124	128	149	22.2	4.3	76	9.4	167.2	2	14	9	0.290448	1.5
6166	20210914	22	100	109	120	114	111	66	22.0	2.5	79	9.4	167.2	2	14	9	0.148984	1.5
6167	20210914	23	117	119	112	91	110	405	22.0	2.5	79	9.4	167.2	2	14	9	0.922551	1.5

6168 rows x 18 columns

[코드 66] 순환 신경망에서 전처리한 df_frame의 전체 구성 확인하기

[단계 ②] 데이터 종류 및 개수 확인

②-1. 데이터 종류 및 개수 확인 가이드

- 어떤 데이터 종류가 있는지 간략히 확인하는 방법에는 ‘head()’를 사용한다. 이는 위에서 5개의 rows를 읽어온다.

df_frame.head()

	날짜	시간	15분	30분	45분	60분	평균	생산량	기온	풍속	습도	강수량	전기요금(계절)	day	d	m	공장인원	인건비
0	20210101	0	62	61	61	61	61	0	-3.2	2.4	71	0.0	109.8	5	1	1	0.000000	1.5
1	20210101	1	96	93	116	113	105	0	-4.5	1.5	77	0.0	109.8	5	1	1	0.000000	1.5
2	20210101	2	106	96	106	107	104	0	-3.9	2.6	58	0.0	109.8	5	1	1	0.000000	1.5
3	20210101	3	92	110	110	109	105	0	-4.1	2.6	56	0.0	109.8	5	1	1	0.000000	1.5
4	20210101	4	108	105	106	108	107	0	-4.6	2.6	60	0.0	109.8	5	1	1	0.000000	1.5
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
6163	20210914	19	152	151	171	139	153	1497	21.7	3.6	85	9.4	167.2	2	14	9	2.442088	1.5
6164	20210914	20	124	130	128	130	128	45	22.2	4.2	78	9.4	167.2	2	14	9	0.087891	1.5
6165	20210914	21	134	130	125	124	128	149	22.2	4.3	76	9.4	167.2	2	14	9	0.290448	1.5
6166	20210914	22	100	109	120	114	111	66	22.0	2.5	79	9.4	167.2	2	14	9	0.148984	1.5
6167	20210914	23	117	119	112	91	110	405	22.0	2.5	79	9.4	167.2	2	14	9	0.922551	1.5

6168 rows × 18 columns

[코드 67] head()를 사용한 데이터 구성 확인하기

- 'info()'를 사용하여, 전체적인 데이터의 구조 및 타입을 살핀다. 아래 그림에서 확인하듯 모든 변수명에 동일한 데이터 타입이 들어가 있는 것을 확인할 수 있으므로, 이는 일관성을 만족시킨다.

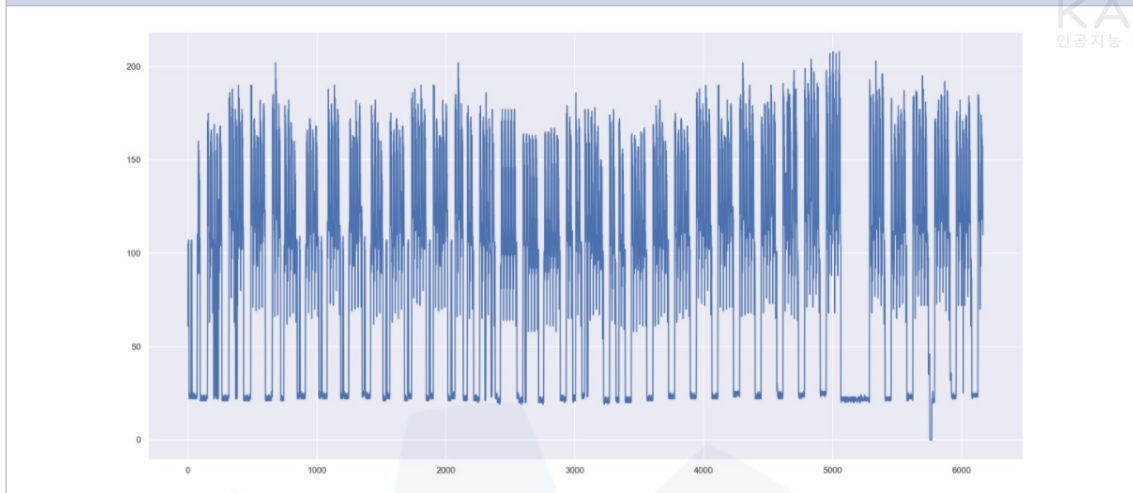
df_frame.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6168 entries, 0 to 6167
Data columns (total 18 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   날짜        6168 non-null   int64
1   시간        6168 non-null   int64
2   15분        6168 non-null   int64
3   30분        6168 non-null   int64
4   45분        6168 non-null   int64
5   60분        6168 non-null   int64
6   평균        6168 non-null   int64
7   생산량      6168 non-null   int64
8   기온        6168 non-null   float64
9   풍속        6165 non-null   float64
10  습도        6168 non-null   int64
11  강수량      6167 non-null   float64
12  전기요금(계절) 6168 non-null   float64
13  day         6168 non-null   int64
14  d           6168 non-null   int64
15  m           6168 non-null   int64
16  공장인원    6151 non-null   float64
17  인건비      6168 non-null   float64
dtypes: float64(6), int64(12)
memory usage: 867.5 KB
```

[코드 68] info()를 사용한 전체 데이터 구성 확인하기

- 본 분석에서 예측해야 할 것은 '15분', '30분', '45분', '60분'에 해당하는 최대수요 전력이다. 이들에 대한 평균을 낸 것이 '평균'에 해당하는 변수명이므로 해당 변수명을 구성을 확인해본다.

```
plt.plot(df_frame['평균'])
```



[코드 69] 시간당 '평균' 데이터 시각화하기

[단계 ③] 데이터 정제 (전처리)

③-1. 불필요 데이터 제거 가이드

- 랜덤 포레스트 모델에서 '평균' 및 '날짜' 데이터는 사용하지 않으므로 제거한다.

```
df_frame.drop(['평균', '날짜'], axis=1, inplace=True)
df_frame
```

	시간	15분	30분	45분	60분	생산량	기온	풍속	습도	강수량	전기요금(계절)	day	d	m	공장인원	인건비
0	0	62	61	61	61	0	-3.2	2.4	71	0.0	109.8	5	1	1	0.000000	1.5
1	1	96	93	116	113	0	-4.5	1.5	77	0.0	109.8	5	1	1	0.000000	1.5
2	2	106	96	106	107	0	-3.9	2.6	58	0.0	109.8	5	1	1	0.000000	1.5
3	3	92	110	110	109	0	-4.1	2.6	56	0.0	109.8	5	1	1	0.000000	1.5
4	4	108	105	106	108	0	-4.6	2.6	60	0.0	109.8	5	1	1	0.000000	1.5
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
6163	19	152	151	171	139	1497	21.7	3.6	85	9.4	167.2	2	14	9	2.442088	1.5
6164	20	124	130	128	130	45	22.2	4.2	78	9.4	167.2	2	14	9	0.087891	1.5
6165	21	134	130	125	124	149	22.2	4.3	76	9.4	167.2	2	14	9	0.290448	1.5
6166	22	100	109	120	114	66	22.0	2.5	79	9.4	167.2	2	14	9	0.148984	1.5
6167	23	117	119	112	91	405	22.0	2.5	79	9.4	167.2	2	14	9	0.922551	1.5

6168 rows x 16 columns

[코드 70] drop()을 사용한 변수 제거하기

③-2. 결측치 값 처리

- 본 실습에서 사용되는 데이터의 결측치 값의 수치를 알기 위해 다음과 같이 코드를 구현하여 확인할 수 있다.

```
for col in df_frame.columns:
    null_count = 'column: {}10}\t Percent of NaN value: {:.2f}%'.format(col, 100 * (df_frame[col].isnull().sum() / df_frame[col].shape[0]))
    print(null_count)
```

```
column:      시간      Percent of NaN value: 0.00%
column:      15분      Percent of NaN value: 0.00%
column:      30분      Percent of NaN value: 0.00%
column:      45분      Percent of NaN value: 0.00%
column:      60분      Percent of NaN value: 0.00%
column:      생산량      Percent of NaN value: 0.00%
column:      기온      Percent of NaN value: 0.00%
column:      풍속      Percent of NaN value: 0.00%
column:      습도      Percent of NaN value: 0.00%
column:      강수량      Percent of NaN value: 0.00%
column:      전기요금(계절)      Percent of NaN value: 0.00%
column:      day      Percent of NaN value: 0.00%
column:      d      Percent of NaN value: 0.00%
column:      m      Percent of NaN value: 0.00%
column:      공장인원      Percent of NaN value: 0.00%
column:      인건비      Percent of NaN value: 0.00%
```

[코드 71] 입력 데이터 내부에 포함된 결측치의 비율 확인하기

- 확인된 데이터의 결측치가 존재하지 않음을 확인할 수 있다.

③-3. 데이터 정리

- 입력 데이터는 15분, 30분, 45분, 60분으로 이루어진 데이터이다. 이를 회귀 모델을 학습시키기 위해 다시 15분씩으로 나누어 데이터를 재구성한다.
- 기존의 6,168개의 데이터 개수가 4배로 늘어나므로, 새로운 입력 데이터인 `x_train` 과 `cost_list` `y_labels`의 길이는 24,672개가 된다. `y_labels`는 `x_train`의 현재 인덱스(index)의 다음 전기 사용량을 나타내는 데이터이다. 예를 들어 2021년 1월 1일 오전 11시 00분의 데이터가 `x[0]`이라면, `y_labels[0]`은 2021년 1월 1일 오전 11시 15분이 되어야한다. 이를 위해 동일하게 추가된 `y_labels`의 첫 번째 위치의 데이터를 제거한다.
- 또한, `cost_list`의 경우, 인공지능 모델이 예측한 전기료를 바탕으로 최적화를 시행하는 변수들이므로 `drop` 함수를 사용하여, 변수명 '인건비', '전기요금(계절)', '공장 직원'의 변수에 대해 따로 분리하도록 한 후 첫 번째 정보를 제거한다.


```
label_cat_names = ['15분','30분','45분','60분']
or_cat_rm_list = ["인건비", "전기요금(계절)", "공장인원", "생산량"]
x_train = []
cost_list = []
for index in range(0, len(df_frame)):
    for col in label_cat_names:
        temp = list(df_frame.drop(or_cat_rm_list, axis=1).drop(label_cat_names, axis=1).iloc[0])
        temp.append(df_frame[col][index])
        cost_list.append(df_frame[or_cat_rm_list].iloc[index])
        x_train.append(temp)
y_labels = []
for index in range(0, len(df_frame)):
    for col in label_cat_names:
        y_labels.append(df_frame[col][index])
y_labels.pop(0)
x_train.pop(len(x_train)-1)
cost_list.pop(len(x_train)-1)
X = np.asarray(x_train)
C = np.asarray(cost_list)
y = np.asarray(y_labels)
```

[코드 72] 회귀 모델 학습을 위한 코드 구성하기

[단계 ④] 모델 구축 및 학습

④-1. 사용할 모델 구축, 학습 및 검증

- 모델 구성에 필요한 라이브러리들을 가져온다.

```
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, mean_absolute_error
from sklearn.utils.random import sample_without_replacement
```

[코드 73] 필요한 패키지 불러오기 6

- 본 모델 구성 전, 입력 데이터에서 학습 및 검증 데이터를 분리한다. 'train_test_split' 함수를 사용하여 랜덤으로 42번 섞은 뒤, 전체 데이터의 30%를 검증 데이터로, 70%를 훈련 데이터로 분리한다. 이 때 추가적으로 하이퍼 파라미터인 random_state를 고정하지 않는다면, 매번 학습 및 검증 데이터가 새롭게 분리된다. 하여 본 분석에서는 random_state는 42로 고정하였다.

```
x_train, x_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state = 42)
c_train, c_test, cy_train, cy_test = train_test_split(C, y, test_size=0.3, random_state = 42)
```

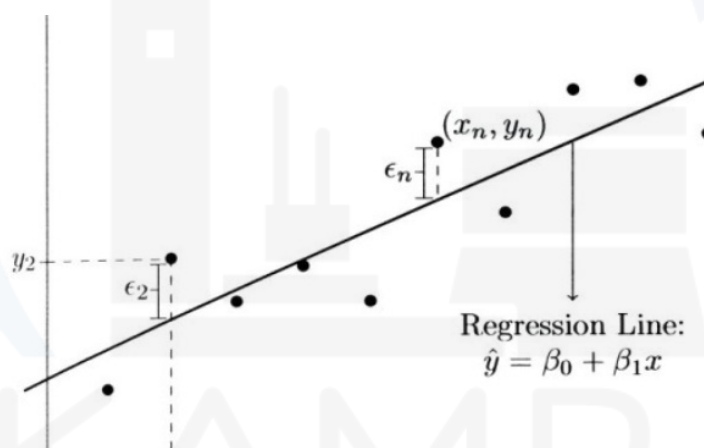
[코드 74] 학습 및 검증데이터 분리하기

- 본 분석에서는 최대수요전력을 예측하는 모델을 구축하므로 회귀 모델을 사용한다. 회귀 모델이란 ‘어떤 자료에 대해서 그 값에 영향을 주는 조건을 고려하여 구한 평균’이다. 통계학적인 관점에서 보면 모든 데이터는 아래와 같은 수식으로 표현할 수 있다고 가정한다.

$$y = h(x_1, x_2, x_3, \dots, x_k; \beta_1, \beta_2, \beta_3, \dots, \beta_k) + \epsilon$$

[그림 16] 회귀 모델 수식

- 이 함수는 어떤 조건($x_1, x_2, x_3, \dots$)이 주어지면 각 조건의 영향력($\beta_1, \beta_2, \beta_3, \dots$)을 고려하여 해당 조건에서의 평균값을 계산해 주는 것이다. 수식의 ϵ 는 ‘오차항’을 의미한다. 측정상의 오차나 모든 정보를 파악할 수 없는 점 등 다양한 현실적인 한계로 인해 발생하는 불확실성이 여기에 포함한다.



[그림 17] 회귀 모델 그래프

- 위의 결정 트리를 회귀 형태로 앙상블 기법을 사용하여 합친 모델이 바로 랜덤 포레스트 회귀 모델이다. 본 분석에서는 트리의 깊이를 20, 초기 파라미터 변수를 42 번으로 고정시킨다. 그 후 위에서 정의한 `x_train`과 `cost_list`를 재정의하고 이를 ‘fit()’ 함수를 이용하여 학습한다.

```
forest_reg = RandomForestRegressor(max_depth=20, random_state=42)
forest_reg.fit(x_train, y_train)
```

[코드 75] 랜덤 포레스트 모델 선언하기

- 본 분석에서 모델이 학습하면서 예측한 값에 대한 손실 값을 추정하기 위해 ‘평균 제곱근 오차(Mean squared error)’ 함수를 사용한다. 평균 제곱근 오차는 실제 정답 값과 모델이 예측한 값의 차이에 대해 제곱을 하여 평균을 계산하는 것이다. 수식으로는 다음과 같다.

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2$$

[그림 18] 평균 제곱근 오차 수식

- 다음의 코드로 간단히 구현할 수 있다. 학습된 모델이 추정한 값을 받기 위해선 ‘predict()’ 함수를 사용한다. 다음의 코드에서 x_train 값에 대한 모델의 추정치를 출력한다.

```
mean_squared_error(forest_reg.predict(x_train), y_train)
```

```
1 mean_squared_error(forest_reg.predict(x_train), y_train)
185.9751419346689
```

[코드 76] 예측 모델 추정치 출력하기

- 검증 데이터에 대한 평균 제곱 오차 값의 계산 코드와 결과는 다음과 같다.

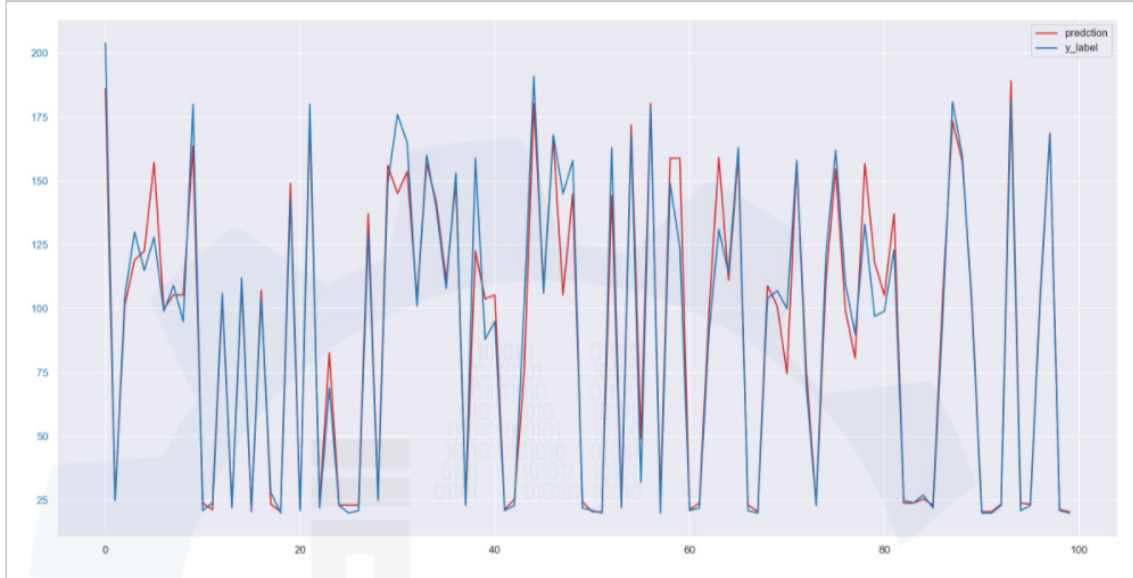
```
mean_squared_error(forest_reg.predict(x_test), y_test)
```

```
1 mean_squared_error(forest_reg.predict(x_test), y_test)
183.06180686559583
```

[코드 77] 검증 데이터의 평균 제곱 오차값 출력하기

- 실제 검증 데이터의 값과 모델이 예측한 수치의 차이의 값을 시각화한다. ‘sample_without_replacement()’ 함수를 이용하여 100개의 임의의 데이터를 따로 추출한다. 예측한 값에 대한 곡선의 색은 파란색으로 실제 정답에 해당하는 곡선의 색은 빨간색으로 설정한다.

```
prediction = forest_reg.predict(x_test)
predicted_sample_indexes = sample_without_replacement(len(prediction), 100)
color = 'tab:red'
plt.tick_params(axis='y', labelcolor=color)
plt.plot(prediction[predicted_sample_indexes], color=color)
color = 'tab:blue'
plt.plot(y_test[predicted_sample_indexes], color=color)
plt.tick_params(axis='y', labelcolor=color)
plt.legend(['prediction', 'y_label'])
plt.show()
```



[코드 78] 평균 제곱근 오차에 대한 그래프 시각화하기

④-2. 자원 최적화 및 분석

- ④-1에서 현재 주어진 값들로 다음에 소비될 최대수요전력을 예측할 수 있었다. 해당 분석에서는 주어진 최대수요전력을 이용하여, 공장에서 발생될 비용을 최소화 하는 변수들을 찾는 것을 목표로 한다. 이를 위해 목적함수와 제약조건들을 정의한다.
- 본 분석에서는 문제의 간소화를 위해 계절별 전기가격과 시간당 인건비를 최소화하는 목적함수 식을 사용하였다.

$$Cost = P_{human} + P_{electric}$$

[그림 19] 목적함수 수식화

- 목적함수를 구축하기 위해 추출하는 전기료의 값과 인건비에 관한 코드는 다음과 같다. 여기서 앞서 정의한 cost_list의 값들을 사용하는데, c_test의 경우 cost_list에서 검증 데이터의 인덱스에 맞추어 분리하였기 때문에 이름이 변경되었다. 최적

화를 시행하기 전, 생산량이 0이 아닌 값에 대해서 적용해야한다. 이는 어떤 생산량을 달성하기 위해 필요한 최소의 자원을 도출하는 과정이므로 반드시 목적값인 생산량은 0이 아닌 양수일 때의 시점을 사용하여야 한다.

```
def build_objective_function_parameters(prediction,x_test, c_test, index =0):
    total_elec_price = prediction[index] * c_test[index][1] /1000 + x_test[index][-1] *1.1 - x_test[index][-2] *1.1
    total_human_cost = c_test[index][2] *50 - x_test[index][-1] *1.1 + x_test[index][-2] *1.1
    return total_elec_price,total_human_cost
```

[코드 79] 전기료와 인건비에 대한 목적함수 구성하기

- 최적화를 위하여 최적화 툴을 사용한다.

```
from ortools.linear_solver import pywraplp
```

[코드 80] 필요한 패키지 불러오기 7

- 목적함수에서 값을 찾아야 할 변수들의 계수는 낮과 밤에 따라 달라진다. 예를 들어 상대적으로 낮에는 전기료가 비싸지만, 인건비는 저렴하다. 반대로 밤에는 전기료는 저렴하지만, 인건비가 높게 책정된다. 즉, 인건비와 전기료는 서로 반대의 특성을 가진다고 볼 수 있다. 그래서 해당 코드에서는 둘 중 하나의 값에 음의 값을 곱하였다. 또한, 목적함수를 최소화하는 것이 목적이기에 'SetMinimization()'을 사용한다.

```
def objective_function(solver,c_test,cx,cy,x,y,index=0):
    objective = solver.Objective()
    if(c_test[index][0]==1):
        objective.SetCoefficient(x, cx*2)
        objective.SetCoefficient(y, -cy)
    else:
        objective.SetCoefficient(x, cx//2)
        objective.SetCoefficient(y, -(cy)//2)
    objective.SetMinimization()
    return objective
```

[코드 81] SetMinimization()을 활용한 목적함수 구성하기

- 상기 목적함수에 맞춰 최적화된 값들을 구할 변수에 대한 제약조건들은 다음과 같이 정의한다. 전기료의 경우 낮에는 상대적으로 비싸지만, 밤의 경우에는 저렴하다. 인력의 경우 최대 75명까지 투입할 수 있는 것으로 정의하였다.

$$1 \leq P_{electric} \leq 2$$

$$1 \leq P_{human} \leq 75$$

[그림 20] 전기료 및 인력에 대한 제약조건

- 또한, 해당 변수들에 대해서 추가적인 제약이 없다면 최적의 답을 찾기에 어려움이 있기에 제약조건을 추가하여 전기료와 인건비가 가질 수 있는 최대 및 최소의 값에 대한 제약조건을 다음과 같이 정의하였다.

$$11 \leq P_{electric} * 2 + P_{human} \leq 95$$

[그림 21] 전기료와 인건비에 대한 제약 조건이 없을 때 가질 수 있는 최대값 제한

- 상기에서 정의한 목적함수와 제약조건을 바탕으로 최적화 함수 코드를 재정의한다.

```
def get_solution(pred,x_test,c_test,index = 118):
    solver = pywraplp.Solver.CreateSolver('GLOP')
    x = solver.NumVar(1, 2, 'x')
    y = solver.NumVar(1, 75, 'y')
    tep,thc=build_objective_function_parameters(pred,x_test, c_test, index)
    objective = objective_function(solver, c_test, tep, thc, x, y, index)
    objective = solver.Objective()
    constraint2 = solver.Constraint(11, 95)
    constraint2.SetCoefficient(x, 1)
    constraint2.SetCoefficient(y, 2)

    solver.Solve()

    opt_solution = tep * x.solution_value() - thc * y.solution_value()
    print("10이면 낮, 0이면 밤 : ", x.solution_value())
    print("공장인원 수 : ", y.solution_value())
    print("final solution: ", opt_solution)
    if x.solution_value() == 1:
        print("생산량이 ",c_test[index][-2],"일 때, 낮에 공장직원 ", y.solution_value(),"을 투입 시켜야한다.")
    else :
        print("생산량이 ",c_test[index][-2],"일 때, 밤에 공장직원 ", y.solution_value(),"을 투입 시켜야한다.")
```

[코드 82] 최적화 함수 구성하기

- 생산량이 0이 아닌 검증 데이터 중 하나를 가지고 분석한다. 본 분석에서는 검증 데이터인 `c_test`에 존재하는 데이터를 사용한다. 이 때 예시로 111번째에 위치하는 데이터를 사용한다.

```
c_test[111][-2]
```

```
0.17183770899999998
```

[코드 83] `c_test`의 111번째에서 생산량을 나타내는 값 출력하기

- '`get_solution()`' 함수를 통해 주어진 결과를 확인할 수 있다.

```
get_solution(prediction, x_test, c_test, 111)
```

```
1이면 낮, 0이면 밤 : 1.0
공장인원 수 : 5.0
final solution: 610.4238440032458
생산량이 0.17183770899999998 일 때, 낮에 공장직원 5.0 을 투입 시켜야한다.
```

[코드 84] `get_solution()` 함수를 통한 최적화 솔루션 확인하기

- 111번째 검증 데이터셋에 대해 `Solve` 함수를 통해 구해진 값은 다음과 같다. 이 변수들의 의미는 검증 데이터의 111의 상황의 경우 낮($x=1$)에 5($y=5$)명의 직원들을 고용하는 것이 전기료를 포함하여 공장을 운영하는 데에 있어 모든 비용을 최소화할 수 있다는 의미이다. 이 값을 가질 때 목적함수의 값 약 610의 값을 갖는다.

3. 유사 타 현장의『자원 최적화 AI 데이터셋』분석 적용

3.1 분석 배경 본 분석이 적용 가능한 제조현장 소개

- 제조 공장 뿐만 24시간 운영해야 하는 모든 산업 현장에 적용할 수 있다. 24시간 운영되는 산업 현장에서 생산량에 따른 전력 소비 데이터를 기록 및 수집이 쉽지는 않으나, 생산량에 소비되는 전력 소비 데이터의 패턴을 추출할 수 있을 만큼의 데이터가 수집이 된다면, 본 가이드 북에서 구축한 모델(순환 신경망, 랜덤 포레스트 회귀 모델)을 사용한 분석을 통하여, 효율적인 생산 계획을 구축할 수 있고 이를 통하여 생산 비용의 기대 값을 최소한으로 줄일 수 있다.

3.2 본 『자원 최적화 AI 데이터셋』분석을 원용하여 타 제조현장 적용시, 주요 고려사항

- 상기 데이터셋의 구조를 그대로 적용하기보다는 각 공정에 맞는 데이터를 수집하여 해당 모델을 적용해야 한다.
- 학습된 모델의 예측값을 사용하여 해당 모델을 적용하기 전에, 관련된 데이터가 무엇인지 정의를 하고 그에 맞는 데이터를 수집해야 한다.
- 본 분석에서 활용한 값은 다른 제조 현장과 맞지 않을 수 있기에 현장 전문가의 의견을 반영하여 적용 여부를 결정해야 한다.



3 부록 _ 분석환경 구축을 위한 설치 가이드

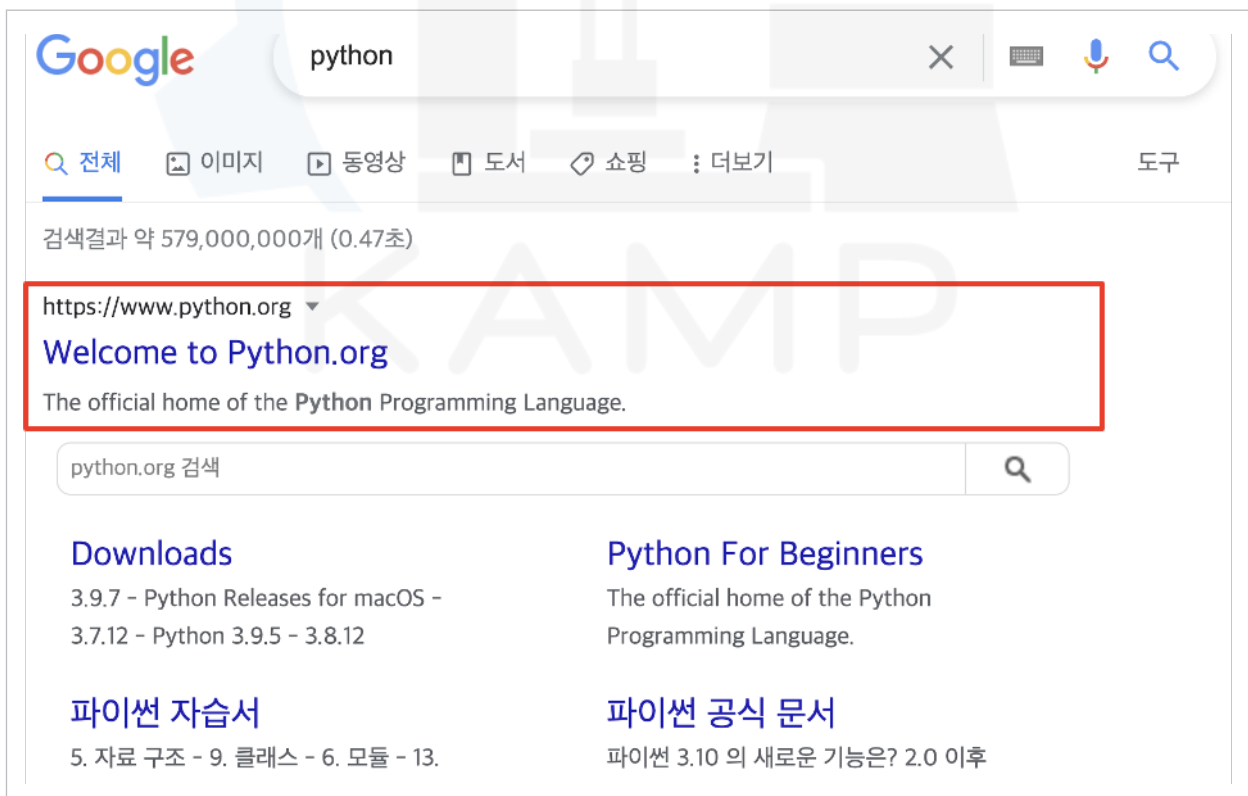
1. 파이썬(Python) 설치

파이썬은 컴퓨터 언어로서 최근 데이터 분석 및 AI 분석모델 개발 등에 널리 사용되는 도구이다. 다운로드 및 설치가 간편하고 활용도가 높은 파이썬을 로컬 컴퓨터에 설치하고 적용하는 방법을 안내한다.

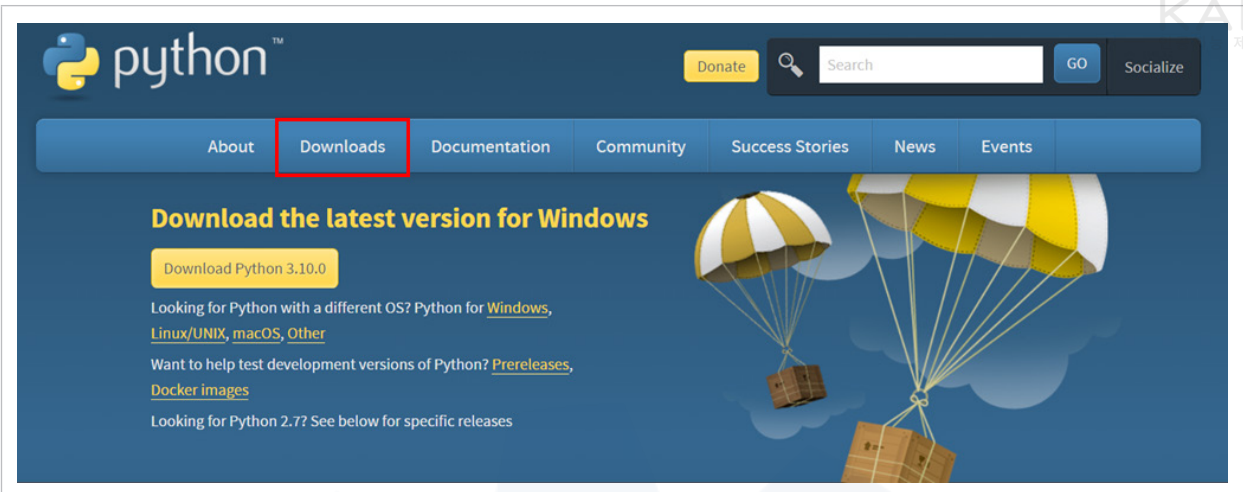
① google.com 등의 검색 엔진에 'python'을 검색한다.



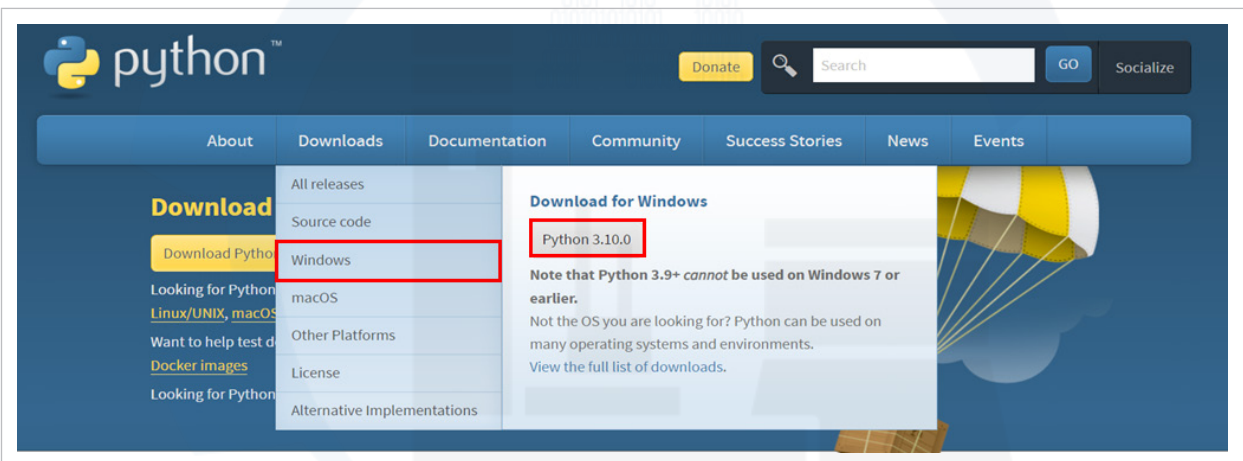
② 제일 처음에 보이는 'Welcome to Python.org'를 클릭한다.



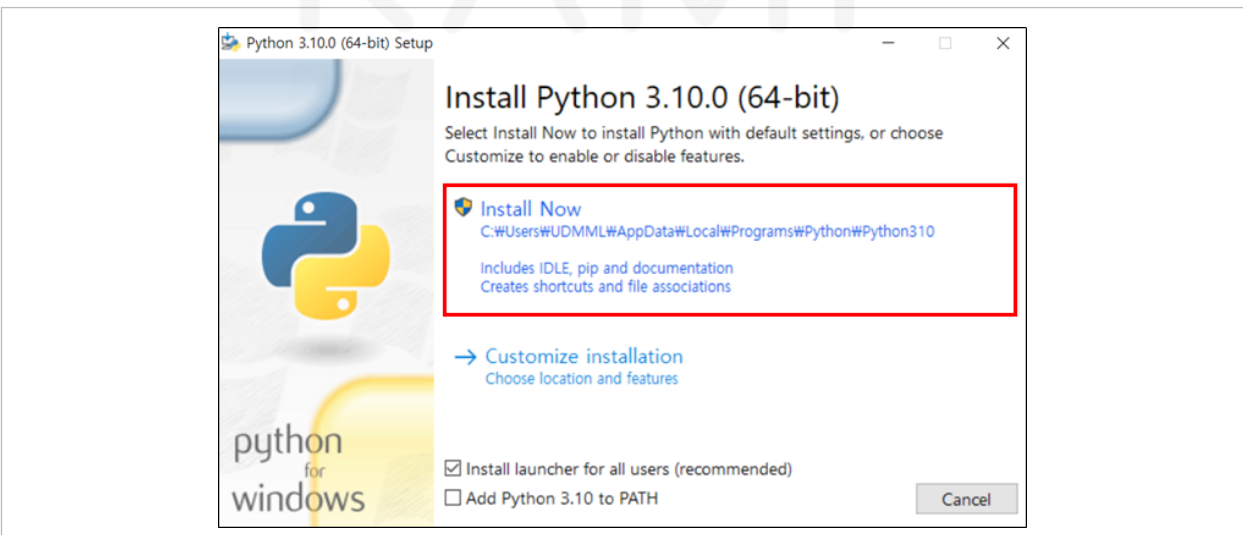
③ 클릭 후 보이는 페이지의 정면에서 상단 좌측 2번째 'Downloads'를 클릭한다.



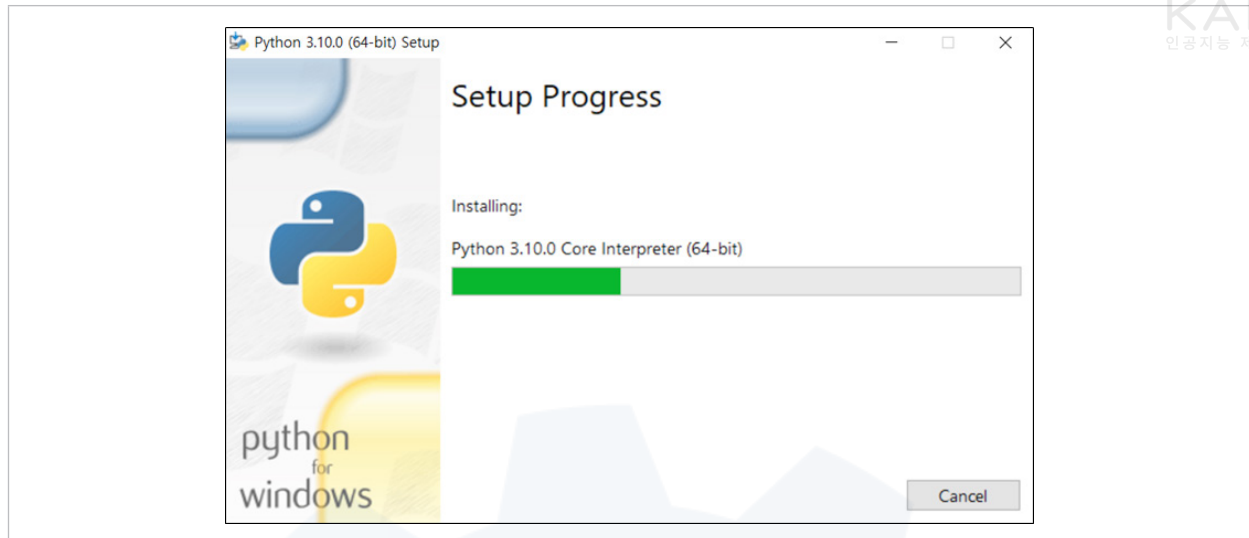
④ 컴퓨터의 운영체제에 따라 상단에서 세번째(macOS의 경우 네번째) 탭을 선택한 후, Python 3.10.0을 다운로드한다. (Python 3.10.0 버전은 업데이트 될 수 있다)



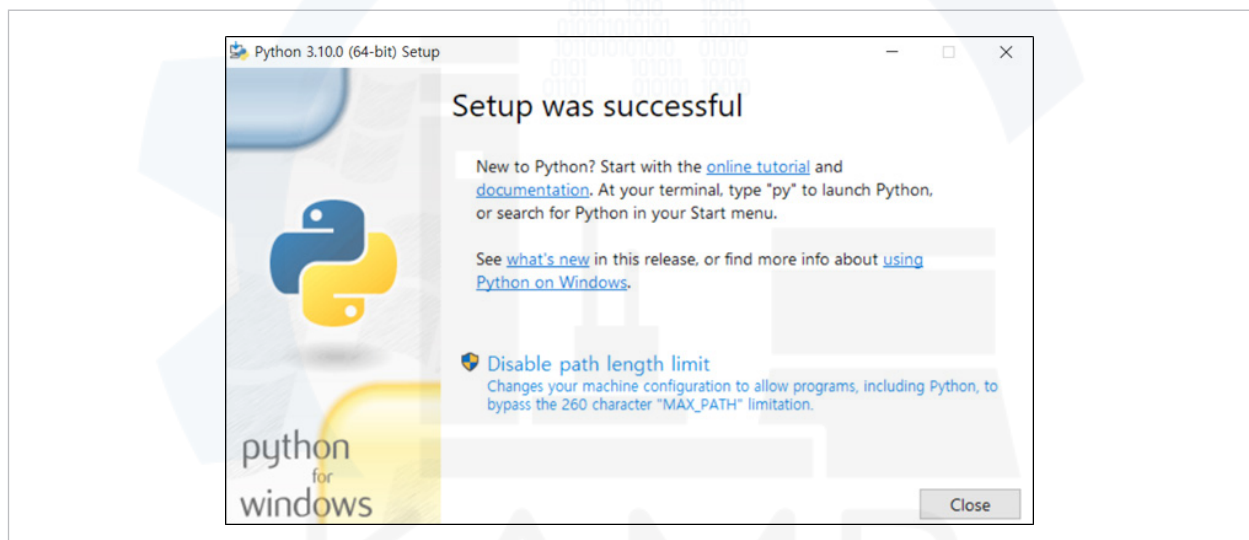
⑤ 아래와 같은 설치창이 뜨면, 'Install Now'를 클릭한다.



⑥ 아래와 같은 설치 진행창이 완료가 될 때까지 유지한다.



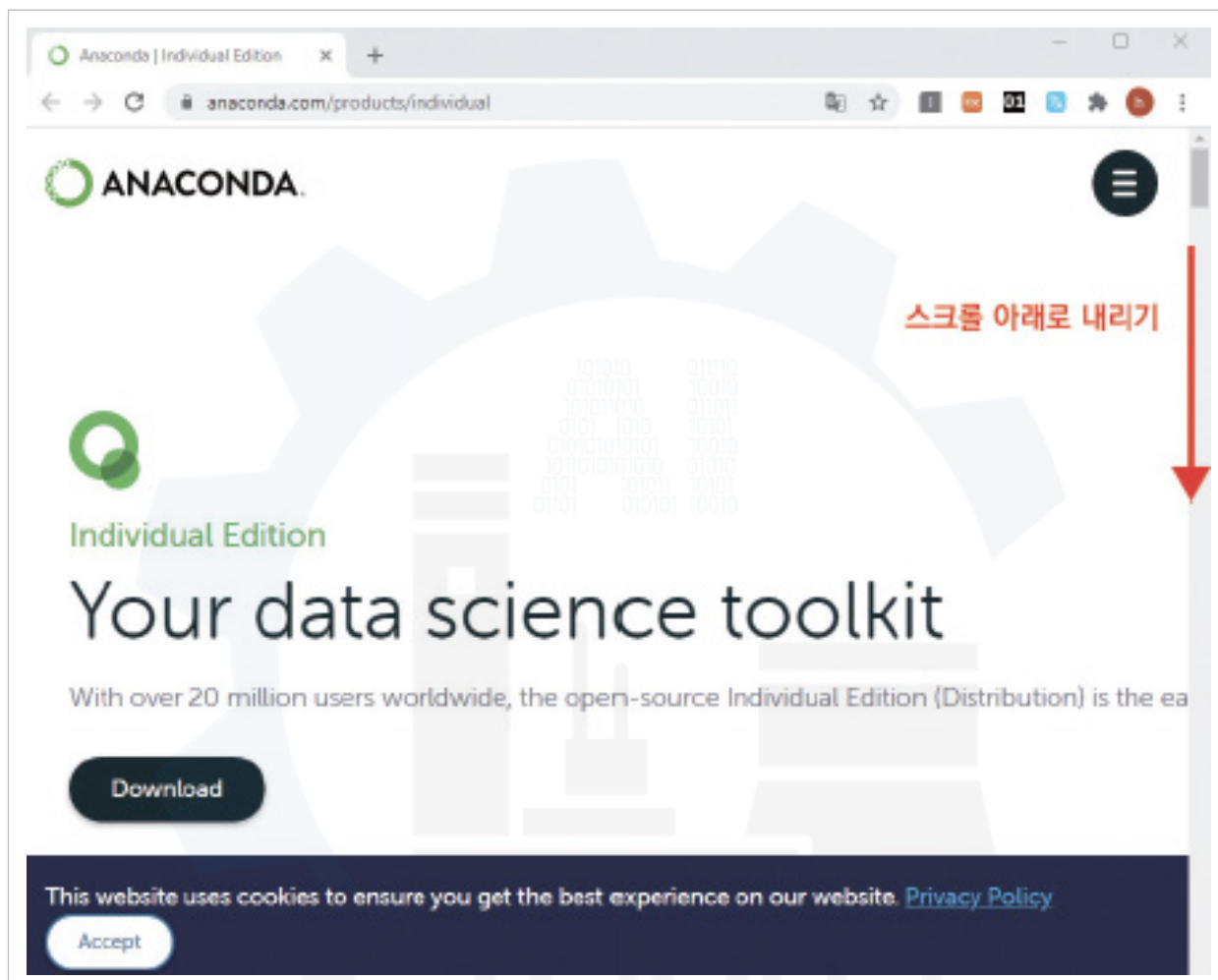
⑦ 완료가 되면 아래와 같은 창이 뜨는 것을 확인 후 종료한다. [설치완료]



2. 아나콘다(Anaconda) 설치

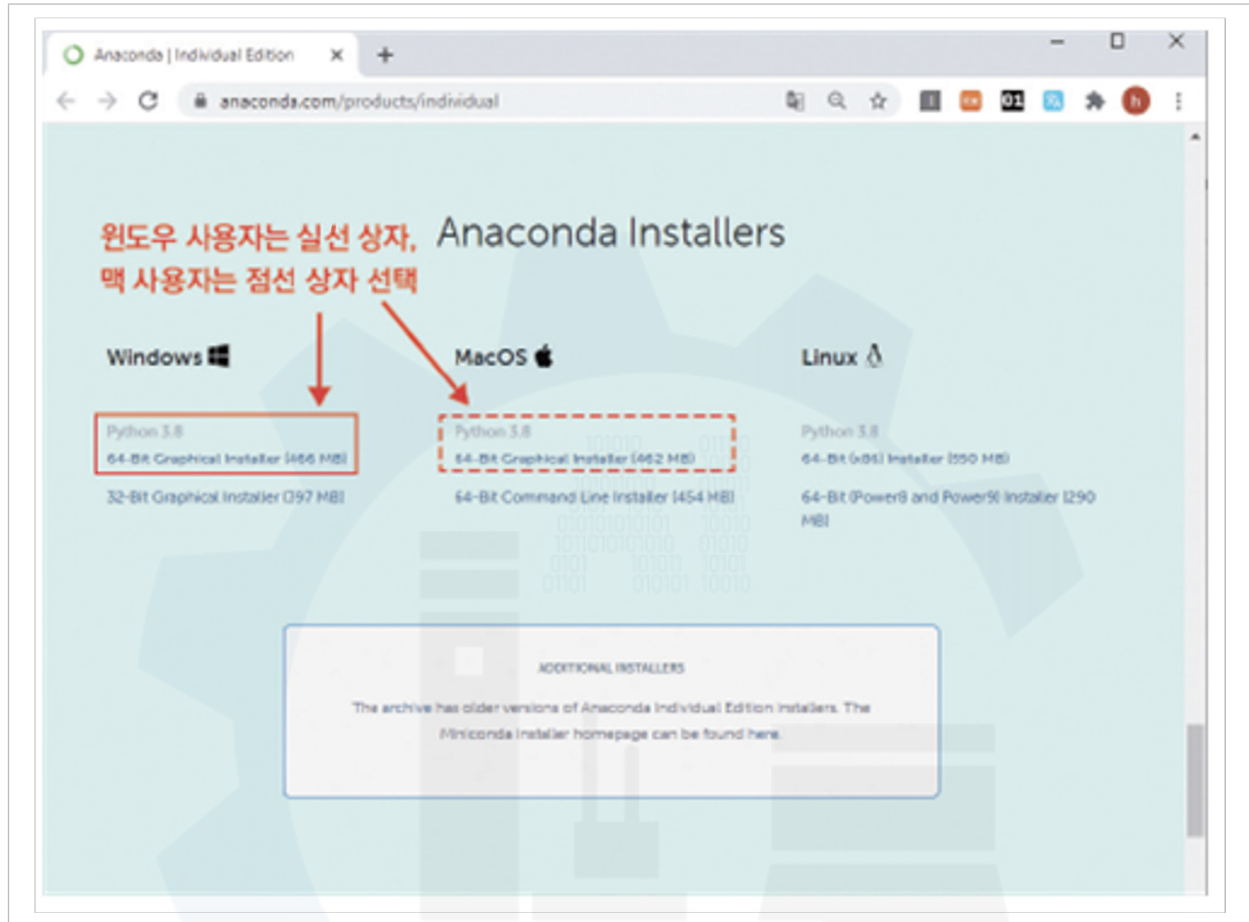
아나콘다란 파이썬과 같은 분석 도구를 사용할 때 필요한 고급 기능 및 분석을 보조하는 도구이다. 아나콘다를 설치함으로써 다양한 기능들을 바로 사용할 수 있고, 결과물을 쉽게 확인할 수 있다. 아나콘다를 설치하고 분석을 실시할 수 있는 환경을 구축하는 방법에 대해 안내한다.

① <https://www.anaconda.com/distribution/> 로 접속한다.



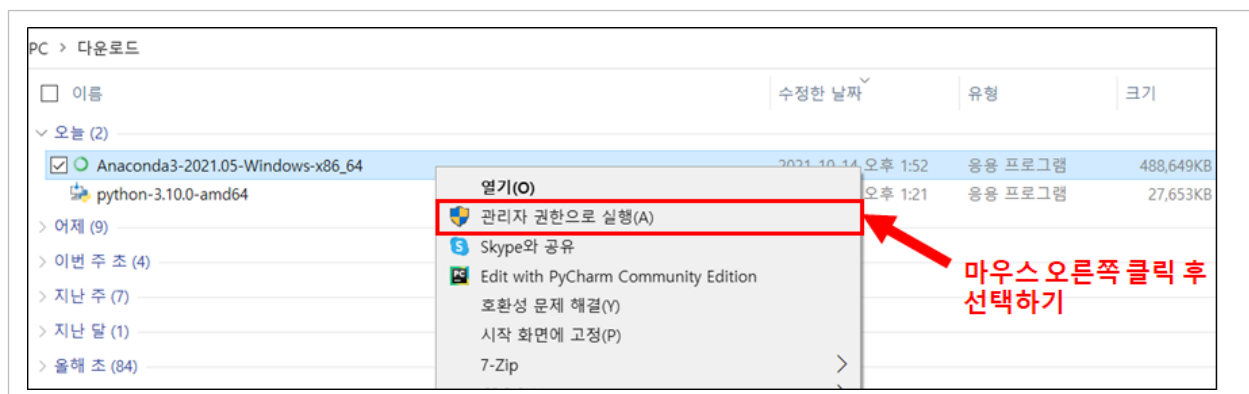
② 다음과 같은 화면이 나올 때 까지 스크롤하여 아래로 내린 후, 로컬 컴퓨터 사용 환경에 맞는 파일을 다운받는다.

- ▶ Windows: 64-Bit Graphical Installer
- ▶ MacOS: 64-Bit Graphical Installer

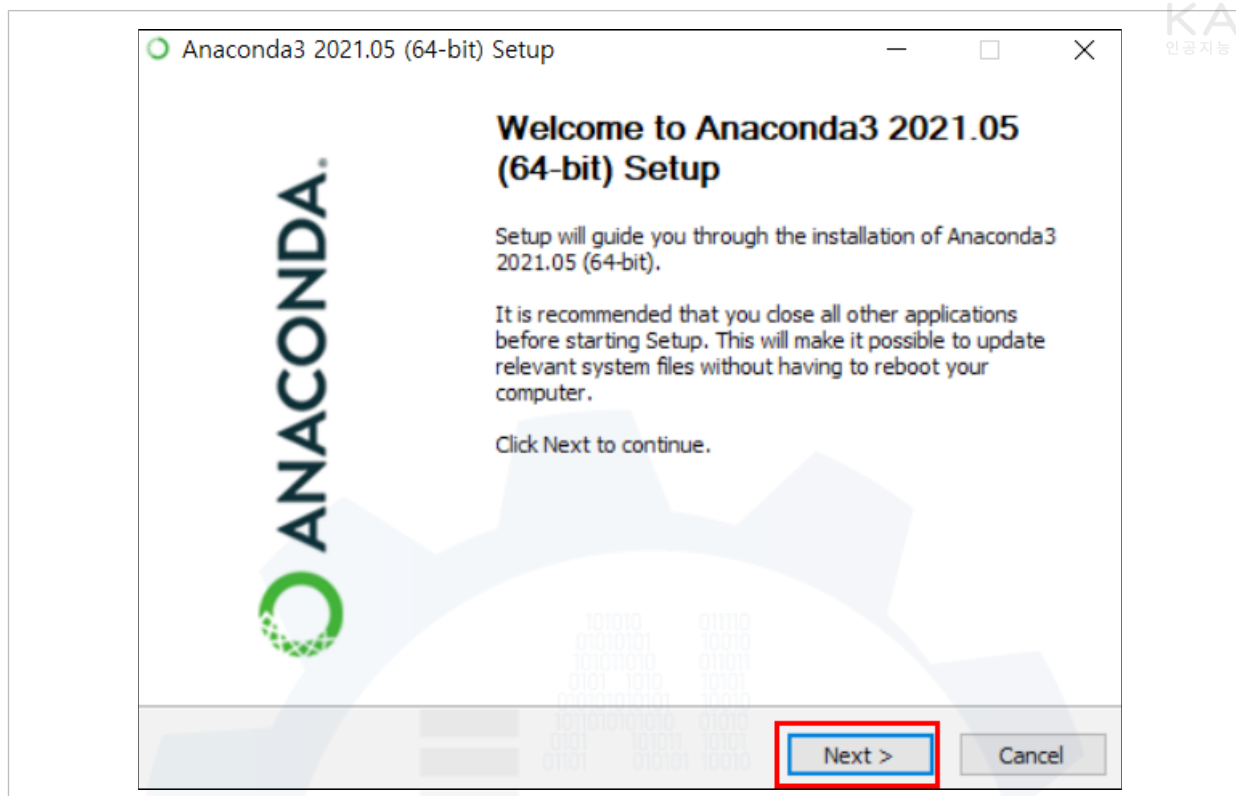


③ 다운로드 받은 파일로 이동하여, 설치된 아나콘다 파일 위에서 마우스 오른쪽을 클릭한 후, 방패모양의 '관리자 권한으로 실행'을 클릭한다.

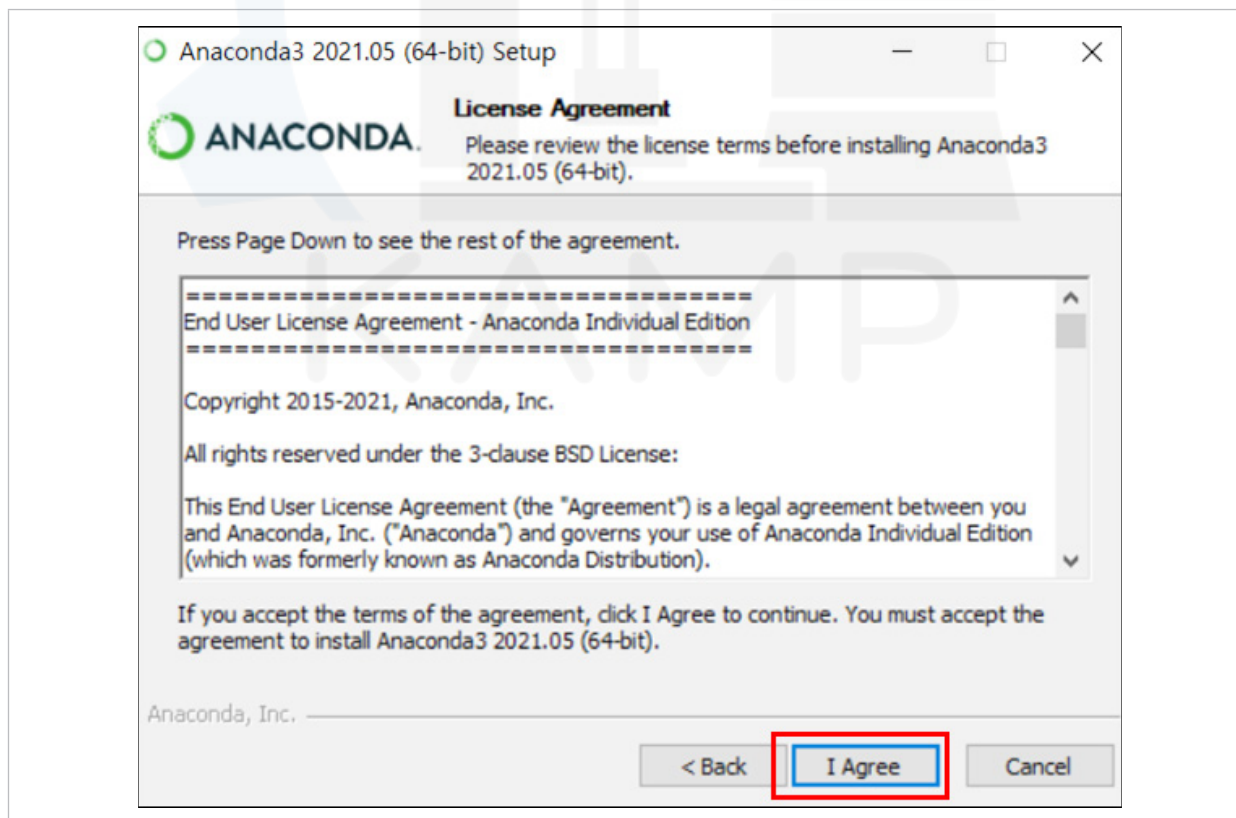
- ▶ (예) '다운로드' 폴더로 아나콘다를 다운 받은 경우



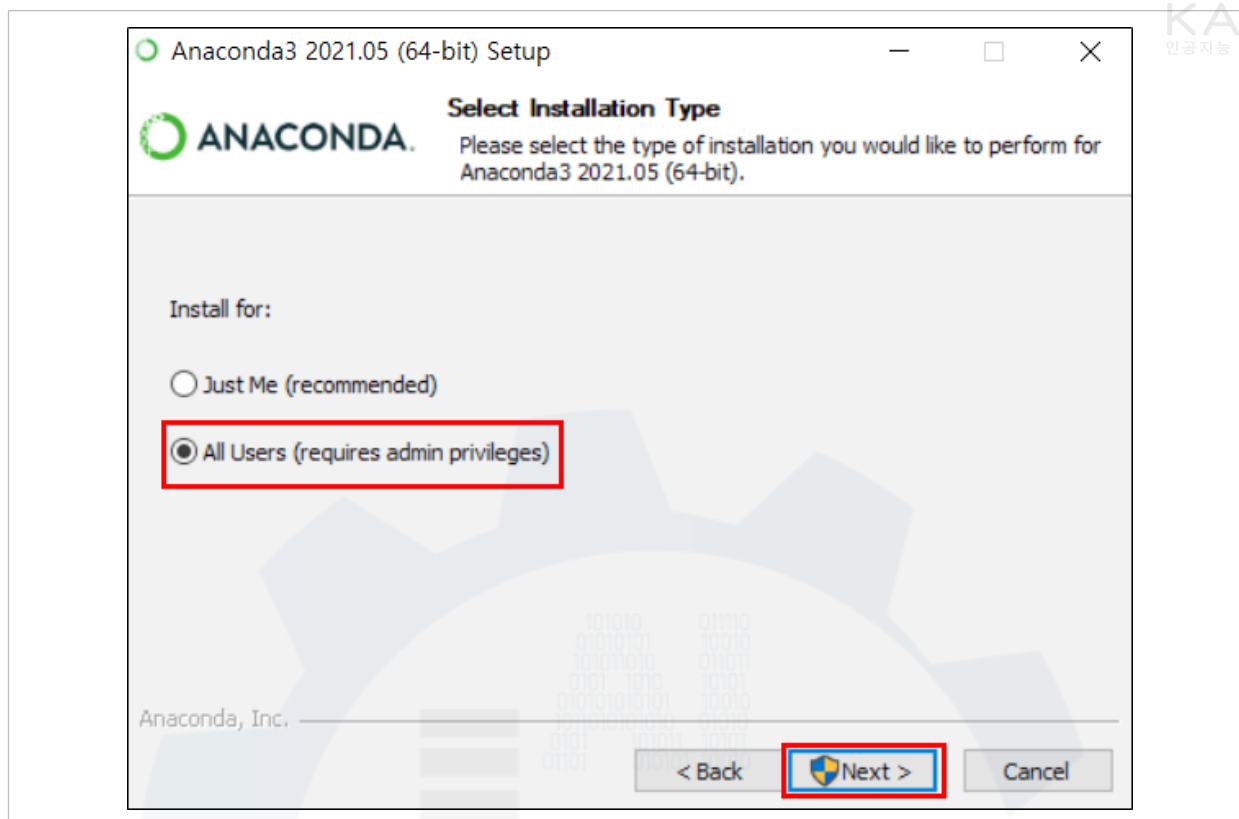
④ 파일을 실행 한 후, 'Next' 버튼을 클릭한다.



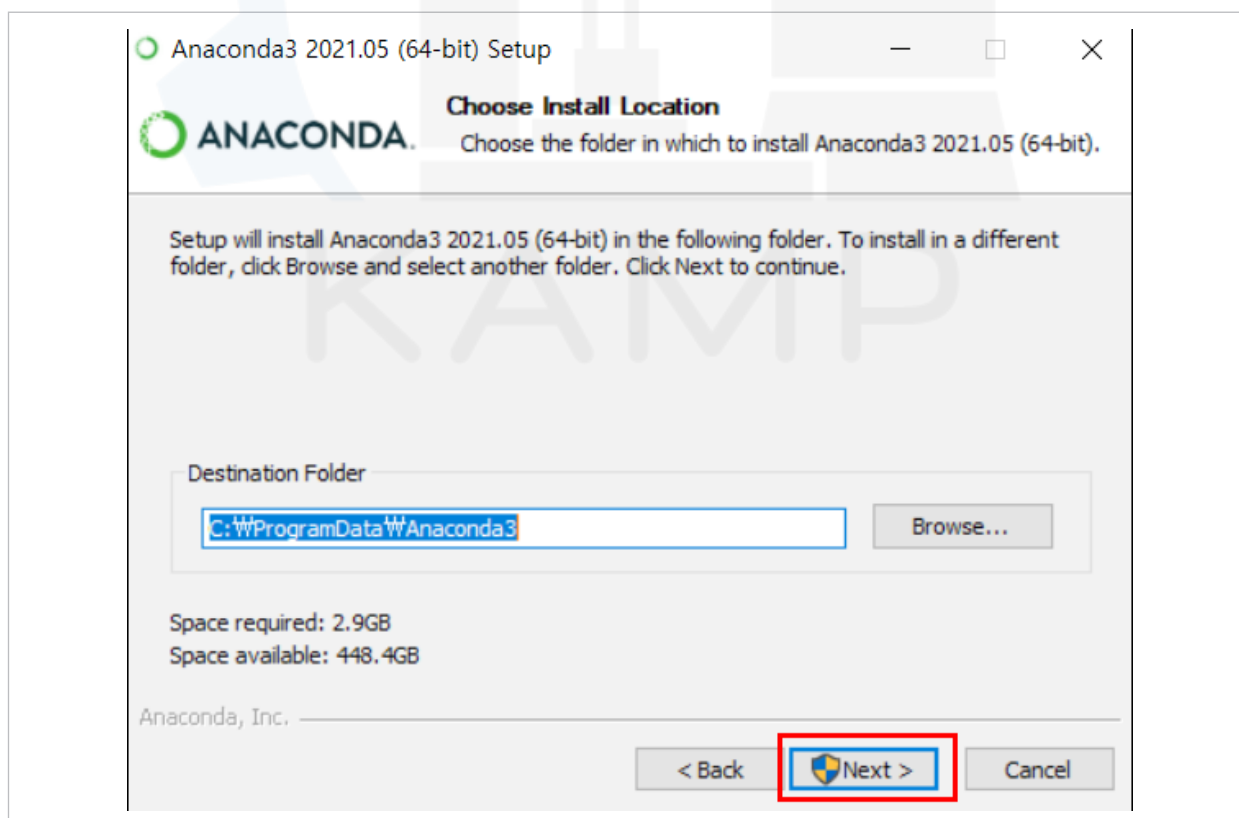
⑤ 다음 창이 나타나면 'I Agree'를 선택한다.



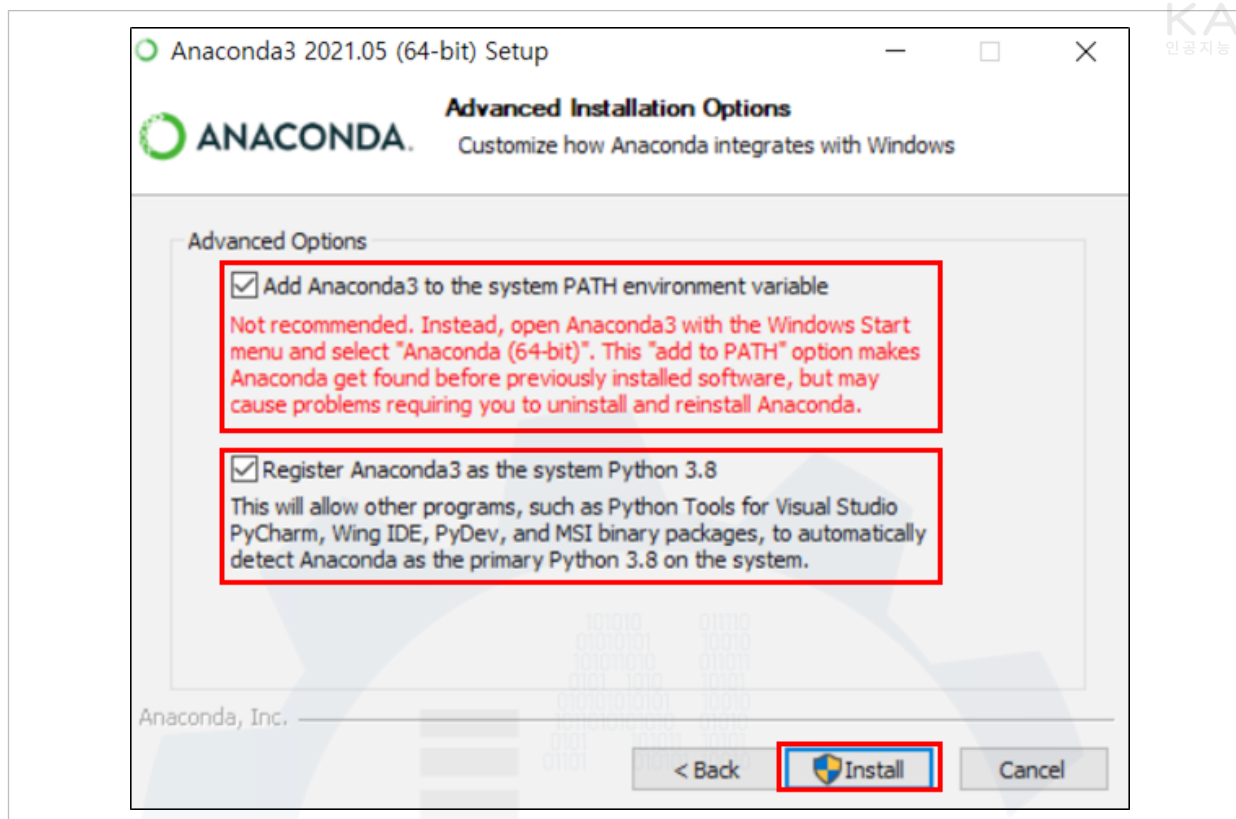
⑥ 셋팅 창이 뜨면 화면의 'All Users'를 선택 후 아래의 'Next'를 클릭한다.



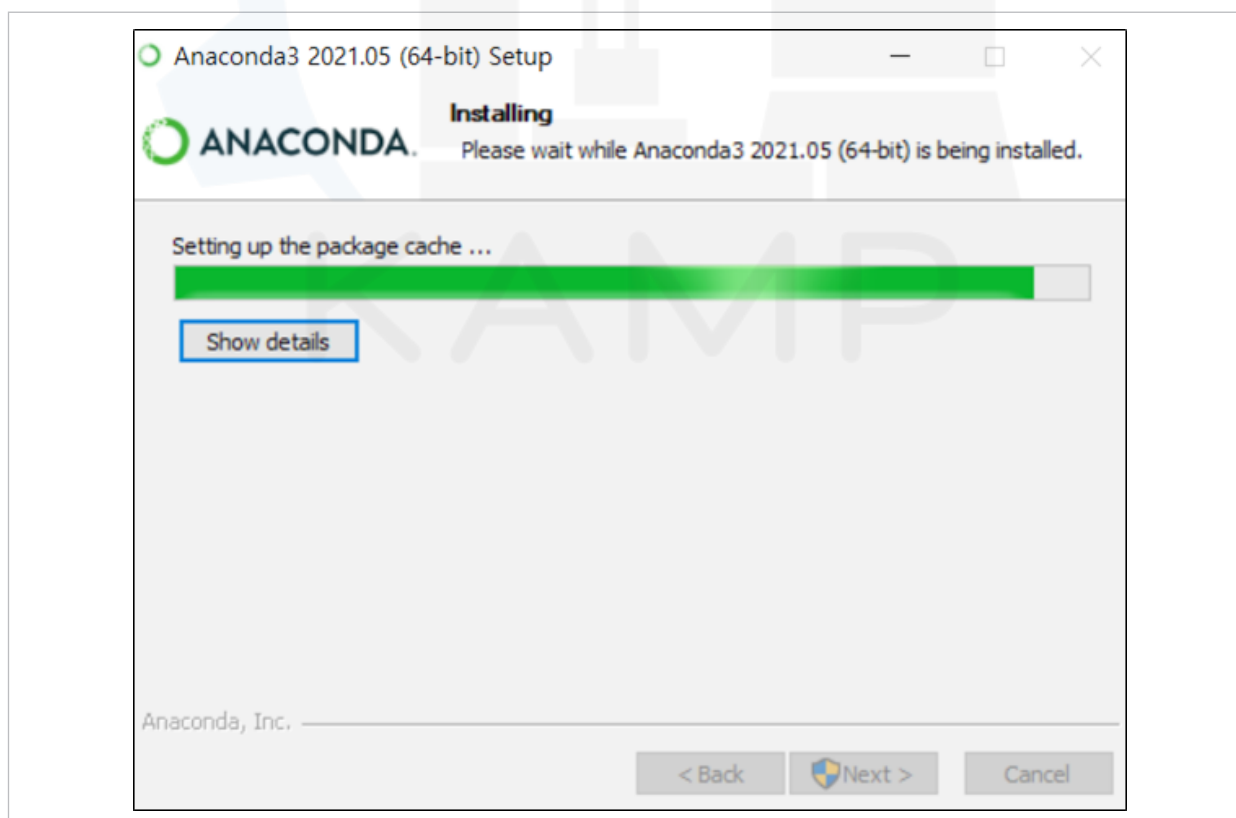
⑦ 다운로드 받을 경로를 물어보는 창이 뜨면, 아래의 'Next'를 클릭한다.



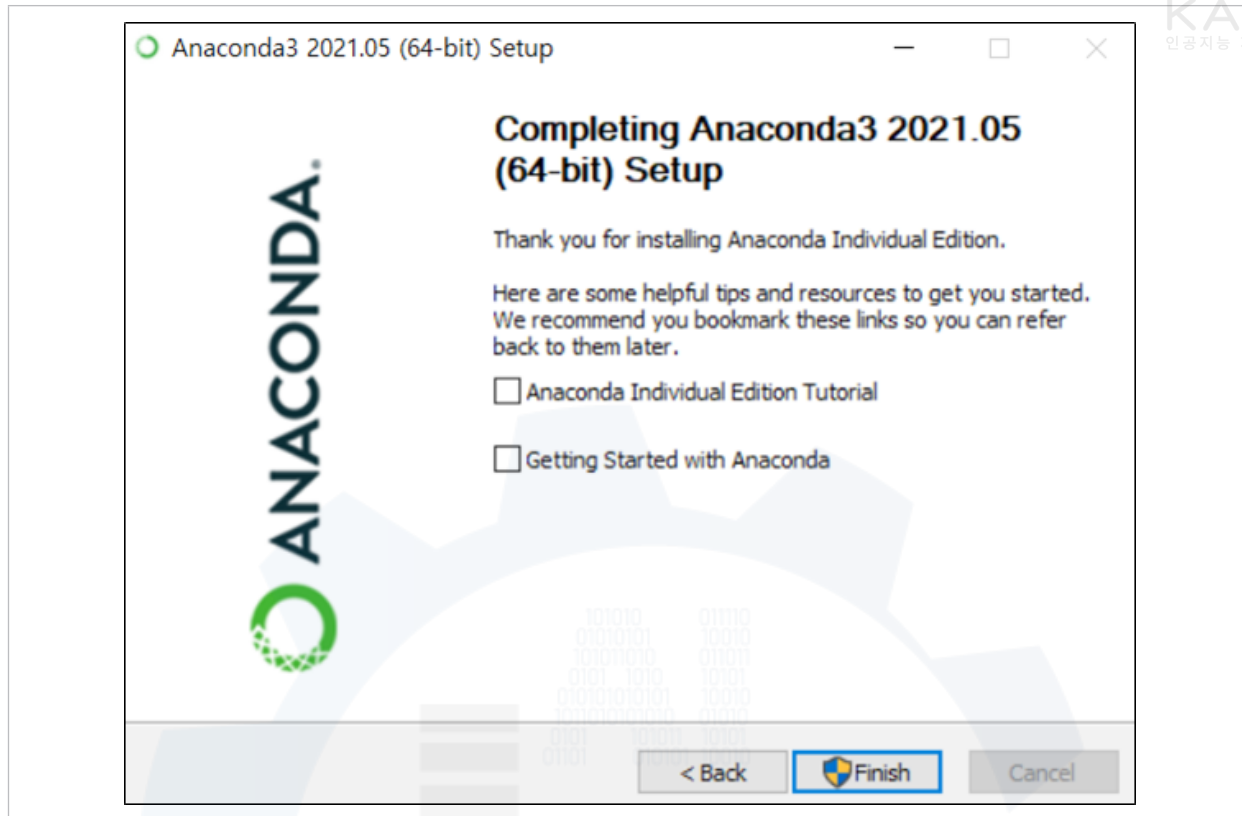
⑧ 고급 옵션 선택창이 뜨면, 아래와 같이 모두 선택 후, 'Install'을 클릭한다.



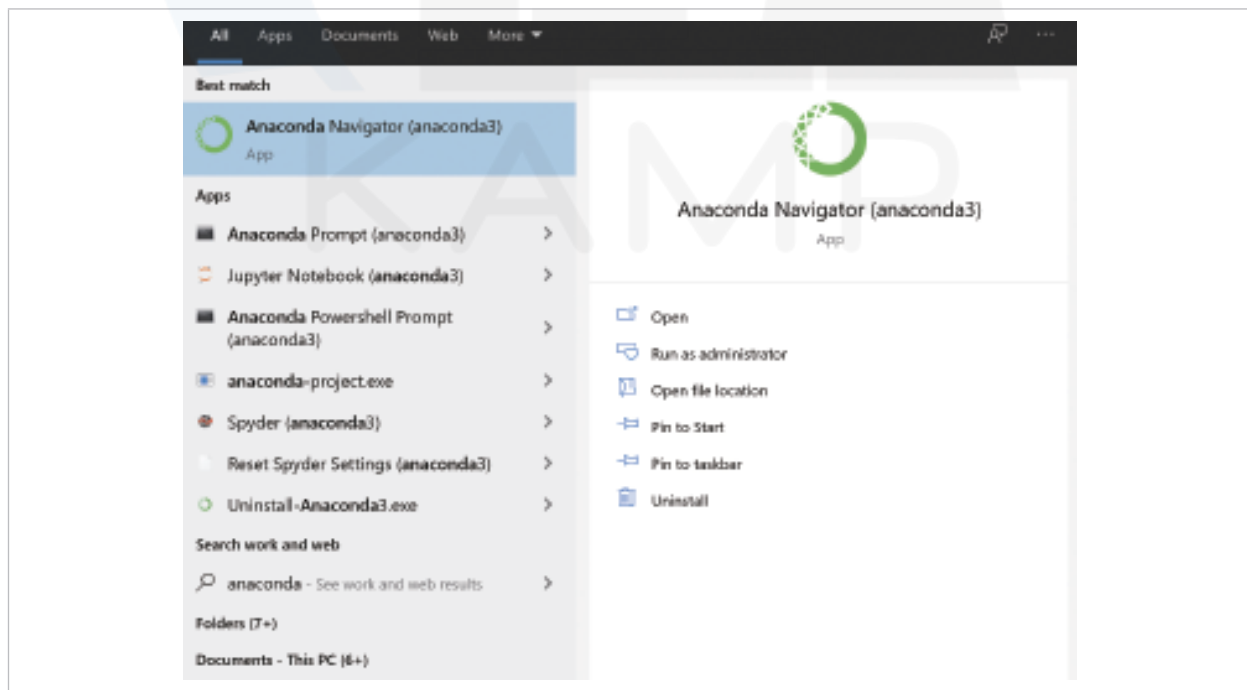
⑨ 다음과 같은 설치창이 뜨면 완료가 될 때까지 대기 (5분이상 소요)



⑩ 마지막 화면에서, 모두 체크 해제한 후, 'Finish'를 눌러 설치를 완료한다.



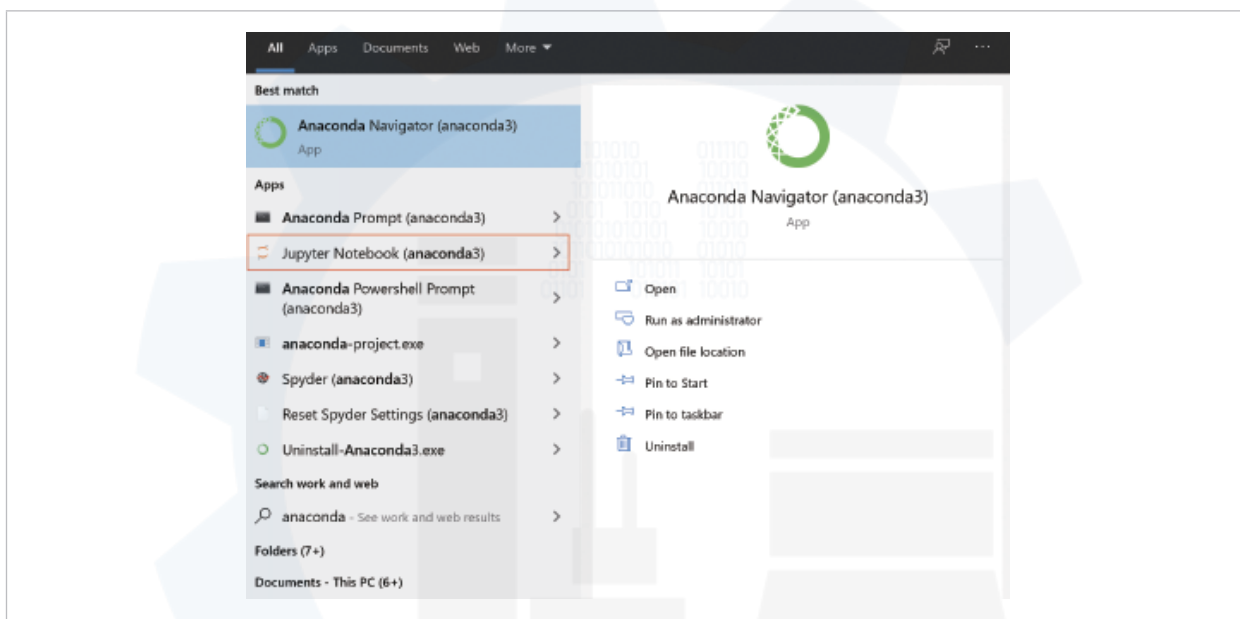
⑪ 화면상의 '홈(Windows)'키를 눌러서 화면과 같이 anaconda prompt가 잘 설치되었는지를 확인한다. Anaconda Navigator, Anaconda Prompt, Jupyter Notebook 등의 다른 응용 프로그램 램들도 함께 확인이 된다면 설치가 완료된 것이다.



3. 주피터 노트북 (Jupyter Notebook) 실행

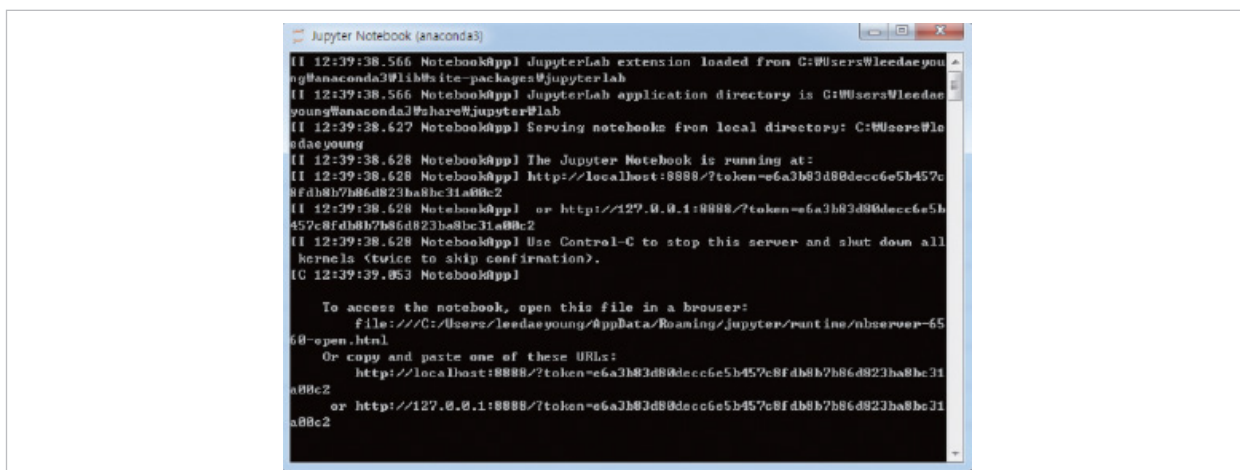
주피터 노트북은 실제로 사용자가 코딩(분석 문장 작성)을 할 수 있는 도구이다. 쉽게 비교하자면, 문서 도구로 마이크로소프트 사의 'Word' 프로그램이나, 한컴소프트 사의 '한글' 프로그램 등과 같은 도구라고 생각할 수 있다. 데이터 분석에 다양한 입력, 실행 도구가 있지만, 본 가이드 북에서는 주피터 노트북을 활용하는 방법을 안내하기로 한다. Anaconda를 설치한 이후 주피터 노트북(Jupyter Notebook) 설치 방법을 확인하면 된다.

- ① 화면상의 '홈()'키를 눌러서 화면과 같이 'anaconda' 검색 및 폴더 리스트 중 'Jupyter Notebook (anaconda3)'을 실행한다.

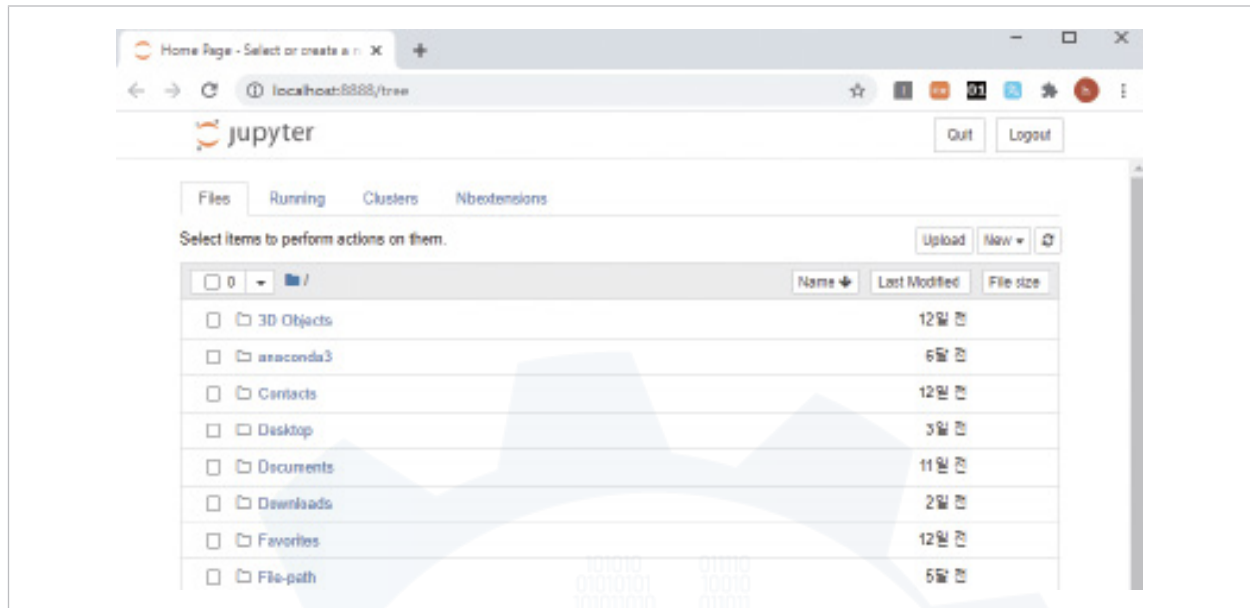


- ② Jupyter Notebook을 클릭시 아래처럼 2개의 윈도우가 실행된다.

- (1) 검은색 배경의 화면은 주피터 노트북이 실행되는 환경에 대한 상태를 나타내주는 상태 표시 창이다. 주피터 노트북을 사용하는 동안 종료하면 안된다.



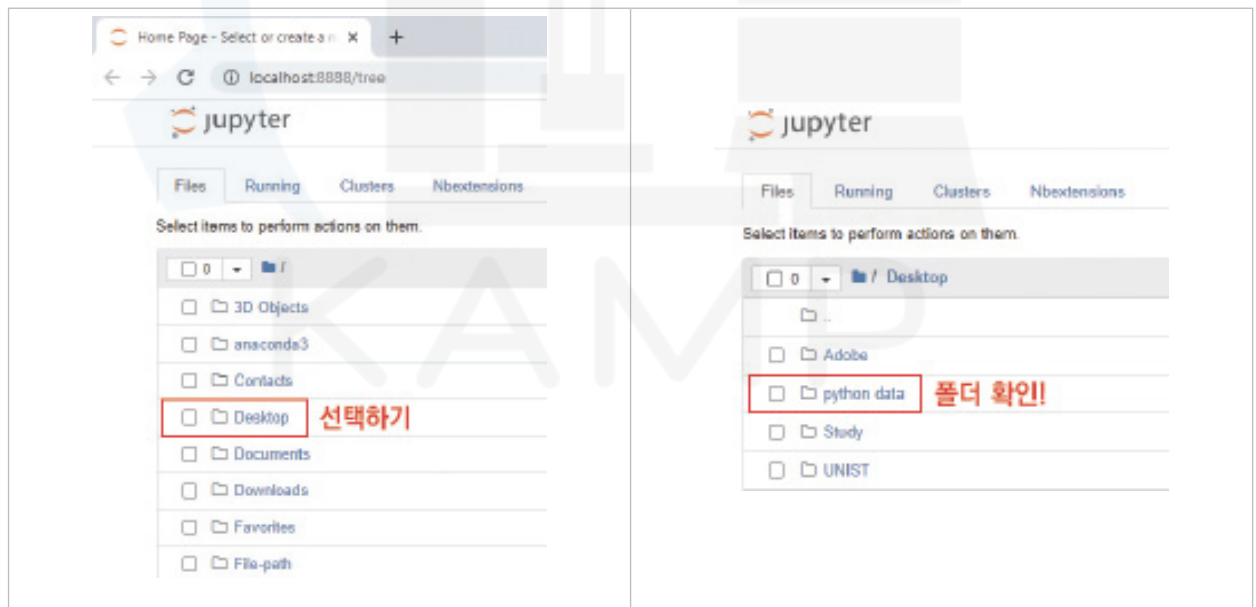
- (2) 사용하는 인터넷 프로그램(크롬, 인터넷 익스플로러)에 주피터 노트북이 열린다. (다른 확장 프로그램 사용하고 싶다면 기본 브라우저를 변경해주어야한다)



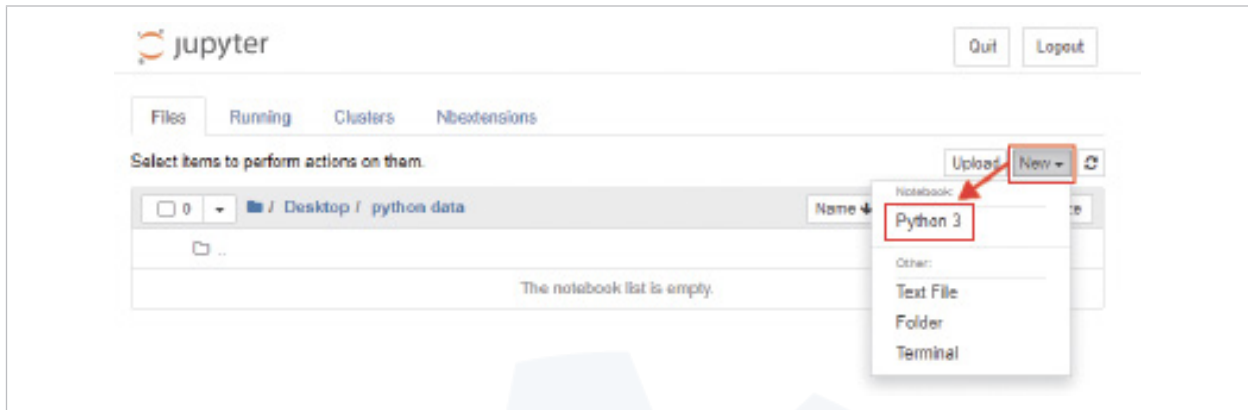
- ③ 데이터를 저장하고, 불러오고, 분석할 경로의 폴더를 하나 생성한다.

▶ (예) '바탕화면'에 'python data' 폴더를 생성 후 (미리 생성), 파이썬 코드 실행하고 저장한다.

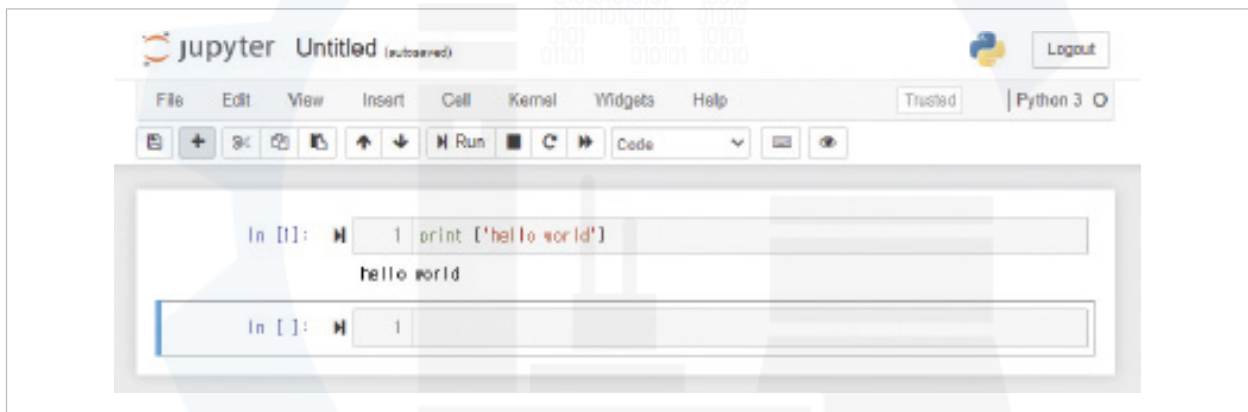
- (1) 주피터 노트북에서 'python data' 폴더를 확인한다.



- (2) python data에서 파이썬 파일을 생성한다. 오른쪽 상단의 'New'를 누른 후, 'Python 3'을 선택한다.



- (3) 'hello world' 출력을 확인해본다. 보이는 In [] 우측 회색 창에 `print('hello world')` 입력 후, 상단의 **Run** 버튼 혹은 shift + Enter 키를 눌러서 실행한다. 코드 파일의 확장자는 '.ipynb'로 저장된다.



4 부 록 _ 데이터 품질 전처리 [실습 코드]

앞선 '2.분석 실습'에서 데이터 품질 전처리에 필요한 6가지 특성에 대하여 논의하였다.
(완전성, 유일성, 유효성, 일관성, 정확성, 무결성)

▶ 완전성 품질 지수 = $(1 - (\text{결측치} / \text{전체 데이터 수})) * 100$

- ① 결측치 값이 30% 이상인 데이터들은 데이터의 완전성이 떨어지기 때문에 변수별 결측치 값의 비율을 확인하여 삭제한다. 본 데이터는 30%를 넘는 변수가 존재하지 않으므로 변수를 제거하지 않는다.

perc=30 df.isna().sum()/len(df)*100		perc=30 df.isna().sum()/len(df)*100>perc	
날짜	0.000000	날짜	False
시간	0.000000	시간	False
15분	0.000000	15분	False
30분	0.000000	30분	False
45분	0.000000	45분	False
60분	0.000000	60분	False
평균	0.000000	평균	False
생산량	0.000000	생산량	False
기온	0.000000	기온	False
풍속	0.048638	풍속	False
습도	0.000000	습도	False
강수량	0.000000	강수량	False
전기요금(계절)	0.000000	전기요금(계절)	False
day	0.000000	day	False
d	0.000000	d	False
m	0.000000	m	False
공장인원	0.275616	공장인원	False
인건비	0.000000	인건비	False
dtype: float64		dtype: bool	

[코드 1] 분석 도구를 활용한 완전성 품질 지수 확인

- ② 데이터의 결측치를 확인하기 위해 'isnull()' 함수를 사용한 뒤 'sum()' 함수를 사용하여 총 결측치 개수를 구한다.

```
cmpt_len = df.isnull().sum().sum()
print("결측치 = %d개 \n완전성 지수 : %.2f%%" % (cmpt_len, (1 - cmpt_len / len(df)) * 100))
```

결측치 = 20개
완전성 지수 : 99.68%

[코드 2] 분석 도구를 활용한 완전성 품질 지수 구하기 1

③ 결측된 데이터들은 실제로 0의 값이므로 0으로 채운 후 그 결과를 확인한다.

```
df = df.fillna(0)
cmpt_len = df.isnull().sum().sum()
print("결측치 = %d개 \n완전성 지수 : %.2f%%"%(cmpt_len,(1-cmpt_len/len(df))*100))
```

```
결측치 = 0개
완전성 지수 : 100.00%
```

[코드 3] 분석 도구를 활용한 완전성 품질 지수 구하기 2

▶ 유일성 품질 지수 = ((유일한 데이터 수)/전체 데이터 수) * 100

① 본 데이터에서는 유일한 데이터 값을 가진 변수는 존재하지 않지만, 날짜와 시간을 결합한 데이터를 생성하여 유일한 데이터셋을 만들 수 있으므로 '날짜'와 '시간'을 합친 새로운 변수를 생성하도록한다.

```
data1 = df
data1['날짜시간'] = (df['날짜']*100)+(df['시간'])
```

[코드 4] 유일한 값을 가지는 새로운 변수 생성하기

② 'groupby()' 함수를 이용하여 '날짜시간' 열이 가지는 유일한 값들을 확인하고, 유일한 데이터 개수를 구한다.

```
data1.groupby(['날짜시간']).size()
```

```
날짜시간
2021010100    1
2021010101    1
2021010102    1
2021010103    1
2021010104    1
..
2021091419    1
2021091420    1
2021091421    1
2021091422    1
2021091423    1
Length: 6149, dtype: int64
```

[코드 5] 분석 도구를 활용한 유일한 값들 확인하기

③ 구한 유일한 데이터 개수를 이용하여 유일성 품질지수를 구한다.

```
uniq_len=len(df.groupby(["날짜시간"]).size())
print("유일성 지수 : %.2f%%"%(uniq_len/len(df)*100))
```

유일성 지수 : 99.69%

[코드 6] 분석 도구를 활용한 유일성 품질지수 구하기

▶ 유효성 품질 지수 = (유효성 만족 데이터 수/전체 데이터 수) * 100

① 데이터가 유효 범위 내에 있는가?

- 데이터셋에 정의된 수집 범위를 이용하여 조건(유효 범위)을 모두 충족하는지 확인한다.
- 다음의 코드는 min 값과 max 값을 확인할 수 있는데, 이 값을 충족하는지 확인한 후 이 범위를 위반하는 데이터가 존재할 시에 경고문을 출력하고 값을 확인한다.

```
nonval_count=0
for col in df.columns:
    print(col+"의 min 값 :",df[col].min(),col+"의 max 값 :",df[col].max())
    col_min = df[col].min()
    col_max = df[col].max()
    tmp_chk=False
    for dc in df[col]:
        tmp_chk = (col_min <=dc)
        if not tmp_chk:
            print("non validation data detected",dc)
            nonval_count+=1
            break
val_rate_1 = (1-nonval_count/len(df))
```

```
날짜의 min 값 : 20210101 날짜의 max 값 : 20210914
시간의 min 값 : 0 시간의 max 값 : 188
15분의 min 값 : 0 15분의 max 값 : 207
30분의 min 값 : 0 30분의 max 값 : 222
45분의 min 값 : 0 45분의 max 값 : 218
60분의 min 값 : 0 60분의 max 값 : 214
평균의 min 값 : 0 평균의 max 값 : 208
생산량의 min 값 : 0 생산량의 max 값 : 9830
기온의 min 값 : -12.0 기온의 max 값 : 33.4
풍속의 min 값 : 0.0 풍속의 max 값 : 7.6
습도의 min 값 : 8 습도의 max 값 : 98
강수량의 min 값 : 0.0 강수량의 max 값 : 122.4
전기요금(계절)의 min 값 : 109.8 전기요금(계절)의 max 값 : 191.6
day의 min 값 : 1 day의 max 값 : 7
d의 min 값 : 1 d의 max 값 : 31
m의 min 값 : 1 m의 max 값 : 9
공장인원의 min 값 : 0.0 공장인원의 max 값 : 48.38636364
인건비의 min 값 : 1.0 인건비의 max 값 : 1.5
```

[코드 7] 분석 도구를 활용한 유효성 품질지수 구하기

- 모든 데이터가 유효 범위 내에 있는 것을 알 수 있다.

② 데이터가 형식에 맞는가?

- 데이터의 형식을 확인한다.

df.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6168 entries, 0 to 6167
Data columns (total 18 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   날짜        6168 non-null   int64
1   시간        6168 non-null   int64
2   15분        6168 non-null   int64
3   30분        6168 non-null   int64
4   45분        6168 non-null   int64
5   60분        6168 non-null   int64
6   평균        6168 non-null   int64
7   생산량      6168 non-null   int64
8   기온        6168 non-null   float64
9   풍속        6165 non-null   float64
10  습도        6168 non-null   int64
11  강수량      6168 non-null   float64
12  전기요금(계절) 6168 non-null   float64
13  day         6168 non-null   int64
14  d           6168 non-null   int64
15  m           6168 non-null   int64
16  공장인원    6151 non-null   float64
17  인건비      6168 non-null   float64
dtypes: float64(6), int64(12)
memory usage: 867.5 KB
```

[코드 8] 데이터 형식 확인하기

- 다음의 코드를 실행하면서 for문을 순회한다. 위에서 확인한 데이터 타입을 참고하여 유효성 품질 지수를 확인한다.

```
nonval_count = 0
for col in df.columns:
    tmp_chk = False
    col_c = ""
    for i, dc in enumerate(df[col]):
        if i == 0:
            if 'int' in str(type(dc)):
                col_c = 'int'
            elif 'float' in str(type(dc)):
                col_c = 'float'
            tmp_chk = (col_c in str(type(dc)))
        if not tmp_chk:
            nonval_count += 1
    val_rate_2 = (1 - nonval_count / len(df))
    print((val_rate_1 + val_rate_2) / 2 * 100, "%")
```

100.0 %

[코드 9] 분석 도구를 활용한 데이터 형식 확인하기

- 모든 데이터가 유효한 형식인 것을 알 수 있다.

▶ 일관성 품질 지수 = (일관성 만족 데이터 수/ 전체 데이터 수) * 100

- 본 데이터가 구조, 값, 형태를 일관성 있게 지키고 있는지 확인한다. 'info()' 함수를 사용하여 각각의 데이터 타입을 확인 하였지만, 'dtypes()' 함수를 이용하여 이외의 전체 데이터 값들의 형태가 int64, float64인지 확인한다.

```
print(df.dtypes)
nonval_count = 0
for col in df.columns:
    tmp_chk = False
    col_c = ""
    for i, dc in enumerate(df[col]):
        if i == 0:
            if 'int' in str(type(dc)):
                col_c = 'int'
            elif 'float' in str(type(dc)):
                col_c = 'float'
            tmp_chk = (col_c in str(type(dc)))
        if not tmp_chk:
            nonval_count += 1
val_rate_2 = (1 - nonval_count / len(df))
print((val_rate_1 + val_rate_2) / 2 * 100, "%")
```

```
날짜      int64
시간      int64
15분      int64
30분      int64
45분      int64
60분      int64
평균      int64
생산량    int64
기온      float64
풍속      float64
습도      int64
강수량    float64
전기요금(계절) float64
day       int64
d         int64
m         int64
공장인원  float64
인건비    float64
날짜시간  int64
dtype: object
100.0 %
```

[코드 10] 분석 도구를 활용하여 데이터 일관성 확인하기

- 본 데이터는 일관성이 있다고 할 수 있다.

▶ 정확성 품질 지수 = (1 - (정확성 위배 데이터 수/전체 데이터 수)) * 100

- 해당 최대수요전력 예측과 관련하여 중복되는 예측값은 존재할 수 있으나 누락이 발생해서는 안된다. 본 데이터에서는 15분, 30분, 45분, 60분에 대한 누락이 있으면 안된다.

```
data1 = df[['15분','30분','45분','60분']]
data1['sum']=data1.mean(axis=0)
(data1['sum']-df['평균']).sum()
```

0.0

[코드 11] 분석 도구를 활용한 정확성 품질 지수 확인하기

- 합해야 할 컬럼들을 data1에 저장을 한 뒤, mean(axis=0)을 이용하여 각 행의 평균을 구하고, 본 데이터의 '평균'과의 차이를 구한다. 값의 차이의 합은 0으로, 정확성을 위배하는 데이터는 없다고 할 수 있다.

▶ 무결성 품질 지수 = $(1 - (\text{유일성, 유효성, 일관성 지수 중 100\%가 아닌 지수 개수} / 3)) * 100$

- 본 데이터에서 유일성은 99.69%, 유일성과 일관성은 100%로, 세 가지 지수 중 유일성 지수가 100%를 만족하지 못하므로 무결성 품질 지수는 66.67%이다.

▶ 가중치지수 = 품질 지수 * 가중치

- 데이터의 특성에 따라 품질 지수의 중요도는 달라진다. 활용되는 데이터의 중요한 품질 지수가 무엇인지를 평가하고, 데이터의 특성에 따라 적절한 가중치를 부여한다.

▶ 데이터 품질 지수 = $\sum$ 가중치지수

- 최종적으로 구한 6가지 데이터 품질 지수에 적절한 가중치를 부여하여 품질 지수를 구한다.

구분	품질 지수	가중치	가중치지수	오류율
완전성	100%	20%	20%	0%
유일성	99.69%	10%	9.97%	0.31%
유효성	100%	20%	20%	0%
일관성	100%	20%	20%	0%
정확성	100%	20%	20%	0%
무결성	66.67%	10%	6.67%	33.33%
품질 지수	96.64%	100%	96.63%	3.36%(3.37%)

※ 제출 시 유의사항

- 1) 제안서와 발표자료는 반드시 각각 1개의 단일파일(PDF)이어야 합니다. (용량이 크더라도 파일분할 금지)
- 2) 전체적으로 시각자료를 최대한 많이 활용하여 초보자(중소 제조기업/대학생)도 이해할 수 있는 가이드북 제작을 요망합니다.
- 3) 본 실습에 활용되는 데이터셋이 기업에 민감한 핵심 레시피 데이터인 경우, 꼭 비식별화하여야 합니다. 이에 대한 법적 책임은 AI 데이터셋 과제 수행기관에 있음을 명확히 합니다.

「자원 최적화 AI 데이터셋」 분석실습 가이드북

